



FRONTEND WEBOVÉ APLIKACE NA UČENÍ SE PROGRAMOVÁNÍ

Bc. Jan Jeníček

Diplomová práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Softwarové inženýrství
Vedoucí: Ing. Jiří Hunka
18. dubna 2026



Zadání diplomové práce

Název:	Frontend webové aplikace na učení se programování
Student:	Bc. Jan Jeníček
Vedoucí:	Ing. Jiří Hunka
Studijní program:	Informatika
Obor / specializace:	Softwarové inženýrství
Katedra:	Katedra softwarového inženýrství
Platnost zadání:	do konce letního semestru 2026/2027

Pokyny pro vypracování

Cílem práce je ve spolupráci s Bc. Jakubem Vondráčkem realizovat vzdělávací webovou aplikaci určenou pro samouky, kteří se chtějí naučit základy programování. Práce vychází z již existující aplikace pro výuku angličtiny, která bude upravena a rozšířena do podoby vzdělávací aplikace zaměřené na výuku programování. Z důvodu velkého rozsahu řešení je předmětem této konkrétní práce frontendová část aplikace.

Postupujte v těchto krocích:

1. Provedte analýzu konkurenčních řešení se zaměřením na frontendovou část aplikace. Dále provedte analýzu existující aplikace pro výuku angličtiny, ve které se zaměřte na stávající funkce a potřebné změny. Následně analyzujte funkční a nefunkční požadavky s důrazem na frontendovou část aplikace.
2. Na základě provedené analýzy navrhnete nové funkce a zdokonalení již existujících funkcí. Dále navrhnete uživatelské rozhraní aplikace.
3. Dle navrženého řešení provedte úpravy frontendu stávající aplikace a implementujte nové funkce.
4. Navrhnete a realizujete vhodné formy testů frontendové části aplikace.
5. Nasadíte svou část řešení do produkčního prostředí.
6. Zhodnotíte výsledné řešení a navrhnete možnosti dalšího rozvoje do budoucna.

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2026 Bc. Jan Jeníček. Všechna práva vyhrazena.

Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Jeníček Jan. *Frontend webové aplikace na učení se programování*. Diplomová práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

Chtěl bych poděkovat především sit amet, consectetur adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Prohlášení

FILL IN ACCORDING TO THE INSTRUCTIONS. VYPLŇTE V SOULADU S POKYNY. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue. Donec ipsum massa, ullamcorper in, auctor et, scelerisque sed, est. In sem justo, commodo ut, suscipit at, pharetra vitae, orci. Pellentesque pretium lectus id turpis.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue. Donec ipsum massa, ullamcorper in, auctor et, scelerisque sed, est. In sem justo, commodo ut, suscipit at, pharetra vitae, orci. Pellentesque pretium lectus id turpis.

V Praze dne 18. dubna 2026

Abstrakt

Fill in the abstract of this thesis in Czech. Lorem ipsum dolor sit amet. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Klíčová slova enter, comma, separated, list, of, keywords, in, CZECH

Abstract

Fill in the abstract of this thesis in English. Lorem ipsum dolor sit amet. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Keywords enter, comma, separated, list, of, keywords, in, ENGLISH

Obsah

Úvod	1
1 Analýza	3
1.1 Analýza stávající aplikace	3
1.1.1 Analýza kódu	4
1.1.2 Analýza testů	5
1.1.3 Analýza existujících funkcí	6
1.1.4 Závěr analýzy	6
1.2 Analýza existujících řešení	6
1.2.1 Mimo	7
1.2.2 Codecademy	11
1.2.3 Sololearn	15
1.2.4 Scrimba	18
1.2.5 Závěr analýzy	21
1.3 Analýza požadavků	24
1.3.1 Funkční požadavky	24
1.3.2 Nefunkční požadavky	40
2 Návrh	43
2.1 Návrh funkcí	43
2.1.1 Popis aktérů	44
2.1.2 Specifikace případů užití	44
2.1.3 Specifikace funkcí pomocí jazyka Gherkin	67
2.2 Návrh vzhledu uživatelského rozhraní	69
2.2.1 Principy návrhu	69
2.2.2 Výběr palety barev	70
2.2.3 Typografie	76
2.2.4 Ikony a ilustrace	77
2.2.5 Animace a zvukové efekty	78
2.3 Návrh prototypu uživatelského rozhraní	78
2.3.1 Tvorba prototypu	79
2.3.2 Responsivita	80
2.3.3 Testování prototypu	82

3 Implementace a nasazení	83
3.1 Postup při realizaci projektu	83
3.1.1 Příprava implementace	84
3.1.2 Postup při implementaci	87
3.2 Implementace funkcí	89
3.2.1 Nová struktura učení	89
3.2.2 Registrace a přihlášení pomocí účtu Google	94
3.2.3 Přizpůsobení aplikace pro PWA	94
3.3 Nasazení aplikace	95
3.3.1 Vývojové prostředí <code>dev</code>	95
3.3.2 Testovací prostředí <code>stage</code>	96
3.3.3 Produkční prostředí <code>production</code>	96
3.3.4 Nasazení knihoven <code>@eduriam/ui-core</code> a <code>@eduriam/ui-x</code>	96
4 Testování	97
4.1 Volba vhodných forem testování	97
4.2 Manuální testování	98
4.3 End-to-end testování s mock API	99
4.4 Uživatelské testování	100
4.4.1 Příprava testování	100
4.4.2 Úvodní dotazník	101
4.4.3 Testovací scénáře	101
4.4.4 Dotazník po testování	104
4.4.5 Průběh testování	104
4.4.6 Výsledky testování	105
4.5 Zhodnocení řešení a návrh úprav do budoucna	105
4.5.1 Zhodnocení řešení	105
4.5.2 Návrh úprav do budoucna	105
Závěr	106
A Nějaká příloha	107
Obsah příloh	112

Seznam obrázků

2.1	Volba primární barvy	71
2.2	Volba barev pro indikaci stavů	74
2.3	Volba barev pozadí a povrchů	75
2.4	Ukázka návrhu hlavní stránky aplikace	80
2.5	Ukázka návrhu cvičení programování	80
2.6	Ukázka návrhu žebříčku uživatelů	81
2.7	Ukázka návrhu uživatelského profilu	81

Seznam tabulek

Seznam výpisů kódu

1	Ukázka specifikace funkce pomocí jazyka Gherkin	69
---	---	----

Seznam zkratek

WCAG 2.2	Web Content Accessibility Guidelines 2.2
TDD	Test-Driven Development
BDD	Behavior-Driven Development
API	Application Programming Interface
UI	User Interface
XP	Experience Points
AI	Artificial Intelligence
IDE	Integrated Development Environment
CLI	Command Line Interface
PWA	Progressive Web Application
CI/CD	Continuous Integration/Continuous Delivery

Úvod

V posledních letech dochází k výrazným změnám v oblasti vzdělávání, a to zejména díky rozvoji nových technologií a způsobů učení. Stále větší popularitu získává tzv. microlearning, tedy učení prostřednictvím krátkých a efektivních lekcí. Ten však zatím není ve většině oborů dostatečně rozšířen. S nástupem umělé inteligence, zejména velkých jazykových modelů, se také zásadně mění možnosti, jak efektivně a personalizovaně přistupovat k učení. V moderních aplikacích je dále kladen čím dál tím větší důraz na uživatelskou zkušenost, která stále u mnoha vzdělávacích systémů není na dostatečně vysoké úrovni. Tradiční systémy, jako jsou například stránky se vzdělávacími kurzy nebo aplikace využívající spaced repetition algoritmy, často nedokáží držet krok s tímto rychlým vývojem.

Na základě těchto skutečností jsme se rozhodli s kolegou Bc. Jakubem Vondráčkem vytvořit moderní vzdělávací systém ve formě webové aplikace, který bude reflektovat aktuální změny v oblasti vzdělávání. Hlavním cílem této aplikace je propojit efektivní metody učení s kvalitní uživatelskou zkušeností a vytvořit tak systém, který bude připravený na změny ve vzdělávání, které nové technologie přináší. V rámci této práce bude aplikace zaměřena především na výuku programování a softwarového inženýrství. Bude ovšem navržena tak, aby byla snadno rozšiřitelná a využitelná v různých oblastech vzdělávání. Důraz bude kladen na efektivitu, zábavnost a kvalitu učení, které jsou klíčové pro dlouhodobou motivaci uživatelů.

Systém bude rozšířením již existující aplikace na výuku jazyků, kterou jsme společně s kolegou Bc. Janem Hlaváčem vytvořili v rámci našich bakalářských prací. Díky tomu bude možné projekt stavět na již vybudovaných základech a zaměřit se tak na implementaci pokročilejších funkcí. [1] [2]

Projekt byl rozdělen do dvou diplomových prací. Tato práce bude zaměřena na podrobnější analýzu požadavků a následný vývoj frontendové části aplikace. Diplomová práce kolegy Bc. Jakuba Vondráčka se pak bude převážně zabývat vývojem backendové části této aplikace.

Z hlavního cíle vytvoření vzdělávací aplikace přirozeně vyplývají dílčí cíle, které reflektují jednotlivé fáze životního cyklu softwarového projektu. Prvním

cílem je provést analýzu existující aplikace na výuku jazyků a sepsat požadavky, které budou sloužit jako podklad pro vytvoření nové aplikace. V rámci analýzy bude také provedena analýza konkurenčních aplikací v oboru výuky programování a softwarového inženýrství. Druhým cílem je sepsat specifikaci nových funkcí a navrhnout nové uživatelské rozhraní. Třetím cílem je provedení samotné implementace funkcí a popsání použitých postupů a nově zavedených technologií. Čtvrtým cílem je volba a provedení vhodné formy testování aplikace. Posledním cílem je pak zhodnocení celého řešení a navržení možných úprav do budoucna.

Práce bude zaměřena výhradně na vývoj aplikace a nebude zahrnovat vytváření samotného obsahu, jako například tvorbu konkrétních kurzů, lekcí nebo cvičení.

Výsledná aplikace bude sloužit studentům a samoukům, kteří chtějí rozvíjet své znalosti a systematicky se zdokonalovat v různých oblastech. Výstup této práce bude mít největší přínos pro uživatele, kteří se chtějí vzdělat v oblastech programování a softwarového inženýrství. Zároveň díky její rozšiřitelnosti bude možné aplikaci využít i v mnoha dalších oborech.

Kapitola 1

Analýza

V první fázi vývoje se zaměřím na analýzu. Nejprve provedu analýzu stávající aplikace na výuku angličtiny. V rámci této analýzy se zaměřím na existující funkce, kód, testy a stávající návrh uživatelského rozhraní.

Dále provedu analýzu konkurenčních aplikací, které se zaměřují na výuku programování a softwarového inženýrství. Tato analýza bude sloužit zejména jako podklad pro sepsání požadavků a pro návrh jednotlivých funkcí.

Na konci této kapitoly sepíšu funkční a nefunkční požadavky, ze kterých budu dále vycházet v kapitolách návrhu a implementace.

1.1 Analýza stávající aplikace

Jako první krok analýzy jsem provedl analýzu existující aplikace. V této části textu nejprve popíšu strukturu celého projektu a následně se zaměřím na analýzu kódu, existujících funkcí, testů a existujícího návrhu uživatelského rozhraní frontendové části aplikace.

Existující aplikace je webová aplikace zaměřena na výuku anglického jazyka. Frontendová část aplikace je postavena na knihovnách React a Next.js. Dále aplikace využívá knihovnu komponentů uživatelského rozhraní Material UI a nástroj Storybook pro vytvoření dokumentace jednotlivých komponentů aplikace. Celé uživatelské rozhraní včetně všech komponentů je navrženo v nástroji Figma. Aplikace je nasazena v testovacím prostředí, kde využívá mock API, a dále v produkčním prostředí, kde je napojena na reálné backendové API. [2]

Frontend aplikace komunikuje s backendovou částí přes REST API. Kromě samotného API existuje i mock tohoto API, který byl vytvořen pomocí nástroje Mockoon. Souběžně s mou diplomovou prací, která se týká frontendové části aplikace, se kolega Bc. Jakub Vondráček věnuje vývoji backendové části aplikace v rámci jeho diplomové práce. Proto se v rámci této analýzy a dalších částech textu budu věnovat zejména frontendové části aplikace. [2]

Kromě frontendové části a backendové části aplikace existují v rámci tohoto projektu také webové stránky a dále systém pro správu obsahu aplikace. Těmto částem projektu se v této práci věnovat nebudu, nicméně i tak je bude třeba zohlednit, například při tvorbě a návrhu komponentů uživatelského rozhraní.

1.1.1 Analýza kódu

Nejprve jsem provedl analýzu kódu aplikace, v rámci které jsem našel několik příležitostí ke zlepšení celkové kvality kódu. Zaprvé, některé části kódu, zejména komponenty týkající se učení a opakování látky, nejsou dobře rozšiřitelné. Komponenty nemají flexibilní strukturu a může tak být obtížné do aplikace například přidávat nové typy cvičení. Dále se v kódu nacházejí části, které jsou přímo spjaty s doménou výuky jazyků, přestože by dané části kódu mohly být navrženy obecněji tak, aby byla implementace nezávislá na vyučované doméně. Proto by bylo vhodné tyto části změnit tak, aby bylo možné kód využívat nezávisle na vyučované doméně.

Všechny komponenty jsou v současné aplikaci seskupeny do jedné složky `components`, v rámci které jsou dále rozděleny do podsložek pro základní komponenty, složitější komponenty a komponenty týkající se rozložení jednotlivých stránek. Tato základní organizace komponentů může být vhodná pro malé projekty, nicméně s narůstajícím počtem komponentů a stránek může být značně nepřehledná a špatně udržitelná. Proto by bylo vhodné přeorganizovat komponenty například tak, aby byly stránky a komponenty seskupeny podle daných funkcí. Do globálních složek, jako například do složky `components`, by tak bylo možné vkládat pouze komponenty, které jsou sdíleny napříč jednotlivými funkcemi. Díky této struktuře by tak mohlo být snazší se v kódu orientovat a přidávat nové funkce a další komponenty.

Základní komponenty aplikace by také bylo vhodné přesunout do samostatné knihovny, aby je bylo možné využívat ve všech částech projektu, jako například na webových stránkách, a zajistit tak konzistenci vzhledu napříč celým projektem.

Další příležitostí pro zlepšení kvality kódu jsou samotné třídy, se kterými frontend pracuje. Třídy v kódu jsou definovány ručně tak, aby reflektovaly OpenAPI specifikaci backendového API. Zásadní nevýhodou tohoto přístupu je časová náročnost definice a udržování jednotlivých tříd. Tento přístup dále může snadno vést k nekonzistencím mezi definovanými třídami a reálným API, což může vést k následným chybám v aplikaci. Z tohoto důvodu navrhuji zavést do kódu automaticky generované třídy na základě OpenAPI specifikace backendového API. Díky tomu pak budou třídy snadno udržitelné a bude zajištěna přesná konzistence mezi specifikací API a frontendovou částí aplikace.

V kódu je dále používána řada knihoven a nástrojů, které nebyly v aplikaci za poslední 2 roky vůbec aktualizovány, přestože byly za tu dobu vydány nové verze těchto nástrojů. Jejich aktualizace by tak mohla přinést řadu výhod, jako například vyšší stabilitu a bezpečnost aplikace, nebo vyšší efektivitu vývoje.

Za posledních několik let došlo dále k významnému nárůstu používání nástrojů umělé inteligence pro tvorbu kódu, dokumentací a testů. Současný frontend aplikace ovšem vůbec není pro práci s těmito nástroji přizpůsoben. Proto by bylo vhodné zavést pravidla, konvence a konfigurace, které by umožnily vývojářům pracovat s těmito nástroji a zefektivnit tak celý proces vývoje.

Kromě samotného kódu je také možné provést změny v nasazení aplikace. Současná aplikace je nasazena v testovacím prostředí, kde využívá mock API, a v produkčním prostředí, kde využívá produkční backendové API. Neexistuje ovšem testovací prostředí, kde by aplikace mohla být testována se skutečným API. Z tohoto důvodu by bylo vhodné toto prostředí vytvořit, aby v něm mohla být aplikace manuálně testována například před nasazením do produkčního prostředí.

1.1.2 Analýza testů

V rámci mé bakalářské práce, která se věnovala frontendu stávající aplikace, jsem provedl testy použitelnosti s uživateli. Většina problémů, které byly v rámci těchto testů objeveny, byla již opravena, nicméně stále existují některé problémy, které opraveny nebyly. Tyto problémy jsou popsány v mé bakalářské práci a bude třeba je zohlednit při sestavování požadavků a při návrhu této aplikace. [2]

Frontend aplikace v současné době neobsahuje žádné automatizované testy. Jelikož se aplikace bude zásadně měnit a rozšiřovat, bude do ní vhodné zavést určitou formu těchto testů. Zavedení automatizovaných testů je také v souladu s mým návrhem úprav do budoucna, který jsem popsal v rámci mé bakalářské práce. [2]

V aplikaci dále existují základní nástroje pro formátování a kontrolu kvality kódu Prettier a ESLint. Tyto nástroje by ovšem mohly využívat přísnější pravidla a konvence pro zamezení vzniku nejednoznačného kódu a chyb. Z tohoto důvodu navrhuji aktualizovat tyto nástroje a jejich konfigurace.

1.1.2.1 Analýza návrhu uživatelského rozhraní

Návrh uživatelského rozhraní stávající aplikace jsem vytvořil v rámci mé bakalářské práce v nástroji Figma. Při analýze jsem našel několik příležitostí pro zlepšení tohoto návrhu.

Zprv, komponenty a stránky aplikace by bylo vhodné lépe zorganizovat. V současném návrhu je struktura přehledná, nicméně s narůstajícím počtem stránek a komponentů se může stát značně nepřehlednou.

Značným problémem je také samotná struktura komponentů. V současném návrhu jsou komponenty tvořeny tak, že rozšiřují či upravují komponenty externí knihovny komponentů. Knihovna ovšem často obsahuje zbytečně komplexní strukturu komponentů a parametrů, které nejsou v rámci návrhu této aplikace nijak využívány. Všechny komponenty jsou tak zbytečně komplexní a

práce s nimi je neefektivní. Z tohoto důvodu by bylo vhodné zvolit jiný přístup při tvorbě komponentů, například využití jiné knihovny, nebo vytvoření komponentů ručně bez využití externích knihoven.

Nakonec by bylo vhodné v rámci návrhu vytvořit stránku s přehledem stylů, barev a typografie, které aplikace využívá. Díky tomu by pak bylo možné se styly snadněji pracovat.

1.1.3 Analýza existujících funkcí

Jelikož bude aplikace zásadně měnit svoje zaměření z domény výuky jazyků na doménu výuky programování, je třeba odebrat některé přebytečné funkce, které již nebudou relevantní pro novou doménu. Mezi tyto funkce patří cvičení týkající se výuky jazyků, vyhledávání slovíček, stránky slovíček, přidávání lekcí a slovíček do seznamů oblíbených, správa vlastních kolekcí slovíček nebo možnost vybírání preferovaných témat slovní zásoby.

Kromě odstranění přebytečných funkcí bude dále třeba upravit některé existující funkce. Příkladem toho je stránka kurzu, která v současné době zobrazuje lekce ve formě bodů, které jsou umístěny na čáře připomínající cestu kurzem. Pod jednotlivými body jsou pak uvedeny zkrácené názvy těchto lekcí. Přestože může být toto vyobrazení lekcí pro uživatele vizuálně poutavé, může být pro uživatele značně obtížné na první pohled poznat, co je obsahem dané lekce. Vyhledání konkrétní lekce tak může být pro uživatele velice zdlouhavé a složité. S měnící se doménou tak bude třeba změnit i strukturu kurzů a jejich sekcí a lekcí, aby je uživatelé mohli snadněji procházet a lépe se v nich orientovat.

1.1.4 Závěr analýzy

V rámci analýzy jsem tak našel řadu podnětů ke zlepšení celkové kvality aplikace. Tyto podněty budu brát v potaz ve všech následujících fázích vývoje, zejména při specifikaci požadavků, návrhu funkcí a následné implementaci.

1.2 Analýza existujících řešení

Po dokončení analýzy stávající aplikace se nyní budu věnovat analýze existujících aplikací, které se zaměřují na výuku programování a softwarového inženýrství. Cílem této analýzy je zjistit, jakým způsobem existující aplikace přistupují výuce programování, jakou mají uživatelskou zkušenost a jakými způsoby motivují uživatele k učení. Dále se zaměřím na jejich uživatelská rozhraní a také na to, jaké funkce poskytují zdarma a jaké jsou placené. Věnovat se budu také silným a slabým stránkám těchto aplikací. Na konci této analýzy se pak zaměřím na to, jakými způsoby se může aplikace, kterou budu v rámci této práce vytvářet, odlišit od již existujících aplikací.

Pro analýzu jsem se rozhodl vybrat aplikace, které se nejvíce blíží vizi tohoto projektu a budou tak přímou konkurencí vznikající aplikace. Vybíral jsem mezi nejpoužívanějšími aplikacemi určenými pro samouky, které vyučují softwarové inženýrství a programování pomocí krátkých a interaktivních lekcí. Do analýzy jsem tak zahrnul aplikace Codecademy, Sololearn, Mimo a Scrimba.

Poznatky z této analýzy využiji při sepisování požadavků této aplikace, návrhu nových funkcí a nového vzhledu uživatelského rozhraní.

1.2.1 Mimo

První analyzovanou aplikací je aplikace Mimo¹, která se nejvíce blíží vizi tohoto projektu a bude tak přímou konkurencí vznikající aplikace. Mimo se zaměřuje zejména na výuku moderních programovacích jazyků a frameworků. V době psaní této práce Mimo nabízí 9 kurzů, 4 tzv. kariérní cesty a má napříč platformami přes 40 milionů registrovaných uživatelů [3]. Mimo je k dispozici jako webová aplikace a dále také jako mobilní aplikace na platformách Android a iOS. Pro analýzu použiji primárně aplikaci na platformě Android.

1.2.1.1 Vzhled a uživatelská zkušenost

Mimo se od dalších konkurentů liší zejména svým vzhledem uživatelského rozhraní a stylem výuky. Uživatelské rozhraní je velice minimalistické a přehledné. Místy může aplikace působit až příliš jednoduchým dojmem. Aplikace využívá fialovou monochromatickou paletu barev a celé rozhraní využívá spíše tmavší barvy. V rámci typografie Mimo využívá pro některé nadpisy neproporcionální písmo. To lze pozorovat zejména v mobilní aplikaci a na webových stránkách. Pro textové prvky pak využívá bezpatkové písmo Aeonik.

Rozložení prvků na obrazovce je přehledné. Na hlavní stránce kurzu jsou umístěny jednotlivé lekce, které jsou vyobrazeny ve formě tlačítek na čáře připomínající cestu kurzem. Až po kliknutí na danou lekci se zobrazí její název a možnost spuštění dané lekce. Toto vyobrazení lekcí může na uživatele působit vizuálně poutavým dojmem, nicméně může být obtížné se pomocí něj orientovat v kurzu, protože o lekcích nejsou na první pohled zobrazeny žádné podrobnější informace.

Pro navigaci v mobilní aplikaci používá Mimo velice jednoduchou spodní navigační lištu s pouze pěti prvky. Díky tomu je navigace v aplikaci velice snadná a rychlá.

Webová verze aplikace Mimo je velice podobná mobilní verzi. Rozdíl je především v tom, že ve webové aplikaci jsou lekce vyobrazeny ve formě seznamu. Pro navigaci je použita horní navigační lišta a pro navigaci mezi kapitolami kurzu pak boční navigační lišta.

V aplikaci jsou použity animace a efekty velice zřídka. Některé prvky uživatelského rozhraní jsou animovány při jejich zobrazení. Při učení jsou také

¹Dostupné na: <https://mimo.org/>

použity animace pro přechody mezi jednotlivými cvičeními.

1.2.1.2 Způsoby učení

Jednotlivé kurzy jsou uspořádány do sekcí, které pak obsahují jednotlivé lekce. Lekce jsou uspořádány lineárně a uživatel se je musí učit v daném pořadí. Kromě kurzů Mimo nabízí tzv. kariérní cesty, které mají podobnou strukturu jako kurzy, nicméně většinou obsahují více témat.

Učení lze spustit kliknutím na danou lekci. Při učení se uživateli zobrazuje sekvence stránek na nichž se nachází buď vysvětlení dané látky nebo různá cvičení. Při vyplňování cvičení je typicky uživateli zobrazena otázka, na kterou musí odpovědět například vyplněním textového pole v kódu. Po vyplnění odpovědi se uživateli zobrazí, zda byla odpověď správná, a u cvičení se spustitelným kódem se uživateli zobrazí výstup takového kódu. V kurzu SQL je tak uživateli například po vyplnění odpovědi zobrazena tabulka s daty, která by byla výsledkem daného SQL dotazu. Po vyplnění odpovědi ve webové aplikaci má uživatel také možnost si nechat odpověď vysvětlit v chatovém okně pomocí chatbota.

V aplikaci se nachází několik typů cvičení, které lze rozdělit do následujících kategorií.

Otázka s více odpověďmi Jedním z nejčastějších typů cvičení je otázka, která typicky obsahuje ukázkou kódu, a výběr z několika odpovědí. Uživatel má za cíl zvolit jednu správnou odpověď.

Doplňování do kódu Dalším častým typem cvičení je doplňování částí kódu. Aplikace uživateli v otázce zobrazí ukázkou kódu, do které je třeba doplnit typicky několik slov nebo znaků. Doplňování je v některých případech usnadněno tím, že může uživatel slova a znaky vybírat z předdefinovaného seznamu. To může značně zrychlit procházení těchto cvičení, protože uživatel nemusí vše psát ručně.

Dále aplikace ve cvičeních využívá řadu komponentů pro vysvětlení a lepší pochopení dané látky. Příkladem toho jsou následující komponenty.

Tabulky V některých kurzech, zejména v kurzech vyučující témata týkající se databází, jsou využívány tabulky s daty. Ty se zobrazují jednak jako součástí zadání cvičení a jednak jako výstup po vyplnění daného cvičení.

Výstup kódu v terminálu Po vyplnění cvičení, které v zadání obsahuje spustitelný kód, může být uživateli zobrazen komponent připomínající terminál s výstupem takového kódu. Tyto komponenty jsou typicky zobrazovány například u kurzů, které vyučují konkrétní programovací jazyky.

Zobrazení webové stránky Po vyplnění cvičení, které v zadání obsahuje kód a zaměřuje se na webové technologie, je typicky zobrazena ukázka webové stránky, která reprezentuje výstup daného kódu. Tento typ cvičení je využíván zejména u výuky jazyků, jako například HTML, CSS nebo JavaScript. Dále je tento typ cvičení také využíván pro výuku frontend frameworků.

1.2.1.3 Způsoby motivace uživatele

Mimo používá různé způsoby motivace uživatelů k učení. Jedním z těchto způsobů je gamifikace, kterou lze definovat jako použití prvků herního designu v mimoherních kontextech [4] (překlad autora). V aplikaci Mimo jsem identifikoval následující formy gamifikace.

Virtuální měna V aplikaci je možné za plnění cvičení získávat virtuální měnu, za kterou je následně možné si kupovat různé položky v obchodě aplikace.

Streak Aplikace zaznamenává, kolik dní v řadě uživatel splnil alespoň jednu lekci. Tato řada je nazývána streak. Pokud uživatel daný den nesplnil žádnou lekci, je mu jeho streak resetován. V obchodě aplikace si může uživatel zakoupit několik položek, které mu pomohou zachovat streak i přes to, že se daný den zapomněl učit. Uživatel si tak může preventivně zakoupit položku, která mu zajistí, že se streak nezruší, pokud se zapomene některý den učit. Svůj streak může uživatel také zpětně napravit zakoupením příslušné položky.

Streak výzva V obchodě aplikace může uživatel za virtuální měnu zakoupit tzv. streak výzvu. Pokud uživatel splní výzvu, která spočívá v učení se každý den po dobu 7 dní, dostane jako odměnu určité množství virtuální měny.

Srdíčka Uživatel má k dispozici 5 srdíček, o která přichází, pokud neodpoví správně na danou otázku v průběhu cvičení. Srdíčka se obnovují po určitém čase a uživatel je také může zakoupit za virtuální měnu.

Body XP Za plnění jednotlivých lekcí v aplikaci uživatel může získat tzv. body XP, které reprezentují pokrok uživatele v rámci aplikace. Tyto body jsou následně zobrazeny na jeho profilu a ovlivňují jeho pozici v lize učení.

Liga učení Uživatelé jsou každý týden rozřazeni do ligy, neboli skupiny uživatelů, na základě toho, kolik bodů XP získali v předchozím týdnu. Na profilu uživatele se pak zobrazuje, ve které lize se nachází.

Systém přátel Mimo umožňuje uživatelům si přidávat ostatní uživatele do přátel a sledovat tak jejich pokrok v aplikaci.

Ikona aplikace Mimo umožňuje uživateli si za virtuální měnu zakoupit a změnit ikonu aplikace.

1.2.1.4 Placená verze

Mimo nabízí uživatelům zdarma pouze prvních několik lekcí každého kurzu. Při učení se uživatelům zobrazují reklamy a učení je omezeno funkcí srdíček.

Uživatelé, kteří si zaplatí premium účet, se mohou všechny kurzy a kariérní cesty učit bez omezení. Při učení se jim nezobrazují reklamy a nejsou omezeni funkcí srdíček. Uživatelé s premium účtem si navíc mohou po dokončení kurzu vygenerovat a stáhnout certifikát o dokončení daného kurzu.

Aplikace nabízí sedmidenní zkušební verzi premium účtu zdarma na webových stránkách Mimo. Zkušební verzi lze také získat pozváním přátel do aplikace.

1.2.1.5 Silné a slabé stránky

Silnou stránkou aplikace Mimo je její jednoduchost používání a krátké lekce. Uživatel se tak může snadno učit v průběhu dne čistě s využitím mobilu, což může být praktické zejména pro uživatele, kteří mají omezený čas na učení.

Další silnou stránkou je použitá gamifikace, která může vést ke zvýšené motivaci uživatelů používat aplikaci pravidelně.

Možnou nevýhodou minimalistického vzhledu aplikace může být pocit, který uživatelské rozhraní svým minimalismem vyvolává. Aplikace může místy působit až příliš jednoduše a vyvolat tak dojem, že je příliš jednoduchá na to, aby vyučovala komplexní a pokročilá témata, kterými jsou například programování a softwarové inženýrství. To by mohlo některé uživatele odradit, zejména ty, kteří potřebují daná témata znát na profesionální úrovni.

Další potenciální nevýhodou je samotná navigace v kurzu. Lekce kurzu jsou vyobrazeny vizuálně poutavým způsobem, nicméně najít konkrétní lekci kurzu může být pro uživatele velice obtížné, zejména v mobilní verzi aplikace.

1.2.1.6 Další poznatky

Mimo kromě základních funkcí poskytuje také tzv. pískoviště, kde může uživatel vytvářet a spouštět svůj kód a vidět jeho výstup. V rámci této funkce jsou k dispozici pouze jazyky HTML, CSS a JavaScript a dále framework React.

Aplikace také poskytuje funkci, v rámci které může uživatel vytvořit vlastní webovou aplikaci přímo uvnitř Mimo. V rámci této funkce uživatel zadá umělé inteligenci, co chce vytvořit. Mimo pak uživateli nastaví virtuální prostředí se systémem souborů, kde umělá inteligence celý projekt připraví. Uživatel pak může tento projekt upravovat a spouštět přímo v Mimo. Uživatel si může v rámci této funkce zobrazit danou webovou stránku, soubory projektu a také jednoduché zobrazení tabulek v databázi. Celý projekt pak uživatel může pu-

blikovat na veřejně dostupné webové adrese. Tato funkce je pro uživatele bez premium účtu značně omezená.

1.2.2 Codecademy

Druhou analyzovanou aplikací je aplikace Codecademy². Codecademy je jednou z nejpopulárnějších platforem na výuku programování a softwarového inženýrství. Nabízí přes 800 kurzů [5] a má napříč platformami přes 50 milionů uživatelů [6]. Codecademy je k dispozici jako webová aplikace a dále také jako aplikace Codecademy Go, která je k dispozici na platformách Android a iOS.

Mobilní aplikace Codecademy Go obsahuje pouze jednoduché funkce pro opakování a procvičování látky. Hlavní náplň kurzů se však nachází ve webové aplikaci Codecademy. Proto se v této analýze primárně zaměřím na webovou aplikaci, nicméně zahrnu do ní i poznatky ze zmíněné mobilní aplikace.

1.2.2.1 Vzhled a uživatelská zkušenost

Codecademy na první pohled působí mnohem komplexnějším dojmem než předchozí analyzovaná aplikace Mimo. Design uživatelského rozhraní je spíše minimalistický, nicméně místy může stále působit příliš složitě. V aplikaci je jako barva pozadí využívána primárně světle růžová barva. Při učení je pak pro pozadí využívána bílá, černá nebo tmavě šedá barva. Pro tlačítka a další důležité prvky rozhraní pak aplikace využívá vysoce kontrastní barvy jako například modrou nebo žlutou. Co se typografie týče, Codecademy využívá napříč celou aplikací, s výjimkou bloků kódu, bezpatkové písmo Apercu.

Rozložení prvků na obrazovce je poměrně přehledné a logické, nicméně některé prvky rozhraní mohou být pro uživatele zbytečně matoucí nebo složité. Kupříkladu hned na hlavní stránce kurzu se nachází výrazné tlačítko pro smažení pokroku celého kurzu. Přestože je tato funkce velice důležitá, nejspíše nebude tak často využívána a pozice a výraznost tohoto tlačítka může akorát mást uživatele a odpoutávat jeho pozornost.

Na hlavní stránce kurzu je obsah kurzu rozdělen do seznamu rozbalovacích modulů, v nichž se mohou nacházet různé lekce, kvízy nebo projekty. Struktura kurzu je tak uspořádána přehledně a je snadné se v ní orientovat.

Pro navigaci v aplikaci je využita horní navigační lišta, ve které se nachází rozbalovací menu s odkazy na všechny důležité části aplikace. Dále bývá na jednotlivých stránkách využívána levá boční navigační lišta, která slouží k navigaci mezi funkcemi, které jsou relevantní pro danou stránku.

Mobilní aplikace Codecademy Go je, co se funkcí týče, výrazně jednodušší. Umožňuje základní funkce zapisování si kurzů a učení se jednotlivých témat. V rámci kurzu jsou k dispozici pouze výrazně zjednodušené lekce, které se liší od lekcí a aktivit ve webové aplikaci.

²Dostupné na: <https://www.codecademy.com/>

V aplikaci jsou animace a efekty použity velice zřídka nebo téměř vůbec. Celá stránka tak působí velice statickým dojmem.

1.2.2.2 Způsoby učení

Co se struktury kurzů týče, Codecademy nabízí hned několik druhů kurzů. Mezi ně patří kurzy, kariérní cesty a cesty schopností. Všechny tyto druhy kurzů mají stejnou strukturu a liší se zejména svojí obsáhlostí. Dále Codecademy nabízí řadu doplňujících materiálů, kterými jsou různé nástroje pro učení, dokumentace, videa, články a další.

Jednotlivé kurzy jsou rozděleny do modulů. V nich se pak nachází různé lekce, kvízy, projekty, články nebo další aktivity.

Na hlavní stránce kurzu je uživateli zobrazeno tlačítko, které mu umožňuje spustit lekci nebo aktivitu, u které uživatel naposledy v rámci kurzu skončil. Jednotlivé lekce se pak typicky skládají ze cvičení, která zabírají přibližně 10 minut.

Cvičení se skládají z textového vysvětlení látky a dále jednotlivých úkolů. Uživatel má k dispozici textový editor, ve kterém může psát a editovat připravený kód. Dále má k dispozici okno, ve kterém je vidět výstup daného kódu.

Kromě zmíněných prvků cvičení si může uživatel také otevřít tzv. tahák se shrnutím daného tématu. Dále si může uživatel spustit přiložené YouTube video, ve kterém lektor prochází daným cvičením a vysvětluje jednotlivé kroky.

Učení je přehledné a uživatel má k dispozici dostatek materiálů pro procvičení a pochopení dané látky. Potenciální nevýhodou může být struktura samotných cvičení. Uživatel je nucen si před každým cvičením přečíst několik stránek textu, což může být pro uživatele zdoluhavé a nezáživné.

V rámci učení má uživatel k dispozici AI asistenta, který mu v chatovém okně může vysvětlit danou látku nebo poradit s řešením daného úkolu.

V rámci lekcí jsou uživateli zobrazována především cvičení, v nichž uživatel doplňuje a přepisuje připravený kód přímo v editoru, nebo píše kód od začátku na základě zadání.

V rámci kvízů jsou uživateli zobrazovány různé otázky, typicky formou výběru z několika odpovědí nebo výběru, zda je daný výrok pravdivý nebo nepravdivý.

Jednou z dostupných aktivit jsou takzvané projekty, které svou strukturou připomínají delší cvičení v jednotlivých lekcích. Hlavním rozdílem je, že uživatel nemá možnost nahlédnout do správného řešení a jeho řešení není nijak kontrolováno. Uživatel tak musí daný úkol vyřešit sám bez pomoci. Jednotlivé úkoly pak sám může označit za splněné.

1.2.2.3 Způsoby motivace uživatele

Na rozdíl od aplikace Mimo nepoužívá Codecademy zdaleka tolik forem gamifikace. Aplikace ovšem motivuje uživatele několika následujícími způsoby.

Ukazatele pokroku Uživatel má jak na domovské stránce, tak na stránce kurzu k dispozici ukazatele pokroku, které znázorňují, kolik procent daného kurzu uživatel dokončil.

Ocenění Za splnění určitých cílů v rámci učení uživatel může získat různá ocenění, která se následně zobrazují na jeho profilu ve formě odznaků.

Certifikáty Codecademy nabízí dva typy certifikátů. První typ certifikátu uživatel může získat po dokončení určitého kurzu. Druhý typ certifikátu je tzn. profesní certifikát. Ten může uživatel získat, pokud splní všechny testy v rámci dané kariérní cesty.

Streak Podobně jako v aplikaci Mimo zaznamenává Codecademy streak uživatele, neboli počet dní, po které se uživatel alespoň jednou denně učil bez přerušení.

Sledování pokroku dovedností Unikátní funkcí Codecademy je sledování pokroku u dané dovednosti. Když se uživatel například učí jazyk JavaScript, zvyšuje se mu dovednost vývoje webových aplikací. Uživatel si pak může u každé dovednosti zobrazit jeho úroveň, jak dobře danou látku ovládá a co by se měl například ještě naučit.

Celkově tak Codecademy obsahuje několik způsobů motivace uživatelů, nicméně zdá se, že má v aplikaci gamifikace spíše vedlejší roli.

1.2.2.4 Placená verze

Codecademy, na rozdíl od ostatních aplikací, neobsahuje žádné reklamy. Místo toho nabízí uživatelům několik různých modelů předplatného. Předplatné se dá rozdělit do kategorie pro samouky, studenty a firmy.

V rámci kategorie předplatného pro samouky Codecademy nabízí uživatelům některé základní kurzy zdarma. V rámci předplatného Plus se pak mohou uživatelé učit neomezené množství kurzů a dostanou přístup k cestám schopností. Dále mohou neomezeně využívat AI asistenta při učení. Poslední předplatné Pro umožňuje uživatelům se učit pomocí kariérní cesty, získávat profesní certifikáty. Uživatelé s tímto předplatným mohou využívat další funkce aplikace, jako například simulátor pracovních pohovorů, programátorské výzvy a další.

Codecademy dále nabízí předplatné pro studenty. Toto předplatné je takřka stejné jako Pro předplatné pro samouky. Hlavním rozdílem je snížená cena, pokud uživatel doloží jeho status studenta.

Poslední kategorií předplatného je předplatné pro firmy. Toto předplatné má dvě varianty Teams a Enterprise. Teams nabízí podobné funkce jako Pro předplatné pro samouky s rozdílem, že firma dostane navíc možnost spravovat jednotlivé účty uživatelů, přiřazovat je do skupin a získávat přehledy o učení

uživatelů. Enterprise navíc nabízí možnost integrací dalších aplikací a hodiny vedené online instruktorem.

1.2.2.5 Silné a slabé stránky

Jednou ze silných stránek Codecademy je velké množství výukových materiálů, které stránka nabízí. Uživatelé mají přístup ke kvízům, lekcím, videím, projektům, shrnutím látky a dalším nástrojům, které jim umožní se naučit danou látku.

Další silnou stránkou Codecademy je minimalistické uživatelské rozhraní. Přestože se některé části uživatelského rozhraní mohou zdát zbytečně složité, aplikace je celkově velice přehledná a snadná na používání, obzvláště pokud se vezme v úvahu množství materiálů, kurzů a funkcí, které aplikace nabízí.

Jednou z potenciálních nevýhod Codecademy je právě samotné zobrazování vzdělávacích materiálů při učení. Na stránce cvičení má uživatel například přístup k zadání úkolu, vysvětlení látky ve formě textu, dále k videu vysvětlující látku, AI asistentovi, shrnutí látky, komunitní sekci a dalším podpurným materiálům. Když uživatel prochází jednotlivými cvičeními, může se pak cítit přehlcn tím, že mu jsou všechny informace zobrazovány na jedné stránce.

Další potenciální nevýhodou Codecademy je nedostatek motivace uživatelů. Aplikace sice obsahuje několik forem gamifikace a motivace uživatele, nicméně zdá se, že tyto způsoby motivace mají v aplikaci spíše vedlejší roli.

Poslední nevýhodou této aplikace může být to, že je uživatel sám zodpovědný za opakování si dané látky. Pokud například uživatel dokončí kurz, aplikace předpokládá že danou látku uživatel zná a nepomáhá mu si danou látku opakovat. Uživatel tak může například po několika měsících látku zapomenout a pokud by ji následně potřeboval umět, musel by projít kurzem znovu.

1.2.2.6 Další poznatky

Kromě výše zmíněných výhod a nevýhod jsem v rámci analýzy narazil na několik funkcí, které stojí za zmínku.

Formátování kódu v editoru V rámci cvičení má uživatel k dispozici tlačítko, kterým může automaticky zformátovat napsaný kód v editoru. To může být velice užitečné a přispívat ke kvalitní uživatelské zkušenosti, protože uživatel nemusí kód formátovat manuálně.

Studijní plán Codecademy umožňuje uživateli si vygenerovat studijní plán. V rámci plánu si může nastavit, jak dlouho a kolik dní v týdnu se chce učit. Aplikace mu pak každý den doporučí konkrétní lekce a aktivity, které má daný den projít. Limitací této funkce je, že si uživatel může sestavit plán pouze pro daný kurz a nemá možnost si vygenerovat plán, který by obsahoval více kurzů

zároveň. Přesto může být tato funkce pro uživatele velice užitečná a může značně zjednodušit každodenní učení a mít tak pozitivní vliv na uživatelskou zkušenost.

1.2.3 Sololearn

Sololearn³ je jednou z nejpoblárnějších aplikací na učení se softwarového inženýrství a programování. Nabízí přes 40 kurzů na různá témata napříč softwarovým inženýrstvím a má napříč platformami přes 58 milionů registrovaných uživatelů [7].

Sololearn je k dispozici jako webová aplikace a dále také jako mobilní aplikace na platformách Android a iOS. Pro analýzu použijí mobilní verzi aplikace pro platformu Android.

1.2.3.1 Vzhled a uživatelská zkušenost

Design aplikace může na první pohled působit starším a lehce zvláštním dojmem, zejména kvůli nekonzistentnímu použití komponentů knihovny Material UI a bez dodržování doporučených standardů designu Material Design 2 [8], podle kterých se tato knihovna řídí [9]. Uživatelské rozhraní tak místy působí příliš nepřehledně. Co se palety barev týče, používá Sololearn světle modrou jako primární barvu a světle zelenou jako sekundární barvu. Pro pozadí a plochy pak používá bílou a modrošedou barvu. V rámci typografie je napříč celou aplikací použito písmo Fira Sans, jen pro bloky kódu je použito neproporcionální písmo Fira Mono.

Rozložení prvků na obrazovce je poměrně přehledné a logické. Místy může působit nepřehledně kvůli malému prostoru mezi jednotlivými prvky, malé velikosti textu a většímu nahuštění prvků uživatelského rozhraní na jednotlivých stránkách.

Na hlavní stránce kurzu jsou lekce seskupeny do sekcí, které může uživatel kliknutím rozbít. Díky tomu si může uživatel jednoduše získat přehled o celém kurzu a rozbít části, které ho zajímají.

Pro navigaci je v aplikaci použita dolní navigační lišta s pěti prvky a také boční navigační lišta s ostatními funkcemi.

Webová verze aplikace Sololearn obsahuje méně funkcí, než verze mobilní. V mobilní aplikaci je kromě základních funkcí také možné plnit programátorské výzvy, soutěžit ve výzvách proti ostatním uživatelům, číst články a procházet lekce vytvořené komunitou.

Sololearn využívá animace a efekty velice zřídka. Základní animace jsou použity zejména pro zobrazování prvků na obrazovce. Nejvíce animací je využito v animovaných obrázcích, které jsou zobrazovány po dokončení lekce nebo v úvodním registračním formuláři.

³Dostupné na: <https://www.sololearn.com/>

1.2.3.2 Způsoby učení

Kurzy jsou v aplikaci rozděleny do sekcí, ve kterých se nacházejí jednotlivé lekce a kvízy. Díky této jednoduché struktuře je velice snadné se v kurzech orientovat. Potenciální nevýhodou této jednoduché struktury může být to, že se pak v delších kurzech nachází příliš mnoho sekcí, což může být pro uživatele nepřehledné.

Pro spuštění učení uživatel přejde na stránku kurzu, kde se u nadcházející lekce nachází tlačítko pro spuštění učení. Samotné učení se velice podobá učení v aplikaci Mimo. Skládá se ze série stránek vysvětlující danou látku a krátkých cvičení, při kterých si uživatel látku procvičí. Po zodpovězení otázky si uživatel může také zobrazit diskuzi a komentáře ostatních uživatelů k danému cvičení. Na rozdíl od aplikace Mimo se po dokončení cvičení nezobrazuje automaticky výstup daného kódu. Uživateli je jen zobrazeno, zda odpověděl správně nebo špatně a může si nechat jeho odpověď vysvětlit pomocí umělé inteligence. Na konci každé lekce je uživateli zobrazeno shrnutí s klíčovými informacemi, které by si měl z lekce odnést.

Sololearn nabízí v rámci učení různé druhy cvičení. Nejčastěji se jedná o doplňování slov z nabídky do kódu nebo textu. Dalším častým typem cvičení je výběr správné odpovědi z nabídky. Sololearn také nabízí cvičení, v rámci kterého může uživatel psát celý kód a zobrazovat si jeho výstupy. Celou strukturou učení se tak velice podobá analyzované aplikaci Mimo.

1.2.3.3 Způsoby motivace uživatele

Sololearn pro motivaci uživatelů využívá několik forem gamifikace. Podobně jako aplikace Mimo využívá streak uživatele, srdíčka při učení, body XP, ligu učení a virtuální měnu.

Kromě těchto forem gamifikace jsou uživateli také zobrazovány na různých místech v aplikaci indikátory pokroku, které zobrazují, jakou část jednotlivých kurzů uživatel dokončil.

1.2.3.4 Placená verze

Aplikace nabízí uživatelům dva typy předplatných Sololearn Pro a Sololearn Max.

Uživatelé bez předplatného mohou procházet standardní lekce ve všech kurzech. Jediným omezením je počet srdíček, které uživatelé mají k dispozici. Uživatelům jsou také po dokončení lekcí zobrazovány reklamy.

Uživatelé s předplatným Pro mají přístup k speciálním lekcím v kurzech, ve kterých si mohou procvičit programátorské úlohy zaměřené na reálné problémy z praxe. Dále mají k dispozici neomezený počet srdíček, nejsou jim zobrazovány reklamy a mají přístup k dodatečným materiálům a kvízům.

Předplatné Max kromě zmíněných výhod nabízí uživateli také možnost využít umělou inteligenci, aby mu pomohla při učení. Díky ní si uživatel může

nechat vysvětlit daný kus kódu, proč byla jeho odpověď špatná, a jak by ji měl opravit.

1.2.3.5 Silné a slabé stránky

Silnou stránkou této aplikace je přehlednost a jednoduchá struktura jednotlivých kurzů. Pro obsáhlejší kurzy může být tato struktura nepřehledná, nicméně pro malé a středně velké kurzy, které se v aplikaci vyskytují, může být tato struktura vhodná.

Další silnou stránkou je samotné učení. Stránky s vysvětlením látky i cvičení se snadno používají. Látka tak může být pro uživatele mnohem stravitelnější, než například ve cvičeních konkurenční aplikace Codecademy.

Poslední silnou stránkou Sololearn je motivace uživatele. Aplikace využívá řadu mechanismů pro motivaci uživatele, které mohou přimět uživatele pravidelně aplikaci používat.

Jednou ze slabých stránek Sololearn je design některých stránek. Kvůli nedodržení některých standardů designu mohou stránky působit na uživatele příliš nepřehledně a zastaralým dojmem.

Druhou slabou stránkou aplikace je délka úvodního dotazníku při registraci uživatele. Dotazník obsahuje přes 10 stránek, což může být pro některé uživatele potenciálně odrazující. Založení účtu tak trvá výrazně déle, než u ostatních aplikací.

1.2.3.6 Další poznatky

Při učení aplikace na mobilu umožňuje uživateli přejít prstem po obrazovce doleva nebo doprava a tím přecházet mezi již dokončenými cvičeními v rámci dané lekce. Díky tomu se může uživatel velice jednoduše vrátit k předchozím cvičením a stránkám, které vysvětlují danou látku.

Další funkcí, která stojí za pozornost, je dostupné a snadné přepínání mezi zapsanými kurzy. Na rozdíl od ostatních aplikací, kde uživatel musí pro přepnutí mezi kurzy přejít na samostatnou stránku, může v Sololearn uživatel jednoduše přepnout kurz kliknutím na název kurzu v horní navigační liště, čímž se rozbalí seznam zapsaných kurzů, ze kterých si může uživatel jeden kurz vybrat a přesunout se tak na jeho hlavní stránku. Díky tomu může uživatel snadno přecházet mezi zapsanými kurzy, aniž by se musel přesouvat na samostatnou stránku se seznamem kurzů. To může být velice užitečné zejména pro uživatele, kteří se chtějí učit více kurzů zároveň a potřebují tak mezi kurzy často přepínat.

1.2.4 Scrimba

Poslední analyzovanou aplikací je webová aplikace Scrimba⁴. Tato aplikace se od ostatních odlišuje zejména svým inovativním způsobem výuky programování, který kombinuje prostředí připomínající skutečné IDE a interaktivní výuková videa. Lekce se tak odehrávají přímo v editoru kódu, ve kterém je uživateli vysvětlována látka a zadávány úkoly. Tento přístup výuky může přinést řadu výhod oproti tradičním metodám výuky a proto jsem se rozhodl tuto aplikaci do analýzy zahrnout.

Platforma Scrimba obsahuje přes 80 kurzů na různá témata napříč softwarovým inženýrstvím [10]. V době psaní tohoto textu jsou kurzy zaměřeny zejména na webové technologie a umělou inteligenci.

Na rozdíl od ostatních analyzovaných aplikací Scrimba nemá v současné době mobilní verzi aplikace. Proto pro analýzu využiji dostupnou webovou aplikaci Scrimba.

1.2.4.1 Vzhled a uživatelská zkušenost

Na první pohled působí Scrimba jako přehledná a minimalistická aplikace. Ve srovnání s konkurenční aplikací Mimo používá aplikace menší velikost písma a zobrazuje na stránkách více textu. Aplikace tak může působit komplexnějším dojmem. Co se palety barev týče, používá Scrimba monochromatickou paletu modré barvy. Odstíny modré jsou použity pro pozadí, primární akce, ikony i textové prvky. Pro upozornění a některé štítky aplikace výjimečně využívá žlutou nebo zelenou barvu. Napříč aplikací je použito systémové písmo, tedy písmo, které je nastaveno v závislosti na operačním systému uživatele. Na systému Windows je tak použito například písmo Segoe, na zařízeních macOS pak písmo San Francisco a na zařízeních Android písmo Roboto [11].

Prvky jsou na obrazovce rozloženy velice přehledně a logicky. Ve srovnání s ostatními aplikacemi Scrimba používá méně prostoru mezi jednotlivými prvky uživatelského rozhraní. V důsledku toho může aplikace působit komplexnějším dojmem.

Na hlavní stránce kurzu jsou jednotlivé sekce, moduly a lekce vizuálně uspořádány do stromové struktury, kterou lze postupně rozbalovat. Díky tomu může uživatel jednoduše získat přehled o celém kurzu a rozbalit části, které ho zajímají. Obsah kurzu je tak uspořádán velice přehledně.

Na rozdíl od ostatních analyzovaných aplikací Scrimba nevyužívá pro navigaci horní navigační lištu, ani žádnou patičku stránky. Pro navigaci je využívána výhradně boční navigační lišta, která je umístěna na levé straně obrazovky.

Ve srovnání s ostatními aplikacemi Scrimba výrazně více využívá animace jednotlivých prvků uživatelského rozhraní a animované přechody mezi stránkami. Na většině stránek aplikace jsou při načtení zobrazeny animace někte-

⁴Dostupné na: <https://scrimba.com/>

rých prvků rozhraní. Další výraznou animaci lze zpozorovat při přejetí kurzorem nad tlačítko. Při animaci je tlačítko zvýrazněno čarou, která ho postupně obklopí po jeho obvodu.

Animace jsou tak napříč aplikací používány často, nicméně minimalisticky a v přiměřeném množství. Díky tomu Scrimba může působit výrazně interaktivnějším a vizuálně poutavějším dojmem, než její konkurenti.

1.2.4.2 Způsoby učení

Scrimba nabízí, podobně jako ostatní analyzované aplikace, kurzy a kariérní cesty. Kariérní cesty jsou takřka stejné jako kurzy a liší se pouze v tom, že obsahují více témat. Lekce v kurzech jsou uspořádány chronologicky a seskupeny do kapitol a podkapitol.

Unikátní funkcí této aplikace je samotné učení. Při spuštění učení je uživateli v prohlížeči zobrazen editor připomínající skutečné IDE, ve kterém je možné prohlížet soubory a editovat kód. V editoru zároveň uživatel vidí okno s výstupem daného kódu. Dále je uživateli zobrazena prezentace doplňující výklad dané látky. Všechna tato okna může uživatel v průběhu výkladu schovávat, zobrazovat a procházet dle libosti.

Při učení uživatel poslouchá audio s vysvětlením dané látky. V průběhu toho je mu buď zobrazena prezentace nebo editor, ve kterém může sledovat, jak lektor přepisuje kód a jak pohybuje svým kurzorem myši. Při výkladu může uživatel procházet libovolné soubory a přepisovat jejich obsah. Dále si může kdykoliv zobrazit okno s prezentací nebo výstupem kódu. Audio s výkladem látky může uživatel kdykoliv přetáčet zpět a nebo dopředu. V průběhu lekce jsou uživateli zadávány úkoly, které musí v editoru splnit. Zadání je provedeno lektorem v audio nahrávce. Po zadání úkolu je audio pozastaveno, dokud uživatel nesplní daný úkol.

Na rozdíl od ostatních analyzovaných aplikací Scrimba nepoužívá předdefinované typy cvičení. Veškerý výklad látky i cvičení se odehrávají přímo v editoru. Cvičení typicky spočívají v úpravě existujícího kódu nebo napsání nového kódu. Celkově je učení velice interaktivní a připomíná kombinaci videí s výkladem a skutečného IDE. Tato forma učení může být pro uživatele velice přínosná, zejména kvůli názornějšímu vysvětlení látky a interaktivnímu způsobu učení.

1.2.4.3 Způsoby motivace uživatele

Na rozdíl od ostatních aplikací Scrimba neobsahuje téměř žádné formy gamifikace. Uživatelé nesbírají žádné body nebo ocenění, a nemohou spolu v aplikaci interagovat. Motivující pro uživatele ovšem může být, že aplikace jasně ukazuje, kolik procent kurzu uživatel dokončil a kolik cvičení uživatel splnil. Všechny splněné lekce jsou také označeny výraznou zelenou barvou a ikonou zaškrtnutí. Tato vizuální indikace pokroku může pro některé uživatele sloužit jako zdroj motivace.

1.2.4.4 Placená verze

Scrimba na rozdíl od ostatních aplikací neobsahuje žádné reklamy a nabízí pouze jeden model předplatného.

Uživatelé bez předplatného mají k dispozici většinu kurzů zdarma. Některé kurzy a kariérní cesty jsou ovšem k dispozici pouze uživatelům s aktivním předplatným. V nich mají uživatelé bez předplatného možnost si vyzkoušet několik prvních lekcí.

Uživatelé s aktivním předplatným mají kromě základních kurzů přístup k pokročilejším kurzům a kariérním cestám. Na konci každého kurzu si navíc mohou vygenerovat a stáhnout certifikát o dokončení daného kurzu.

1.2.4.5 Silné a slabé stránky

Silnou stránkou této aplikace je její unikátní přístup k učení. Uživatelé se učí přímo v editoru připomínající IDE a veškerá látka je vysvětlena lektorem a doplněna o prezentaci s obrázky, GIFy a diagramy. Učení je tak zábavné, interaktivní a může být pro uživatele více srozumitelné, zejména u složitější látky.

Další silnou stránkou aplikace je minimalistický design a rozložení obsahu. Aplikace se velice snadno používá a je velice jednoduché najít, co uživatel potřebuje. Důvodem může být také to, že aplikace obsahuje pouze funkce, které jsou klíčové pro výuku programování a nezahluje uživatele dodatečnými materiály a nástroji.

Mírnou nevýhodou inovativního způsobu učení je volnost uživatele při učení. Uživatel může v průběhu výkladu svým počínáním například omylem vypnout okno s prezentací nebo výstupem kódu. Proto se může velice jednoduše stát, že lektor v audio nahrávce mluví například o diagramu v prezentaci, který se uživateli na obrazovce nezobrazuje, protože uživatel dříve kliknul na editor, což zamezilo zobrazení okna s prezentací. Pro uživatele tak může být místy velice matoucí, kde se zrovna lektor nachází v průběhu výkladu a může být těžké výklad sledovat.

Další potenciální nevýhodou může být nedostatek motivace uživatele. Aplikace neobsahuje téměř žádné funkce, které by se zaměřovaly na dlouhodobou motivaci uživatelů. Proto může být pro uživatele obtížné se dlouhodobě přimět k používání této aplikace.

Poslední slabou stránkou aplikace Scrimba je místy nefunkční editor kódu. Protože do kódu může zároveň zapisovat virtuální lektor a uživatel, několikrát se při mém průchodem aplikací stalo, že byl například kurzor posunutý o několik znaků vedle, nebo že psaní do kódu nefungovalo a přepisovaly se jiné znaky, než by uživatel očekával. Tento problém může být pro uživatele velice odrazující při používání aplikace, protože se pak aplikace jeví jako nefunkční.

1.2.4.6 Další poznatky

V rámci analýzy jsem došel k pozoruhodnému poznatku. Na rozdíl od ostatních aplikací Scrimba nemá dedikovanou domovskou stránku, která by například uživateli prezentovala výhody aplikace a jiné marketingové materiály. Místo toho je uživateli na hlavní stránce zobrazen seznam všech kurzů a krátké video popisující, jak se aplikace používá. Uživatelské rozhraní nepřihlášeného a přihlášeného uživatele se tak téměř neliší. Uživatelé mohou prohlížet všechny kurzy bez přihlášení a mohou spustit až dvě libovolné lekce, než po nich bude aplikace vyžadovat přihlášení. Díky tomu mohou uživatelé zjistit, zda jim vyhovuje formát výuky, než se do aplikace zaregistrují.

1.2.5 Závěr analýzy

V této části textu shrnu nejdůležitější výstupy provedené analýzy konkurenčních řešení. Tyto výstupy budou sloužit jako podklad pro sestavení požadavků tohoto projektu.

1.2.5.1 Vzhled a uživatelská zkušenost

Většina aplikací se snaží poskytnout uživateli přehledný a snadno použitelný způsob učení. Vzhled uživatelského rozhraní je u většiny aplikací minimalistický. Tomu odpovídá i struktura stránek a samotných kurzů, která bývá velice jednoduchá.

Aplikace často používají animace a další vizuální efekty, aby udělaly učení a používání aplikace vizuálně poutavější. To může potenciálně pomoci udržet pozornost uživatele při učení. Současná aplikace animace a efekty téměř nepoužívá a proto stojí za zvážení jejich použití.

1.2.5.2 Způsob výuky

Analyzované aplikace volí různé způsoby výuky. Aplikace Mimo a Sololearn učí uživatele pomocí krátkých cvičení, Codecademy pomocí delších úkolů a Scrimba pomocí vysoce interaktivních lekcí. Každý způsob výuky poskytuje určité výhody a nevýhody a bude třeba zvážit, který způsob je pro uživatele nejvhodnější.

V rámci učení jsou uživateli zobrazovány různé typy cvičení. Typicky se jedná o doplňování slov do kódu, psaní kódu do editoru a výběr správné odpovědi z nabídky.

Ve většině aplikací je uživateli látka vysvětlena pomocí krátkých textových popisů. Výjimkou je aplikace Scrimba, ve které je látka uživateli vysvětlena pomocí audio nahrávky v kombinaci s prezentací a interaktivním editorem. Použití audio nahrávek a interaktivních prvků může být pro uživatele potenciálně poutavější a srozumitelnější, než pouhé čtení textu. Podobnou formu výuky bude vhodné zvážit při návrhu struktury učení v této práci.

Většina aplikací také zobrazuje lekce kurzu ve formě seznamu. To se zásadně liší od současné aplikace, ve které jsou lekce vizuálně uspořádány do tvaru připomínající cestu kurzem. Přestože tato struktura může být vizuálně poutavější, a například aplikace Mimo ji také využívá, bude třeba zvážit, zdali by pro uživatele nebylo přehlednější uspořádání lekcí do přehlednějšího seznamu.

1.2.5.3 Motivace uživatelů

Ve většině aplikací jsou používány určité funkce, které mají za cíl motivovat uživatele, aby se učil a dlouhodobě používal danou aplikaci.

Mezi tyto funkce se řadí zejména různé způsoby gamifikace. Mezi nejpoužívanější způsoby v analyzovaných aplikacích patří například streak, body XP, ligy učení, srdíčka při učení a virtuální měna.

Kromě těchto prvků aplikace také často využívají indikátory pokroku v rámci daných kurzů. Dokončené lekce jsou například označovány zelenou barvou a uživateli je zobrazován procentuální pokrok v daném kurzu. Tyto ukazatele pokroku ve stávající aplikaci nejsou vůbec zahrnuty a stojí za zvážení jejich použití.

Posledním výrazným prvkem pro motivaci uživatelů jsou certifikáty o dokončení daného kurzu. Po dokončení kurzu a splnění podmínek si může uživatel ve většině aplikací stáhnout certifikát o jeho dokončení ve formátu PDF. Tento certifikát pak může uživatel například využít při hledání práce nebo prezentaci svých znalostí. Tuto funkci bude opět vhodné zvážit při sestavování požadavků tohoto projektu.

1.2.5.4 Odlišení aplikace od existujících řešení

Na závěr analýzy se zaměřím na specifické funkce, kterými se bude moci tato aplikace odlišit od analyzovaných konkurenčních řešení. Tyto funkce budou moci aplikaci poskytnout na trhu konkurenční výhodu a uživatelům lepší uživatelskou zkušenost.

Interaktivní lekce s audio výkladem Hlavní konkurenční výhodou této aplikace budou interaktivní lekce, které kombinují výhody přístupů analyzovaných aplikací. Lekce budou krátké a budou se skládat z interaktivních stránek výkladu a cvičení. Způsob výuky tak bude podobný způsobu výuky aplikací Mimo a Sololearn. Průběh učení ovšem bude doplněn o audio nahrávku s výkladem, podobně jako tomu je v aplikaci Scrimba. Uživatelům tak bude látka prezentována po malých krocích, což jim umožní látku snadněji pochopit a lépe se v ní orientovat. Aplikace bude moci díky tomuto formátu výuky přesně identifikovat, se kterými koncepty má daný uživatel problém, a tyto koncepty mu bude moci častěji ukazovat při opakování. Audionahrávky, doplněné o animace, kód a obrázky, navíc budou moci uživatelům přinést lepší uživatelskou

zkušenost. Uživatelé tak nebudou muset číst dlouhé odstavce textu, ale veškerá látka jim bude tímto způsobem prezentována a názorně vysvětlena.

Důraz na opakování látky Další konkurenční výhodou oproti analyzovaným aplikacím bude větší důraz na opakování již naučené látky. Většina konkurenčních aplikací na opakování klade velmi malý důraz. Pro uživatele tak může být obtížné si koncepty a informace dlouhodobě zapamatovat.

Stávající aplikace již obsahuje určité formy opakování pomocí spaced repetition algoritmu [1]. Tuto formu opakování bude třeba změnit a přizpůsobit tak, aby bylo možné ji využít pro výuku programování a softwarového inženýrství. Při návrhu aplikace tak bude třeba zvážit, jakým způsobem bude opakování látky v aplikaci prováděno.

Zaměření na gamifikaci Některé konkurenční aplikace využívají prvky gamifikace pro zvýšení a udržení motivace uživatele. Tyto prvky ovšem často nejsou použity příliš efektivně. Některé aplikace tyto prvky využívají velice zřídka a jiné je zase nevyužívají způsobem, jaký by byl pro uživatele optimální.

Aplikace se tak může na tento aspekt učení zaměřit a poskytnout uživatelům efektivnější a propracovanější použití gamifikace.

Zaměření na uživatelskou zkušenost Některé aplikace, jako například Sololearn nebo Scrimba, mají řadu nedostatků, co se návrhu vzhledu a uživatelské zkušenosti týče. Tyto nedostatky mohou být pro uživatele odrazující. Při vývoji této aplikace tak bude možné se více zaměřit na vzhled, uživatelskou zkušenost a testování, díky čemuž by aplikace mohla poskytnout kvalitnější uživatelskou zkušenost.

Kvalitní učení Efektivní a kvalitní učení je klíčovým aspektem vzdělávacích aplikací. Přesto konkurenční aplikace obsahují řadu nedostatků, které mohou uživateli znepříjemnit učení. Při zapnutí aplikace Sololearn je například uživatel odkázán na svůj profil, kde je zahlcen řadou nepodstatných informací. Místo toho by například mohl být odkázán přímo na stránku kurzu, kde by mohl rovnou pokračovat v učení.

V průběhu učení je například v aplikaci Codecademy zobrazeno uživateli značné množství materiálů a informací, které pro něj mohou být velice rozptylující a snižovat tak efektivitu samotného učení.

Aplikace by se tak mohla zaměřit na předejití těchto nedostatků a poskytnout uživateli příjemnější a kvalitnější zkušenost při samotném učení.

Studijní plány V rámci učení se jednoduše může stát, že se uživatel potřebuje učit několik kurzů zároveň. Začínající programátor se například musí naučit řadu technologií a jazyků, se kterými se musí naučit pracovat.

Pokud se ovšem uživatel chce učit více kurzů zároveň, musí mezi jednotlivými kurzy neustále složitě přepínat a sám plánovat, jakou látku se bude učit dál.

Uživatel by si tak mohl například v aplikaci zapsat jednotlivé kurzy a určit, v jakém pořadí se je chce učit. Aplikace by pak mohla uživateli sestavit studijní plán a předkládat mu lekce k učení podle tohoto plánu. Uživatel by si tak mohl zapsat kurzy a následně se jen učit lekce, které mu aplikace doporučí.

Sociální aspekty Další slabou stránkou aplikací je nedostatek sociálních interakcí mezi uživateli. Aplikace často neobsahují uživatelské profily, sledování aktivit ostatních uživatelů, vyhledávání jejich profilů a sdílení pokroku mezi ostatními uživateli. Tento sociální aspekt používání může být zásadním zdrojem motivace pro některé uživatele a mohl by přispět ke zlepšení uživatelské zkušenosti.

Využití animací, přechodů a efektů Konkurenční aplikace zřídka využívají animace, přechody a efekty mezi stránkami. Přestože je třeba používat tyto prvky v přiměřeném množství, může jejich použití docílit toho, že bude aplikace pro uživatele poutavější a bude se tak moci odlišit od konkurentů.

1.3 Analýza požadavků

V této sekci sepišu funkční a nefunkční požadavky vznikající aplikaci. Při sepisování požadavků budu vycházet zejména z analýzy stávajícího řešení a analýzy konkurenčních aplikací. Dále budu vycházet z výsledků uživatelských testů, které jsem provedl v rámci mé bakalářské práce [2] zabývající se stávající aplikací. Vycházet budu také z návrhu úprav do budoucna, které jsem popsal ve zmíněné práci.

Po sepsání jednotlivých požadavků se zaměřím na určení jejich priorit pomocí metody MoSCoW [12]. Tato metoda umožňuje snadno rozdělit požadavky do čtyř kategorií dle jejich priority. Požadavky v kategorii *must have* musí být při implementaci splněny, požadavky kategorie *should have* by měly být splněny, požadavky kategorie *could have* mohou být splněny, a požadavky kategorie *won't have* nebudou v rámci této práce implementovány.

1.3.1 Funkční požadavky

Na základě předchozí analýzy stávajícího řešení a analýzy konkurenčních aplikací jsem sestavil funkční požadavky této aplikace. Funkční požadavky specifikují chování a funkce, které by měla aplikace obsahovat [13, s. 4]. Na základě těchto požadavků budu následně provádět návrh a implementaci jednotlivých funkcí. V rámci této analýzy jsem specifikoval následující funkční požadavky.

1.3.1.1 FP-1 Úvodní stránka aplikace

Priorita: Should have

Aplikace by měla obsahovat úvodní stránku, která bude uživateli zobrazena, když není přihlášen, nebo když poprvé otevře aplikaci. Na této stránce by měl mít uživatel možnost zvolit, jestli se chce přihlásit nebo registrovat. Stránka by měla obsahovat logo a název aplikace a krátký popis hlavní hodnoty, kterou aplikace uživateli přináší.

Tento požadavek vychází z analýzy konkurence, ve které většina aplikací obsahovala kromě stánky pro přihlášení a registraci i úvodní stránku s grafikou či textem, který prezentuje hlavní přínosy aplikace pro uživatele.

1.3.1.2 FP-2 Úvodní dotazník a doporučování kurzů

Priorita: Must have

Aplikace by měla obsahovat úvodní dotazník zaměřený na získání informací o uživateli. Tyto informace by se měly týkat zejména cílů uživatele a jeho zkušeností s programováním. Na základě dotazníku by měla aplikace nabídnout uživateli několik kurzů, které odpovídají jeho cílům a zkušenostem.

Tato funkce vychází z analýzy konkurenční aplikace Codecademy, která využívá úvodní dotazník pro doporučování kurzů uživateli.

1.3.1.3 FP-3 Prohlížení a doporučování kurzů

Priorita: Must have

Uživatel by měl mít možnost si v aplikaci zobrazit přehled všech dostupných kurzů a jednoduše mezi nimi hledat. U každého kurzu by měl mít dále možnost se přesunout na stránku s podrobnějšími informacemi o daném kurzu.

Součástí přehledu kurzů by také měla být sekce s doporučenými kurzy. Tato doporučení by měla být založena na úvodním dotazníku, který uživatele vyplnil při registraci. Uživatel by měl mít možnost dotazník kdykoliv vyplnit znovu a přizpůsobit si tak svoje preference při zapisování kurzů.

1.3.1.4 FP-4 Stránka kurzu

Priorita: Must have

Každý kurz v aplikaci by měl mít vlastní stránku s detailním popisem a přehledem kapitol, které jsou v kurzu obsaženy. Uživatel by měl mít možnost se do kurzu přihlásit. Pokud má uživatel již kurz zapsaný, měl by mít možnost snadno pokračovat v učení se daného kurzu. Dále by měl mít uživatel možnost se z kurzu odhlásit.

Uživatelé s premium účtem by měli mít také možnost si zobrazit svůj certifikát o dokončení kurzu, pokud již kurz dokončili.

Tento požadavek vychází z analýzy konkurence, ve které většina aplikací obsahovala stránky kurzů s jejich popisem a s přehledem lekcí, které jsou v kurzu obsaženy.

1.3.1.5 FP-5 Stránka kapitoly kurzu

Priorita: Must have

Aplikace by měla umožnit uživateli si zobrazit stránku kapitoly kurzu. Na této stránce by měly být zobrazeny všechny lekce dané kapitoly, které by měly být seskupeny dle jejich témat do sekcí. Jednotlivé lekce by měly být označeny jako dokončené či nedokončené podle toho, zda uživatel danou lekci dokončil. V aplikaci by měla být také vizuálně zvýrazněna lekce, kterou by se uživatel měl učit dále.

Ve stávající aplikaci jsou lekce kurzu vyobrazeny jako body na čáře připomínající cestu, po které se uživatel při postupování kurzem pohybuje. Jak jsem již popsal v kapitole Analýza stávající aplikace, toto zobrazení může být vizuálně poutavé, nicméně z hlediska použitelnosti může být nepraktické. Proto by měl být v rámci tohoto požadavku navržen nový vzhled přehledu lekcí. V rámci přehledu by měly být lekce uspořádány do seznamu, ve kterém se bude moci uživatel snadno orientovat. Seznam by ovšem měl stále zachovat vizuálně poutavý vzhled a znázorňovat chronologickou posloupnost jednotlivých lekcí.

1.3.1.6 FP-6 Certifikáty o dokončení kurzu

Priorita: Should have

Po dokončení kurzu by měl být uživateli s premium účtem vygenerován certifikát o dokončení daného kurzu. Certifikát by měl být zobrazen na veřejně dostupné stránce a měl by obsahovat základní informace o dokončeném kurzu a uživateli, který kurz dokončil. Uživatel by měl mít možnost odkaz na certifikát jednoduše sdílet. Uživatelům, kteří nemají premium účet, by měla být zobrazena informace, že si budou moci certifikát vygenerovat po zakoupení premium účtu.

1.3.1.7 FP-7 Cesty učení

Priorita: Must have

Aplikace by kromě kurzů měla nabízet tzv. cesty učení, neboli kariérní cesty. Každá cesta učení by měla seskupovat několik kurzů a poskytnout tak uživateli snadnější možnost se učit více kurzů týkajících se dané profese nebo daného tématu.

Podobně jako u kurzů by měla mít každá kariérní cesta vlastní stránku s podrobnějším popisem. Na této stránce by měl být také zobrazen přehled všech kurzů, které jsou v této cestě obsaženy.

Uživatel by měl mít možnost si kariérní cestu zapsat. Pokud je již do kariérní cesty zapsaný, měl by mít možnost na stránce cesty jednoduše pokračovat

v učení se nadcházející lekce. Dále by měl mít uživatel možnost se z kariérní cesty odhlásit.

Uživatelé s premium účtem by měli mít také možnost si zobrazit svůj certifikát o dokončení kariérní cesty, pokud již kariérní cestu dokončili.

Tento požadavek vychází z analýzy konkurence, ve které většina analyzovaných aplikací nabízela určitou formu kariérních cest, díky kterým se uživatel mohl snadno naučit témata týkající se konkrétní profese.

1.3.1.8 FP-8 Správa zapsaných kurzů a cest učení

Priorita: Must have

Uživatel by měl mít možnost si v aplikaci zobrazit přehled zapsaných kurzů a cest učení. V rámci tohoto přehledu by měl mít dále možnost se přesunout na stránku libovolného kurzu či cesty učení. Dále by měl mít uživatel možnost se z libovolného kurzu nebo cesty učení odhlásit.

1.3.1.9 FP-9 Studijní plán

Priorita: Must have

Aplikace by měla obsahovat stránku studijního plánu, na které bude zobrazen seznam všech zapsaných kurzů daného uživatele. Na této stránce by měl mít uživatel možnost si zvolit, které kurzy se chce učit, které si chce pouze opakovat a které se nechce momentálně vůbec učit. Podle studijního plánu pak bude aplikace doporučovat uživateli lekce při učení a opakování.

Funkce studijního plánu vychází z analýzy konkurenční aplikace Codecademy, která umožňuje uživateli si vygenerovat studijní plán. Funkce dále vychází z problému, který nastává v konkurenčních aplikacích, pokud se chce uživatel učit více kurzů zároveň. Uživatel při učení více kurzů bývá nucen často přepínat mezi jednotlivými kurzy a sám si plánovat, kterou látku se bude dále učit. Studijní plán by měl tomuto problému předejít a umožnit tak uživateli se snadno učit látku napříč mnoha kurzy.

1.3.1.10 FP-10 Učení se a opakování podle studijního plánu

Priorita: Must have

Aplikace by měla uživateli na hlavní stránce zobrazit nadcházející lekci, kterou by se měl podle studijního plánu učit. Lekce by měla být vybrána ze zapsaných kurzů uživatele, které jsou ve studijním plánu v režimu učení. Uživatel by měl mít možnost si tuto lekci snadno spustit.

Dále by měl mít uživatel možnost si na hlavní stránce aplikace spustit opakování, v rámci kterého mu aplikace vybere látku k zopakování podle toho, jak dobře si uživatel danou látku pamatuje. Látka k opakování by měla být vybrána z kurzů, které jsou ve studijním plánu buď v režimu učení, nebo v režimu opakování.

Tato hlavní stránka by měla být nastavena jako výchozí stránka aplikace, na kterou bude uživatel přesměrován po spuštění aplikace.

Cílem této funkce je poskytnout uživateli co nejjednodušší způsob spuštění učení a využít tak princip návrhu uživatelského rozhraní *microbreaks*. Hlavní myšlenkou tohoto principu je přizpůsobit aplikaci tak, aby ji mohl uživatel použít kdykoliv, kdy má v průběhu dne jen pár minut volného času a získat tak hodnotu, kterou aplikace nabízí. Když uživatel například čeká chvíli na zastávce autobusu, může aplikaci na pár minut zapnout a využít tak efektivně svůj čas. Klíčovým prvkem tohoto principu je umožnit uživateli se ihned po spuštění aplikace přesunout k dané hodnotě, kterou aplikace nabízí. V případě této aplikace je tak cílem umožnit uživateli se ihned po spuštění aplikace přesunout k samotnému učení. Tento princip je využíván například aplikacemi sociálních sítí, které jsou pečlivě navrženy tak, aby je mohl uživatel kdykoliv v průběhu dne na pár minut zapnout a použít. [14]

1.3.1.11 FP-11 Opakování po dokončení lekce

Priorita: Must have

Zásadní funkcí této aplikace je možnost si jednoduše opakovat již naučenou látku tak, aby ji uživatel nezapomněl. Po dokončení každé nové lekce by měla být uživateli nabídnuta možnost spustit opakování již naučené látky. Uživatel bude moci opakování odmítnout a přesunout se tak na stránku, ze které spustil předchozí učení. Dále bude mít možnost spustit opakování již naučené látky.

Po spuštění opakování bude uživateli vybrána látka k zopakování. Tato látka by měla být vybrána z kurzů, které jsou ve studijním plánu buď v režimu učení, nebo v režimu opakování.

1.3.1.12 FP-12 Nová struktura učení a opakování

Priorita: Must have

Učení by mělo být strukturováno tak, aby byla uživateli po malých krocích vysvětlena probíraná látka a aby si uživatel mohl danou látku snadno procvičit. Učení se tak bude skládat z tzv. bloků učení, přičemž některé budou zaměřené na vysvětlení látky a některé na procvičení dané látky. Při procvičování látky by měla aplikace sbírat data o tom, jak uživatel danému tématu rozumí. Tato data budou později využita pro doporučování látky k zopakování.

Při učení by měla aplikace uživateli vysvětlovat látku formou připomínající video s audio nahrávkou. V rámci procvičování by pak měla aplikace zobrazovat cvičení, která mohou být také doprovázena audio nahrávkou. Cvičení i vysvětlení látky budou dále doplněna o různé vizuální komponenty, které pomohou uživateli lépe porozumět dané látce. Učení tak bude kombinovat přístup konkurenční aplikace Mimo, která využívá krátká cvičení a vizuální komponenty pro vysvětlení látky, a přístup konkurenční aplikace Scrimba, v

níž je látka vyučována pomocí interaktivního prostředí s audio nahrávkou vysvětlující danou látku a cvičení.

Při učení by měl mít uživatel možnost se vracet k již dokončeným blokům, tedy k dokončeným cvičením a k předchozím vysvětlením látky.

Při řešení tohoto požadavku je třeba také brát v potaz výsledky uživatelského testování původní aplikace, které jsem provedl v rámci mé bakalářské práce [2]. Výsledky tohoto testování ukázaly, že aplikace nedává uživateli dostatečně jasnou zpětnou vazbu o tom, zdali odpověděl v rámci cvičení správně nebo špatně. Proto by měla aplikace uživateli jasněji poskytovat zpětnou vazbu v rámci cvičení.

1.3.1.13 FP-13 Bloky cvičení

Priorita: Must have

V aplikaci by v rámci učení a opakování měly být uživateli zobrazovány bloky cvičení, díky kterým si bude moci procvičit nebo zopakovat danou látku. Aplikace by měla obsahovat různé komponenty, ze kterých se budou jednotlivé bloky cvičení skládat. Tyto komponenty lze rozdělit do dvou kategorií.

První kategorií jsou pasivní komponenty, které mají za cíl pouze zobrazovat informace ve formě textu, obrázků, tabulek, diagramů apod. Tyto komponenty budou sloužit zejména pro specifikaci zadání daného cvičení. Aplikace by měla obsahovat následující pasivní komponenty:

- nadpis
- odstavec textu
- jednoduchou tabulku
- tabulku připomínající tabulku databáze
- číselné a nečíselné seznamy
- obrázky ve formátech PNG, JPG, SVG a GIF
- diagramy definované pomocí jazyka Mermaid⁵.
- ukázkou kódu
- ukázkou výstupu kódu v podobě výpisu v terminálu
- ukázkou výstupu kódu v podobě webové stránky
- ukázkou výstupu kódu v podobě tabulky

⁵Dostupné na: <https://mermaid.js.org/>

Druhou kategorií jsou aktivní komponenty, se kterými bude uživatel moci interagovat a splnit tak dané cvičení. Cílem těchto komponentů je získat data o tom, jak uživatel danému tématu rozumí. Aplikace by měla obsahovat následující aktivní komponenty:

- otázku s výběrem správné odpovědi z nabídky odpovědí
- kód s vynechanými textovými políčky, které lze vyplnit psaním
- kód s vynechanými textovými políčky, které lze vyplnit výběrem z nabídky slov
- kód, který lze psát nebo upravovat ručně

Aplikace by měla dále umožňovat jednotlivé komponenty doplnit o audio nahrávky, které budou komponenty komentovat a doprovázet tak uživatele při vyplňování cvičení. Uživatel si tak bude moci například poslechnout zadání cvičení, aniž by ho musel číst. Při zobrazení bloku cvičení by se měly postupně spustit všechny audio nahrávky jednotlivých komponentů. Uživatel by měl mít možnost si audio kdykoliv ztlumit a používat aplikaci i bez něj.

1.3.1.14 FP-14 Editor kódu

Priorita: Should have

V rámci složitějších cvičení by měla aplikace obsahovat jednoduchý editor kódu, který bude umožňovat uživateli jednoduše psát a upravovat kód ve více záložkách.

V jednotlivých záložkách by mělo být možné uživateli zobrazit následující komponenty:

- cvičení na vyplnění chybějícího textu v kódu s vynechanými políčky
- cvičení na vyplnění chybějícího textu v kódu s vynechanými políčky pomocí nabídky slov
- cvičení na napsání nebo přepsání kódu ručně
- ukázkou výstupu kódu v podobě výpisu v terminálu
- ukázkou výstupu kódu v podobě webové stránky
- ukázkou výstupu kódu v podobě tabulky

1.3.1.15 FP-15 Bloky vysvětlení látky

Priorita: Must have

V rámci učení by měly být zobrazovány bloky vysvětlení látky, které mají za cíl uživateli látku krátce a srozumitelně vysvětlit před tím, než začne uživatel plnit jednotlivá cvičení. Celé vysvětlení látky bude probíhat formou audio výkladu, doplněného o různé animace, vizualizace a další komponenty. Tyto prvky budou sloužit jednak jako nástroj pro umožnění lepšího pochopení látky a jednak jako způsob zachycení a udržení pozornosti uživatele během výkladu. Komponenty by tak, na rozdíl od komponentů bloků cvičení, měly obsahovat i různé animace a přechody, aby byl výklad vizuálně poutavý.

V rámci bloků vysvětlení látky by mělo být možné uživateli zobrazovat následující komponenty:

- text s možnostmi zvýraznění určitých slov a s možnostmi vypsání určitých slov ve stylu kódu
- jednoduchou tabulku
- tabulku připomínající tabulku v databázi
- číselné a nečíselné seznamy
- obrázky ve formátech PNG, JPG, SVG a GIF
- diagramy tříd, ER diagramy, stavové diagramy, diagramy architektury, diagramy git větví a vývojové diagramy definované pomocí jazyka Mermaid⁶
- ukázkou kódu s animovanými přechody mezi různými kroky upravování kódu
- ukázkou terminálu s animovanými přechody mezi různými kroky zadávání příkazů do terminálu a vypisování jejich výstupů
- ukázkou webové stránky
- pozadí ve formě jednobarevného pozadí
- pozadí ve formě obrázku
- pozadí ve formě videa

Výklad látky by mělo být možné ovládat následujícími způsoby. Uživatel by měl mít možnost výklad zastavit, přeskocit na libovolnou část výkladu, nebo ho po přehrání znovu spustit.

Jelikož se může stát, že uživatel nemá vždy možnost poslouchat audio výklad, měl by mít možnost si zvuk vypnout. V takovém případě by mu měly být zobrazeny titulky, které budou vysvětlovat látku namísto audio výkladu.

⁶Dostupné na: <https://mermaid.js.org/>

Zásadní vlastností bloků s vysvětlením látky je jejich responzivita. Bloky s vysvětlením látky by měly být responzivní a mělo by tak být možné výklad sledovat jak na malých, tak na velkých displejích.

Dále by se měly bloky s vysvětlením látky přizpůsobovat tmavému resp. světlému režimu aplikace a zobrazovat tak všechny komponenty v příslušném režimu.

1.3.1.16 FP-16 Vysvětlení odpovědí ve cvičeních

Priorita: Should have

V rámci učení uživatel může vyplňovat jednotlivá cvičení. Po vyplnění cvičení by měla aplikace uživateli poskytnout možnost si zobrazit vysvětlení správné odpovědi. Díky tomu bude uživatel moci získat podrobnější vysvětlení, proč byla jeho odpověď ve cvičení špatná nebo správná.

1.3.1.17 FP-17 Nahlašování problému při učení

Priorita: Should have

Uživatel by měl mít při učení možnost u každého cvičení nebo vysvětlení látky nahlásit problém, který při dané aktivitě nastal. Při nahlašování by měl mít možnost zvolit, o který typ problému se jedná. K dispozici by měly být následující typy problémů:

- uživatel považuje svou odpověď ve cvičení za správnou, i přes to, že ji aplikace označila jako špatnou
- uživatel považuje svou odpověď ve cvičení za špatnou, i přes to, že ji aplikace označila jako správnou
- zadání cvičení je pro uživatele matoucí nebo nejasné
- vysvětlení látky je pro uživatele matoucí nebo nejasné
- audio nahrávka chybí nebo není v pořádku
- nastala jiná chyba

Uživatel by měl mít dále možnost detailněji popsat daný problém vlastními slovy.

Uživatelé s rolí korektora by také měli mít, kromě předchozích možností, k dispozici následující typy problémů:

- vysvětlení látky je třeba upravit
- strukturu cvičení je třeba upravit
- struktura celé lekce by měla být upravena

Korektoři, stejně jako běžní uživatelé, by také měli mít možnost u každé odpovědi detailněji popsat vlastními slovy daný problém.

Nahlášení problému od uživatelů s rolí korektora by mělo být označeno tak, aby bylo možné jej rozlišit od nahlášení problému běžných uživatelů. Role korektora bude nastavována uživatelům mimo frontend této aplikace. Tuto roli využijí zejména lidé zodpovědní za tvorbu a korekci jednotlivých kurzů.

1.3.1.18 FP-18 Statistiky o učení

Priorita: Should have

Aplikace by měla při učení a opakování nově sbírat data o tom, jak dlouho uživatel plnil jednotlivá cvičení, jak dlouho procházel vysvětlení látky a jakou úspěšnost měl při plnění jednotlivých cvičení.

Na konci učení nebo opakování by pak měla aplikace zobrazit uživateli přehled se statistikami. Na této stránce by měly být zobrazeny informace o tom, kolik času se uživatel učil, jakou měl úspěšnost při plnění cvičení, kolik konceptů se naučil a kolik bodů XP získal.

1.3.1.19 FP-19 Stav energie

Priorita: Should have

Uživatel by měl mít možnost si v aplikaci zobrazit svůj stav energie, který bude reprezentovat, kolik lekcí nebo opakování látky může uživatel ještě za daný den spustit.

Každým spuštěním lekce nebo opakování látky se stav energie sníží. Pokud uživatel všechny body energie spotřebuje, nebude mu umožněno se dále učit. Body energie se budou každý den obnovovat.

Uživatelé s premium účtem budou mít k dispozici neomezené množství energie. Učení a opakování látky jim tak nebude body energie snižovat.

Tento požadavek vychází z analýzy konkurence, ve které většina aplikací obsahovala určitou podobu omezení učení pro uživatele, kteří nemají premium účet. Funkce tak bude sloužit k motivaci uživatele, aby si vybudoval pravidelný návyk učení. Dále bude také sloužit k tomu, aby byl uživatel motivován si zakoupit premium účet.

1.3.1.20 FP-20 Registrace a přihlášení pomocí účtu Google

Priorita: Could have

Uživatel by měl mít možnost se registrovat a přihlásit do aplikace pomocí účtu Google. Tento požadavek vychází z analýzy konkurence, ve které většina konkurenčních aplikací tuto možnost nabízí.

1.3.1.21 FP-21 Stránka nastavení

Priorita: Must have

Původní aplikace obsahovala velice jednoduchou stránku nastavení, která umožňovala uživateli nastavit pouze několik základních funkcí. Jelikož bude do aplikace přidána řada nových funkcí, bude nutné tuto stránku zásadně změnit a rozšířit tak, aby umožňovala uživateli přehledně nastavovat potřebné funkce. Stránka nastavení by tak měla poskytovat uživateli následující možnosti:

- nastavení účtu uživatele, tedy uživatelského jména, emailu, hesla a avatara
- nastavení režimu vzhledu aplikace dle požadavku FP-22 Tmavý režim
- nastavení notifikací, tedy které notifikace uživatel chce dostávat a které ne
- odhlášení se ze zapsaného kurzu
- zobrazení základní nápovědy v aplikaci
- nahlášení problému nebo chyby v aplikaci
- zobrazení podmínek používání a dalších právních informací o používání aplikace

1.3.1.22 FP-22 Tmavý režim

Priorita: Should have

Rozhraní aplikace by mělo být dostupné ve dvou režimech, ve světlém a tmavém. Uživatel by měl mít možnost v nastavení zvolit, zda chce používat světlý režim, tmavý režim, nebo režim dle systémového nastavení. Rozhraní aplikace by se pak mělo zobrazovat uživateli v tomto režimu.

1.3.1.23 FP-23 Push Notifikace

Priorita: Should have

Aplikace by měla uživateli posílat push notifikace na základě událostí v systému. Tyto notifikace by se měly uživateli posílat v následujících případech:

- pokud se uživatel ještě neučil v daný den
- pokud se uživatel delší dobu v aplikaci nic neučil
- pokud uživatele začal sledovat jiný uživatel
- pokud bylo použito zmrazení streaku z požadavku FP-31 Zmrazení streaku
- pokud se blíží konec týdne a uživateli hrozí propadnutí do nižší ligy

V rámci této práce se budu zabývat pouze zprovozněním zobrazování push notifikací uživateli ve frontendové části aplikace. Podrobným fungováním odesílání push notifikací se bude zabývat kolega Bc. Jakub Vondráček jeho diplomové práci, která se zabývá backendovou částí této aplikace.

1.3.1.24 FP-24 Připomínání učení

Priorita: Could have

Uživatel by měl mít možnost si zapnout nebo vypnout připomínání učení. Pokud uživatel toto připomínání zapne, aplikace by mu měla posílat push notifikace, které ho upozorní, že se má daný den učit. Čas odeslání notifikace by se měl odvíjet od času, kdy se uživatel typicky učí. Možnost připomínání učení by měla být ve výchozím nastavení zapnutá.

Tato funkce je inspirována konkurenčními aplikacemi, které umožňují uživateli si nastavit připomínání učení.

1.3.1.25 FP-25 Systém zobrazování reklam

Priorita: Should have

V původní aplikaci existoval velice jednoduchý systém pro zobrazování reklam. Po úspěšném dokončení učení se uživateli zobrazila jednoduchá reklama formou banneru.

Nově by se měly zobrazovat různé velikosti reklamy v závislosti na velikosti dostupného místa na obrazovce uživatele. Reklamy by měla aplikace zobrazovat v následujících případech:

- po úspěšném dokončení učení
- po předčasném ukončení učení uživatelem
- pokud uživatel otevřel truhlu s odměnou a zvolil možnost získání dvojnásobku odměny při zhlédnutí reklamy

Tento požadavek vychází z analýzy konkurenčních řešení, kde většina aplikací zobrazuje reklamy v průběhu používání aplikace.

1.3.1.26 FP-26 Nový systém předplatného

Priorita: Must have

V původní aplikaci měl uživatel možnost si zaplatit předplatné a tím získat přístup k pokročilejším funkcím aplikace. Jelikož se aplikace v rámci této práce zásadně změní a budou přidány nové funkce, je s těmito změnami třeba přizpůsobit i systém předplatného. Uživatelé by měli být v rozdělení do dvou skupin.

Uživatelé bez předplatného budou mít přístup k základním kurzům a kariérním cestám v aplikaci. Uživatelé budou mít také přístup k opakování látky. Při učení i opakování budou ovšem omezeni systémem energie popsaném v požadavku FP-19 Stav energie. Těmto uživatelům budou dále zobrazovány v aplikaci reklamy a nebude jim umožněno získat certifikáty o dokončení kurzů a kariérních cest.

Uživatelé s předplatným budou mít neomezený přístup ke všem kurzům a kariérním cestám aplikace. Při učení a chytrém opakování nebudou nijak omezeni systémem energie. Těmto uživatelům nebudou zobrazovány reklamy a budou moci získávat certifikáty o dokončení kurzů a kariérních cest.

V rámci tohoto požadavku bude dále třeba zajistit lepší informovanost uživatelů o tom, které funkce jsou dostupné zdarma a které funkce jsou dostupné pouze s aktivním předplatným. Pokud uživatel bez předplatného například chce využít funkci, která je dostupná pouze uživatelům s aktivním předplatným, měla by mu aplikace tuto skutečnost oznámit a odkázat ho na možnost zaplatit si předplatné.

1.3.1.27 FP-27 Zkušební verze předplatného

Priorita: Should have

Aplikace by měla umožnit uživateli spustit zkušební verzi předplatného na určitou dobu zdarma. Tuto možnost by měli mít pouze uživatelé bez premium účtu, kteří zkušební verzi v posledním roce nevyužili.

Pro spuštění zkušební verze premium účtu bude po uživateli vyžadováno, aby zadal své platební údaje. Před zadáním údajů by měla aplikace uživateli vysvětlit, jak zkušební doba funguje.

Několik dní před koncem zkušební doby by mělo být uživateli odesláno oznámení, že mu bude zkušební doba předplatného končit. Toto oznámení bude zobrazeno jednak jako oznámení přímo v aplikaci podle požadavku FP-32 Rozšířená oznámení v aplikaci a jednak jako oznámení ve formě e-mailu.

Uživatel se bude moci kdykoliv během zkušební doby od předplatného odhlásit. Pokud tak neučiní do konce zkušební doby, bude mu automaticky spuštěno plnohodnotné předplatné.

Aplikace by také měla pravidelně nabízet zkušební verzi předplatného uživatelům, kteří zkušební verzi v posledním roce nevyužili, pomocí rozšířených oznámení popsaných v požadavku FP-32 Rozšířená oznámení v aplikaci.

1.3.1.28 FP-28 Přizpůsobení avatara uživatele

Priorita: Should have

Uživatel by měl mít možnost si přizpůsobit svého avatara, tedy postavu na profilovém obrázku, pomocí položek, které zakoupil v obchodě za virtuální měnu. To se liší od původní aplikace, kde měl uživatel pouze možnost změnit obrázek na svém profilu.

Uživatel by měl mít možnost si u svého avatara přizpůsobit barvu pleti, očí, barvu očí, výraz v obličeji, vlasy, barvu vlasů, doplňky, barvu brýlí, vousy, barvu vousů, pokrývku hlavy, oblečení a barvu pozadí profilového obrázku.

Tento požadavek vychází ze základních principů gamifikace, konkrétně z principu vlastnictví [15]. Tento princip říká, že mohou být uživatelé více motivováni používat aplikaci, pokud mají pocit, že v rámci aplikace něco vlastní.

Příkladem toho je právě přizpůsobování avatara, díky kterému si uživatelé mohou vytvořit vlastní postavu a vytvořit si tak s ní osobní pouto. Přizpůsobování avatara tak může mít za následek častější a pravidelnější používání aplikace [16]. Tento princip využívá například i velice známá vzdělávací aplikace Duolingo⁷, kterou jsem analyzoval v rámci mé bakalářské práce [2] a která podobnou funkci přizpůsobení avatara nabízí.

1.3.1.29 FP-29 Rozšíření obchodu a odemykání položek

Priorita: Could have

V aplikaci již existuje stránka obchodu, v němž si uživatel může zakoupit pomocí virtuální měny profilové obrázky. Vzhledem k novým požadavkům aplikace je třeba tento obchod rozšířit tak, aby bylo možné si zakoupit položky, které jsou popsány v požadavku FP-28 Přizpůsobení avatara uživatele.

Dále by v obchodu měla být zavedena nová funkce odemykání položek. Některé položky by měly být dostupné ke koupi pouze pokud uživatel dosáhl určitého ocenění v aplikaci. Pokud uživatel ještě nedosáhl daného ocenění, měly by se položky uživateli zobrazovat jako zamčené. U těchto položek by měl mít uživatel možnost si zobrazit informace o tom, co musí splnit, aby danou položku odemknul.

1.3.1.30 FP-30 Uživatelský profil

Priorita: Should have

V původní aplikaci se již nachází stránka s uživatelským profilem, která obsahuje základní informace o daném uživateli. Po zavedení nových funkcí do aplikace je třeba tuto stránku aktualizovat a upravit tak, aby kromě již zobrazených informací o uživateli přehledně zobrazovala i následující informace:

- avatara uživatele, kterého si bude uživatel moci přizpůsobit dle požadavku FP-28 Přizpůsobení avatara uživatele
- ligu, ve které se uživatel nachází, podle požadavku FP-34 Žebříček uživatelů
- přehled zapsaných kurzů a kariérních cest uživatele s pokrokem daného uživatele
- přehled ocenění, která uživatel v rámci aplikace získal

1.3.1.31 FP-31 Zmrazení streaku

Priorita: Should have

Aplikace uživateli zobrazuje tzv. streak, neboli skóre, které reprezentuje počet dní v řadě, v rámci kterých uživatel splnil alespoň jednu lekci nebo opakování látky.

⁷Dostupné na: <https://www.duolingo.com/>

Uživatel by nově měl mít v obchodě s virtuální měnou možnost si zakoupit do zásoby několik položek tzv. zmrazení streaku. Pokud uživatel bude mít zakoupenou jednu nebo více těchto položek, a nesplní daný den žádné učení nebo opakování, jeho streak se tím nezruší a jedna tato položka se spotřebuje. Díky tomu bude uživatel moci zachovat svůj streak, i když se některý den zapomene učit.

Tato funkce vychází z analýzy vzdělávací aplikace Duolingo, kterou jsem se zabýval v rámci analýzy konkurence v mé bakalářské práci [2].

1.3.1.32 FP-32 Rozšířená oznámení v aplikaci

Priorita: Should have

V původní aplikaci existoval jednoduchý systém, který umožňoval uživateli zobrazovat oznámení při otevření hlavní stránky aplikace. Tento systém bude třeba z důvodu přidání nových funkcí rozšířit tak, aby informoval uživatele, pokud v aplikaci nastala některá z následujících událostí:

- aplikace automaticky použila zmrazení streaku z požadavku FP-31 Zmrazení streaku
- aplikace resetovala streak uživatele
- uživatel dosáhl nové významné hranice streaku, například 100 dní
- uživatel přesunut do vyšší ligy z požadavku FP-34 Žebříček uživatelů
- uživatel přesunut do nižší ligy z požadavku FP-34 Žebříček uživatelů
- uživatel dosáhl nového úspěchu
- uživatel získal truhlu s odměnou z požadavku FP-33 Truhla s odměnou
- uživatel nemá zapnuté notifikace z požadavku FP-23 Push Notifikace
- uživatel bez premium účtu ještě nevyzkoušel za dané období zkušební verzi předplatného z požadavku FP-27 Zkušební verze předplatného
- pokud bude uživateli za několik dní končit zkušební verze předplatného z požadavku FP-27 Zkušební verze předplatného

1.3.1.33 FP-33 Truhla s odměnou

Priorita: Should have

Po dokončení učení by se uživateli měla zobrazit stránka s obrázkem truhly, kterou bude moci otevřít. Po otevření truhly by měl uživatel obdržet náhodné množství virtuální měny, jako odměnu za jeho učení. Virtuální měnu bude moci následně využít k nákupu položek v obchodě s virtuální měnou dle požadavku FP-29 Rozšíření obchodu a odemykání položek.

Po otevření truhly by měla být uživateli nabídnuta možnost získat dvojnásobek této odměny, pokud uživatel zhlédne reklamu. Pokud uživatel tuto možnost zvolí, měla by mu být zobrazena reklama a následně by měl obdržet dvojnásobek získané virtuální měny.

1.3.1.34 FP-34 Žebříček uživatelů

Priorita: Could have

V aplikaci by měly být zavedeny žebříčky uživatelů, které by měly fungovat následujícím způsobem. Každý uživatel bude přiřazen do tzv. ligy učení. Každá liga bude reflektovat, jak často se daný uživatel učí. Ve vyšších ligách se tak budou nacházet uživatelé, kteří se učí častěji.

Začátkem každého týdne by měl být uživatel zařazen do skupiny s omezeným počtem uživatelů, kteří se nachází ve stejné lize. V rámci této skupiny budou uživatelé průběžně řazeni podle počtu bodů XP, které získali v daném týdnu. Skupina tak bude tvořit žebříček uživatelů seřazený podle toho, kolik lekcí a opakování jednotliví uživatelé v daném týdnu dokončili.

Na konci každého týdne by měl být uživatel přesunut do vyšší ligy, pokud se nacházel v žebříčku uživatelů na prvních několika pozicích. Pokud se uživatel nacházel v žebříčku na posledních několika pozicích, měl by být přesunut do nižší ligy. Pokud se uživatel nacházel v žebříčku mezi těmito pozicemi, měl by zůstat v aktuální lize. Uživatel by měl mít možnost si kdykoliv zobrazit žebříček a svou pozici v něm.

Tento požadavek vychází z analýzy konkurenčních řešení, zejména z analýzy aplikací Mimo a Sololearn, v nichž jsou podobné ligy implementovány.

1.3.1.35 FP-35 Sdílení pokroku mezi uživateli

Priorita: Should have

V aplikaci existuje základní funkce pro sdílení pokroku mezi uživateli. V rámci této funkce si může uživatel zobrazit stránku se zprávami o pokroku ostatních uživatelů, které daný uživatel sleduje. Uživatel následně může reagovat na tyto zprávy pomocí emotikonů. Tato funkce je ovšem příliš jednoduchá a je třeba ji přizpůsobit novým funkcím aplikace.

Nově by stránka měla zobrazovat zprávy o následujících událostech týkajících se pokroku sledovaných uživatelů:

- zpráva o tom, že uživatel dosáhl významné hranice streaku
- zpráva o tom, že uživatel získal určité ocenění
- zpráva o tom, že uživatel postoupil do vyšší ligy učení
- zpráva o tom, že uživatel dokončil některý kurz nebo kariérní cestu

1.3.2 Nefunkční požadavky

Po specifikaci funkčních požadavků jsem dále definoval nefunkční požadavky. Nefunkční požadavky definují omezení kladená na daný systém a vlastnosti, které by systém měl splňovat. Tyto požadavky se týkají například spolehlivosti, použitelnosti, výkonu a rozšiřitelnosti systému. [13, s. 4-5]

Nefunkčním požadavkům původní aplikace se věnoval kolega Bc. Jan Hlaváč v jeho bakalářské práci [1] a proto je zde nebudu opakovat. Zaměřím se tedy zejména na nové požadavky, které se týkají frontendové části aplikace.

Na základě předchozí analýzy jsem tak definoval následující nefunkční požadavky. Podobně jako u funkčních požadavků jsem specifikoval priority jednotlivých požadavků. Dále jsem požadavky rozdělil do kategorií dle jejich charakteru.

1.3.2.1 NP-1 Odstranění přebytečných funkcí

Priorita: Must have

Kategorie: Udržovatelnost systému

Jelikož se nově vznikající aplikace bude zaměřovat na výuku programování, bude třeba odstranit funkce týkající se původní domény výuky jazyků. Tyto funkce jsem specifikoval v rámci analýzy stávajícího řešení.

1.3.2.2 NP-2 Nový design uživatelského rozhraní

Priorita: Must have

Kategorie: Použitelnost systému

Jelikož aplikace mění své zaměření z výuky jazyků na výuku programování, je třeba kompletně změnit a přizpůsobit i celý vzhled uživatelského rozhraní. Měl by tak být vytvořen kompletně nový návrh uživatelského rozhraní, který se bude zahrnovat jak nové, tak i existující funkce aplikace.

1.3.2.3 NP-3 Zpřehlednění navigace v aplikaci

Priorita: Should have

Kategorie: Použitelnost systému

Aplikace by měla uživateli poskytovat přehlednější a jednodušší způsob navigace mezi jednotlivými stránkami. Z výsledků uživatelského testování v mé bakalářské práci [2], která se zabývala původní aplikací, vyplynulo, že mají uživatelé problém s navigací mezi jednotlivými stránkami aplikace, zejména mezi jednotlivými sekcemi kurzů. Jelikož bude do aplikace také přidána řada nových funkcí, bude třeba navigaci aktualizovat tak, aby se uživatel mohl snadno orientovat v aplikaci a využívat všechny dostupné funkce.

1.3.2.4 NP-4 Vytvoření knihovny komponentů

Priorita: Should have

Kategorie: Rozšiřitelnost systému

Frontend aplikace využívá různé komponenty uživatelského rozhraní. Tyto komponenty by měly být přesunuty do samostatné knihovny tak, aby je bylo možné využít i v dalších částech projektu, jako například na webových stránkách nebo v systému pro správu obsahu aplikace.

Tato knihovna komponentů by měla stavět na již existujících komponentech a měla by využívat již zavedené technologie, tedy jazyk TypeScript a knihovny React, Storybook a Material UI.

1.3.2.5 NP-5 Zvýšení udržitelnosti kódu

Priorita: Should have

Kategorie: Udržitelnost systému

Na základě analýzy stávající aplikace jsem se rozhodl přizpůsobit kód aplikace tak, aby byl lépe udržitelný a rozšiřitelný. Konkrétní změny, které bude třeba provést, jsem popsal v rámci analýzy stávajícího řešení.

1.3.2.6 NP-6 Zavedení automatizovaných testů

Priorita: Should have

Kategorie: Použitelnost systému

Do frontendové části aplikace by měly být zavedeny automatizované testy s cílem zajištění správného fungování aplikace a zvýšení její kvality a použitelnosti.

Tento požadavek vychází jednak z analýzy existujícího řešení a jednak z návrhu úprav do budoucna, který jsem popsal v rámci mé bakalářské práce [2], která se věnovala původní aplikaci.

1.3.2.7 NP-7 Automaticky generované třídy API

Priorita: Should have

Kategorie: Udržitelnost systému

V kódu frontendové části aplikace by měly být automaticky generovány třídy reprezentující backendové API na základě OpenAPI specifikace [17], která je poskytována backendovou částí aplikace.

Požadavek vychází z analýzy existujícího řešení. Cílem tohoto požadavku je zjednodušit implementaci a zajistit konzistenci mezi frontendovou a backendovou částí aplikace.

1.3.2.8 NP-8 Animace a efekty

Priorita: Could have

Kategorie: Použitelnost systému

Do aplikace by měly být přidány základní animace a efekty s cílem zlepšit uživatelskou zkušenost a udržet pozornost uživatelů. Uživatelské rozhraní by tak mělo působit vizuálně poutavějším dojmem a mělo by výrazněji reagovat na interakce s uživatelem.

Tento požadavek vychází zejména z analýzy konkurenčních řešení, ve které konkurenční aplikace Mimo nebo Scrimba využívají různé animace a efekty.

1.3.2.9 NP-9 Nasazení aplikace jako PWA

Priorita: Should have

Kategorie: Použitelnost systému

Aplikace by měla být přizpůsobena tak, aby bylo možné ji sestavit a nainstalovat jako PWA, neboli jako progresivní webovou aplikaci. V důsledku toho by mělo být možné aplikaci stáhnout a nainstalovat na zařízení s operačními systémy Windows, Android či iOS. K aplikaci by tak mělo být možné přistupovat buď na webových stránkách, nebo přímo na zařízení uživatele.



Kapitola 2

Návrh

Výstupem předchozí kapitoly byl seznam požadavků, které definují jednotlivé funkce a vlastnosti aplikace. Cílem této kapitoly bude na základě těchto požadavků provést návrh a podrobnější specifikaci jednotlivých funkcí a vytvořit tak podrobné podklady pro samotný vývoj aplikace.

V rámci této kapitoly se nejprve zaměřím na podrobnou specifikaci jednotlivých funkcí aplikace pomocí případů užití a metodiky Behavior-Driven Development [18] s využitím jazyka Gherkin [19]. Dále se budu věnovat návrhu samotného vzhledu uživatelského rozhraní včetně výběru palety barev, písma, obrázků, animací a efektů. Následně s využitím tohoto návrhu vytvořím *high fidelity* prototyp uživatelského rozhraní aplikace.

Díky detailnímu návrhu uživatelského rozhraní a podrobné specifikaci jednotlivých funkcí tak bude aplikace připravena na samotný vývoj, kterému se budu věnovat v následující kapitole.

2.1 Návrh funkcí

V této sekci se budu věnovat podrobnější specifikaci funkcí navrhované aplikace. Nejprve se zaměřím na popis jednotlivých aktérů, kteří budou s aplikací interagovat. Následně na základě funkčních požadavků sepišu seznam případů užití, které budou podrobněji specifikovat, jak mají jednotlivé části aplikace fungovat z pohledu uživatele. Nakonec pro jednotlivé případy užití sepišu podrobnou specifikaci jejich scénářů v jazyce Gherkin. Tyto scénáře budou sloužit jako zadání pro implementaci, ale také jako podklad pro spouštění automatizovaných testů pomocí nástroje Cucumber. Jelikož scénáře přesně popisují, jak se aplikace skutečně chová, bude je také možné v budoucnu využít jako podklad pro AI agenty, kteří budou moci například generovat nové funkce, testy nebo například uživatelskou dokumentaci aplikace.

2.1.1 Popis aktérů

V rámci specifikace funkcí je třeba popsat tzv. aktéry, kteří reprezentují role, které mohou s aplikací interagovat a vykonávat v ní určité akce [13, s. 21-22]. Tito aktéři budou využiti jak v popisu případů užití, tak ve specifikaci funkcí pomocí jazyka Gherkin. Na základě požadavků aplikace jsem identifikoval následující aktéry.

2.1.1.1 Uživatel

Uživatel je osoba, která využívá aplikaci k učení se programování. Mezi jeho hlavní aktivity patří například zapisování si kurzů, učení se programování, opakování látky, interakce s ostatními uživateli a interakce s prvky gamifikace v aplikaci.

2.1.1.2 Nepřihlášený uživatel

Nepřihlášený uživatel je osoba, která v aplikaci není přihlášená a nemá tak přístup k většině funkcí aplikace. Může se však přihlásit nebo se registrovat v aplikaci.

2.1.1.3 Premium uživatel

Premium uživatel je osoba, která má aktivní předplatné nebo jeho zkušební verzi. Premium uživatel může využívat všechny funkce aplikace jako běžný uživatel. Kromě těchto akcí může vykonávat i další akce, ke kterým nemá běžný uživatel přístup.

2.1.1.4 Korektor

Korektor je osoba zodpovědná za kontrolu kvality výukového obsahu, tedy například za kontrolu lekcí, cvičení a vysvětlení látky. Funkce aplikace může využívat ve stejném rozsahu jako premium uživatel. Kromě toho může využívat podrobnější nahlašování chyb při učení.

2.1.2 Specifikace případů užití

Jako další krok při návrhu funkcí jsem se rozhodl sepsat případy užití. Případy užití představují způsob, jak specifikovat funkční chování systému z pohledu jeho aktérů. Každý případ užití popisuje konkrétní cíl aktéra a interakce se systémem, které k dosažení tohoto cíle vedou. Díky tomuto přístupu je tak možné detailněji formulovat požadavky a identifikovat okrajové případy, které je třeba při návrhu a implementaci ošetřit. Případy užití tak mohou být užitečné pro vývojáře, protože jim poskytnou detailnější popis toho, jak má daná část systému fungovat. Dále mohou být využity testery například jako podklad pro vytváření automatizovaných testů. Díky tomu, že bývají případy užití

specifikovány pomocí přirozeného jazyka, mohou být čitelné i pro osoby, které nejsou technicky zaměřené. Celkově tak tvoří přesnější specifikaci systému, na základě které lze následně postupovat při návrhu a implementaci samotné aplikace. [13]

V průběhu let se vyvinulo několik způsobů, jak k případům užití přistupovat a jak je specifikovat. Základy specifikace případů užití položil Ivar Jacobson v roce 1986. Alistair Cockburn [20] následně popsal metodiku pro psaní podrobnějších případů užití. Přesto však prosazoval názor, že by forma specifikace případů užití měla odpovídat potřebám a rozsahu konkrétního projektu. Další techniku specifikace případů užití také popsal například Martin Fowler, který prosazoval názor, že neexistuje jeden standardní způsob, jak specifikovat případy užití, a že formát a rozsah případů užití by měly být zvoleny dle potřeb konkrétního projektu. [21, s. 22-27]

V této práci jsem se tak rozhodl vycházet z práce Alistaira Cockburna [20] a specifikovat případy užití způsobem, který bude užitečný pro vývoj této konkrétní aplikace. Pro každý případ užití tak budu specifikovat jeho identifikátor, název, popis, hlavního aktéra, hlavní scénář a případné alternativní scénáře.

Pro specifikaci scénářů jsem se rozhodl využít jazyk Gherkin, který přináší několik zásadních výhod, kterým se budu věnovat v dalších částech této kapitoly. Scénáře se v jazyce Gherkin specifikují přímo v repozitáři aplikace v souborech s příponou `.feature` [19]. Proto jsem se rozhodl v tomto textu popsat jednotlivé scénáře pouze stručným popisem a detailně je pak specifikovat přímo v repozitáři aplikace. Díky tomuto přístupu tak budu moci využít jak výhod specifikace případů užití, tak i výhod jazyka Gherkin.

V rámci specifikace případů užití jsem tak sestavil následující seznam případů užití.

2.1.2.1 UC-1 Volba přihlášení nebo registrace

Popis:	Uživatel si zobrazí úvodní stránku aplikace a zvolí, zda se chce přihlásit nebo registrovat.
Požadavek:	FP-1 Úvodní stránka aplikace
Hlavní aktér:	Nepřihlášený uživatel
Hlavní scénář:	Nepřihlášený uživatel otevře aplikaci a aplikace mu zobrazí úvodní stránku. Uživatel zvolí možnost přihlášení a aplikace ho přesměruje na stránku přihlášení.
Alternativní scénář 1:	Nepřihlášený uživatel otevře aplikaci a aplikace mu zobrazí úvodní stránku. Uživatel zvolí možnost registrace a aplikace ho přesměruje na stránku registrace.
Alternativní scénář 2:	Přihlášený uživatel otevře aplikaci na úvodní stránce. Aplikace ho automaticky přesměruje na hlavní stránku v aplikaci.

2.1.2.2 UC-2 Vyplnění úvodního dotazníku

Popis:	Uživatel vyplní úvodní dotazník zaměřený na jeho cíle a zkušenosti s programováním. Aplikace uživateli na základě dotazníku doporučí kurzy a kariérní cesty, které si může zapsat.
Požadavek:	FP-2 Úvodní dotazník a doporučování kurzů
Hlavní aktér:	Uživatel
Hlavní scénář:	Po registraci uživatele aplikace zobrazí dotazník. Uživatel vyplní svoji úroveň programování, témata, která se chce učit a cíl, kterého chce dosáhnout. Aplikace zobrazí uživateli stránku, která ujistí uživatele, že daného cíle může s pomocí aplikace dosáhnout. Následně uživateli zobrazí seznam doporučených kurzů a kariérních cest. Uživatel vybere jeden z doporučených kurzů. Následně vybere svůj denní cíl. Aplikace uživateli oznámí, že byl úvodní dotazník dokončen. Uživatel zvolí možnost pokračovat a aplikace ho přesměruje na hlavní stránku aplikace.
Alternativní scénář 1:	Uživatel při průchodu hlavním scénářem nevybere kurz ze seznamu doporučených kurzů, ale zvolí možnost zobrazení všech kurzů v aplikaci. Aplikace mu zobrazí přehled všech dostupných kurzů. Uživatel vybere jeden z kurzů a pokračuje dle hlavního scénáře vyplněním svého denního cíle.
Poznámka:	Zapsání kariérní cesty se od zápisu kurzu nijak neliší a bude fungovat analogicky.

2.1.2.3 UC-3 Zobrazení přehledu kurzů a kariérních cest

Popis:	Uživatel si zobrazí stránku s přehledem kurzů a kariérních cest. Kliknutím na kurz nebo kariérní cestu se přesune na jeho resp. její stránku.
Požadavek:	FP-3 Prohlížení a doporučování kurzů
Hlavní aktér:	Uživatel
Hlavní scénář:	Aplikace zobrazí uživateli stránku s přehledem kurzů a kariérních cest. Na této stránce aplikace zobrazí krátký přehled doporučených kurzů a kariérních cest a dále seznam všech dostupných kurzů a kariérních cest rozřazených do kategorií. Uživatel se vybráním kurzu přesune na jeho stránku.
Alternativní scénář 1:	Při průchodu hlavním scénářem uživatel nevybere kurz ale kariérní cestu a přesune se tak na stránku kariérní cesty.

2.1.2.4 UC-4 Zobrazení přehledu doporučených kurzů a kariérních cest

Popis:	Uživatel si na stránce přehledu kurzů a kariérních cest zobrazí přehled všech doporučených kurzů a kariérních cest. Kliknutím na kurz nebo kariérní cestu se pak přesune na jeho resp. její stránku.
Požadavek:	FP-3 Prohlížení a doporučování kurzů
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se ze stránky přehledu kurzů a kariérních cest přesune na stránku přehledu doporučených kurzů a kariérních cest. Aplikace mu zobrazí seznam všech doporučených kurzů a kariérních cest. Uživatel se vybráním kurzu přesune na jeho stránku.
Alternativní scénář 1:	Při průchodu hlavním scénářem uživatel nevybere kurz ale kariérní cestu a přesune se tak na stránku kariérní cesty.

2.1.2.5 UC-5 Vyplnění dotazníku pro úpravu doporučení kurzů a kariérních cest

Popis:	Uživatel na stránce přehledu všech doporučených kurzů a kariérních cest vyplní dotazník pro úpravu doporučení kurzů a kariérních cest.
Požadavek:	FP-3 Prohlížení a doporučování kurzů
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel zvolí možnost úpravy doporučených kurzů a kariérních cest a aplikace ho přesune na stránku dotazníku. V dotazníku uživatel vyplní jeho úroveň programování, témata, která se chce učit a cíl, kterého chce dosáhnout. Aplikace mu zobrazí informaci, že uživatel dokončil dotazník. Uživatel zvolí možnost zobrazení nových doporučení a aplikace ho tak přesune zpět na stránku doporučených kurzů a kariérních cest.

2.1.2.6 UC-6 Zobrazení stránky kurzu a zápis do kurzu

Popis:	Uživatel si zobrazí stránku kurzu a přihlásí se do něj.
Požadavek:	FP-4 Stránka kurzu, FP-26 Nový systém předplatného
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se přesune na stránku daného kurzu, aplikace mu zobrazí přehled kurzu včetně jeho popisu a přehledu kapitol kurzu. Uživatel zvolí možnost zapsání se do kurzu. Aplikace uživatele zapíše do kurzu a přesměruje ho na stránku učení se první lekce kurzu.
Alternativní scénář 1:	Uživatel bez aktivního předplatného se přesune na stránku kurzu, který je dostupný pouze pro uživatele s premium účtem, a zvolí možnost zapsání do kurzu. Aplikace uživatele přesměruje na stránku s přehledem výhod premium účtu a informuje ho, že kurz je dostupný pouze s aktivním předplatným.

2.1.2.7 UC-7 Zobrazení stránky kurzu a pokračování v učení

Popis:	Uživatel si zobrazí stránku kurzu, který má již zapsaný, a zvolí možnost pokračovat v učení.
Požadavek:	FP-4 Stránka kurzu
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se přesune na stránku kurzu, který má zapsaný, a aplikace mu zobrazí přehled kurzu včetně popisu a přehledu kapitol kurzu. Uživatel zvolí možnost pokračovat v učení. Aplikace přesměruje uživatele na stránku učení, kde mu zobrazí následující lekci, kterou by se měl uživatel v rámci daného kurzu učit.

2.1.2.8 UC-8 Spuštění lekce na stránce kapitoly kurzu

Popis:	Uživatel se ze stránky kurzu přesune na stránku kapitoly kurzu a spustí danou lekci.
Požadavek:	FP-5 Stránka kapitoly kurzu
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se nachází na stránce daného kurzu, zvolí některou kapitolu kurzu a aplikace ho přesměruje na stránku dané kapitoly, kde dále zobrazí seznam lekcí rozdělených do sekcí. Uživatel vybere jednu z lekcí a aplikace ho přesměruje na stránku učení se dané lekce.
Poznámka 1:	Pokud uživatel ještě nemá kurz zapsaný a spustí lekci, aplikace ho automaticky zapíše do kurzu a následně ho přesměruje na stránku učení dané lekce.
Poznámka 2:	Uživatel bude moci v kurzu spustit libovolnou lekci neohledně na to, které lekce již dříve dokončil.

2.1.2.9 UC-9 Zobrazení certifikátu o dokončení kurzu

Popis:	Uživatel s premium účtem si zobrazí certifikát o dokončení daného kurzu.
Požadavek:	FP-6 Certifikáty o dokončení kurzu, FP-26 Nový systém předplatného
Hlavní aktér:	Premium uživatel
Hlavní scénář:	Premium uživatel, který dokončil daný kurz, se přesune na stránku daného kurzu a zvolí možnost zobrazit certifikát. Aplikace zobrazí uživateli stránku s certifikátem, který uživatel získal za dokončení daného kurzu.
Alternativní scénář 1:	Premium uživatel, který daný kurz ještě nedokončil, se přesune na stránku daného kurzu a zvolí možnost zobrazit certifikát. Aplikace zobrazí uživateli informaci, že musí nejprve dokončit kurz, aby mohl certifikát zobrazit.
Alternativní scénář 2:	Uživatel, který nemá premium účet, se přesune na stránku daného kurzu a zvolí možnost zobrazit certifikát. Aplikace uživatele přesměruje na stránku s přehledem výhod premium účtu a informuje ho, že daná funkce je dostupná pouze s aktivním předplatným.

2.1.2.10 UC-10 Odhlášení se z kurzu

Popis:	Uživatel si zobrazí stránku pro správu zapsaných kurzů a zvolí možnost odhlášení se z daného kurzu.
Požadavek:	FP-8 Správa zapsaných kurzů a cest učení
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se přesune na stránku pro správu zapsaných kurzů, aplikace mu zobrazí seznam zapsaných kurzů. Uživatel zvolí kurz, ze kterého se chce odhlásit. Aplikace uživateli zobrazí upozornění, že se daná akce nedá vrátit. Uživatel potvrdí akci odhlášení se z kurzu. Aplikace uživatele odhlásí z kurzu a přesměruje ho na stránku přehledu zapsaných kurzů.

2.1.2.11 UC-11 Zápis do kariérní cesty

Popis:	Uživatel si zobrazí stránku kariérní cesty a zapíše si danou kariérní cestu.
Požadavek:	FP-7 Cesty učení, FP-26 Nový systém předplatného
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se přesune na stránku kariérní cesty, aplikace zobrazí přehled kariérní cesty včetně jejího popisu a přehledu kurzů, které jsou v kariérní cestě obsaženy. Uživatel zvolí možnost zapsání se do kariérní cesty. Aplikace uživatele zapíše do kariérní cesty a přesměruje ho na stránku učení se první lekce prvního kurzu v rámci dané kariérní cesty.
Alternativní scénář 1:	Uživatel bez aktivního předplatného se přesune na stránku kariérní cesty, která je dostupná pouze pro uživatele s premium účtem, a zvolí možnost zapsání se do kariérní cesty. Aplikace uživatele přesměruje na stránku s přehledem výhod premium účtu a informuje ho, že kariérní cesta je dostupná pouze s aktivním předplatným.

2.1.2.12 UC-12 Zobrazení stránky kariérní cesty a pokračování v učení

Popis:	Uživatel si zobrazí stránku kariérní cesty, kterou má již zapsanou, a zvolí možnost pokračovat v učení se kariérní cesty.
Požadavek:	FP-7 Cesty učení
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se přesune na stránku kariérní cesty, kterou má již zapsanou, a aplikace mu zobrazí přehled kariérní cesty včetně jejího popisu a přehledu kurzů, které jsou v kariérní cestě obsaženy. Uživatel zvolí možnost pokračovat v učení se kariérní cesty. Aplikace přesměruje uživatele na stránku učení, kde mu zobrazí následující lekci, kterou by se měl uživatel učit v rámci dané kariérní cesty.

2.1.2.13 UC-13 Zobrazení certifikátu o dokončení dané kariérní cesty

Popis:	Uživatel s premium účtem si zobrazí stránku kariérní cesty a zvolí možnost zobrazit certifikát o dokončení kariérní cesty.
Požadavek:	FP-7 Cesty učení, FP-26 Nový systém předplatného
Hlavní aktér:	Premium uživatel
Hlavní scénář:	Premium uživatel, který dokončil danou kariérní cestu, se přesune na stránku dané kariérní cesty a zvolí možnost zobrazit certifikát. Aplikace zobrazí uživateli stránku s certifikátem.
Alternativní scénář 1:	Premium uživatel, který danou kariérní cestu ještě nedokončil, se přesune na stránku dané kariérní cesty a zvolí možnost zobrazit certifikát. Aplikace zobrazí uživateli informaci, že musí nejprve dokončit kariérní cestu, aby mohl certifikát zobrazit.
Alternativní scénář 2:	Uživatel, který nemá premium účet, se přesune na stránku dané kariérní cesty a zvolí možnost zobrazit certifikát. Aplikace uživatele přesměruje na stránku s přehledem výhod premium účtu a informuje ho, že je daná funkce dostupná pouze s aktivním předplatným aplikace.

2.1.2.14 UC-14 Zobrazení studijního plánu

Popis:	Uživatel si zobrazí stránku se svým studijním plánem, tedy přehledem všech zapsaných kurzů a jejich aktuálního režimu učení.
Požadavek:	FP-9 Studijní plán
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel se přesune na stránku svého studijního plánu. Aplikace mu zobrazí seznam zapsaných kurzů a jejich aktuální režimy učení.

2.1.2.15 UC-15 Změna režimu učení kurzu ve studijním plánu

Popis:	Uživatel změní režim učení kurzu v jeho studijním plánu. Uživatel může vybrat mezi režimy učení, opakování a pozastavení kurzu.
Požadavek:	FP-9 Studijní plán
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel si zobrazí stránku studijního plánu a změní režim učení jednoho z kurzů. Aplikace změnu uloží a začne režim kurzu zohledňovat při doporučování látky k učení a opakování.

2.1.2.16 UC-16 Spuštění lekce podle studijního plánu

Popis:	Uživatel na hlavní stránce aplikace spustí nadcházející lekci podle jeho studijního plánu.
Požadavek:	FP-10 Učení se a opakování podle studijního plánu
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře hlavní stránku aplikace. Aplikace uživateli zobrazí náhled nadcházející lekce podle studijního plánu, uživatel spustí tuto lekci a aplikace ho přesměruje na stránku učení, kde mu zobrazí látku z dané lekce.
Alternativní scénář 1:	Uživatel otevře hlavní stránku, ale aplikace nenalezne žádnou nadcházející lekci, například protože má uživatel všechny lekce zapsaných kurzů dokončené. Aplikace zobrazí informaci, že nenalezla žádnou lekci k učení a zobrazí možnost zapsání si nového kurzu. Uživatel zvolí možnost pro zapsání nového kurzu a aplikace ho přesměruje na stránku s přehledem kurzů.

2.1.2.17 UC-17 Spuštění opakování podle studijního plánu

Popis:	Uživatel na hlavní stránce aplikace spustí opakování látky, kterou aplikace doporučí na základě studijního plánu a na základě toho, jak si uživatel naučenou látku pamatuje.
Požadavek:	FP-10 Učení se a opakování podle studijního plánu
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře hlavní stránku aplikace a zvolí možnost opakování. Aplikace podle studijního plánu a historie učení vybere vhodnou látku k zopakování ze zapsaných kurzů, které jsou v režimu učení nebo opakování. Aplikace zobrazí informaci, ze kterého kurzu látku k zopakování vybrala a dále zobrazí možnost spustit opakování. Uživatel spustí opakování a aplikace ho přesměruje na stránku opakování.
Alternativní scénář 1:	Uživatel otevře hlavní stránku aplikace a zvolí možnost opakování. Aplikace nenajde žádnou látku k zopakování a zobrazí informaci, že byla všechna látka ze zapsaných kurzů již zopakována.

2.1.2.18 UC-18 Spuštění lekce ze stránky studijního plánu

Popis:	Uživatel se rozhodne učít jinou lekci, než doporučenou lekci podle studijního plánu. Přesune se tak na stránku studijního plánu a spustí nadcházející lekci v jiném kurzu.
Požadavek:	FP-10 Učení se a opakování podle studijního plánu
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře hlavní stránku aplikace a zvolí možnost otevření studijního plánu. Aplikace mu zobrazí studijní plán, tedy seznam zapsaných kurzů a jejich aktuální režimy učení. U každého kurzu dále zobrazí možnost spustit učení se daného kurzu. Uživatel zvolí možnost učení se konkrétního kurzu a aplikace ho přesměruje na stránku učení, kde mu zobrazí látku z nadcházející lekce v rámci daného kurzu.

2.1.2.19 UC-19 Spuštění opakování po dokončení lekce

Popis:	Umožňuje uživateli ihned po dokončení lekce spustit opakování dříve naučené látky.
Požadavek:	FP-11 Opakování po dokončení lekce
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel dokončí učení a aplikace mu nabídne možnost spustit opakování. Uživatel opakování spustí, aplikace vybere vhodnou látku z kurzů ve studijním plánu v režimu učení nebo opakování a přesměruje uživatele na stránku opakování.
Alternativní scénář 1:	Uživatel po dokončení nové lekce možnost opakování odmítne. Aplikace uživatele vrátí na stránku, ze které učení dané lekce spustil.

2.1.2.20 UC-20 Učení se dané lekce

Popis:	Uživatel spustí učení dané lekce, splní jednotlivá cvičení a dokončí učení.
Požadavek:	FP-12 Nová struktura učení a opakování, FP-13 Bloky cvičení, FP-15 Bloky vysvětlení látky
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel spustí učení se dané lekce. Aplikace ho přeměruje na stránku učení a začne uživateli postupně zobrazovat bloky s vysvětlením látky a bloky se cvičeními. Uživatel projde všechny bloky s vysvětlením látky a splní správně všechna cvičení. Po dokončení posledního bloku aplikace zobrazí uživateli stránku se statistikami o provedeném učení. Uživatel zvolí možnost pokračovat a tím dokončí učení.
Alternativní scénář 1:	Uživatel v průběhu plnění lekce odpoví špatně. V takovém případě mu aplikace zobrazí možnost si zobrazit vysvětlení odpovědi a zkusit cvičení znovu. Uživatel si zobrazí vysvětlení odpovědi a vyplní znovu cvičení, tentokrát správně. Dále scénář pokračuje dle hlavního scénáře.
Alternativní scénář 2:	Uživatel v průběhu plnění lekce odpoví špatně. V takovém případě mu aplikace zobrazí možnost si zobrazit vysvětlení odpovědi a zkusit cvičení znovu. Uživatel zkusí cvičení ještě dvakrát a v obou případech odpoví špatně. Po třetí špatné odpovědi aplikace zobrazí uživateli možnost přeskočit dané cvičení a pokračovat v učení. Uživatel tuto možnost zvolí a aplikace mu zobrazí následující blok lekce.

2.1.2.21 UC-21 Opakování látky

Popis:	Uživatel spustí opakování látky, splní jednotlivá cvičení a dokončí opakování.
Požadavek:	FP-12 Nová struktura učení a opakování, FP-13 Bloky cvičení
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel spustí opakování a aplikace ho přeměruje na stránku opakování. Aplikace začne uživateli postupně zobrazovat bloky cvičení tak, aby si zopakoval vybranou látku. Uživatel postupně splní všechna cvičení. Po dokončení posledního cvičení aplikace zobrazí uživateli stránku se statistikami o opakování. Uživatel zvolí možnost pokračovat a tím dokončí opakování.
Alternativní scénář 1:	Uživatel odpoví ve cvičení špatně. Aplikace zobrazí stejnou negativní zpětnou vazbu jako při učení a dané cvičení znovu zařadí do opakování, aby bylo uživateli během stejného opakování zobrazeno ještě jednou. Uživatel následně odpoví na cvičení správně a pokračuje v dalších cvičeních.
Alternativní scénář 2:	Uživatel odpoví na cvičení špatně. Aplikace zobrazí stejnou negativní zpětnou vazbu jako při učení. Uživatel zkusí cvičení ještě dvakrát a v obou případech odpoví špatně. Po třetí špatné odpovědi aplikace zobrazí uživateli možnost přeskočit cvičení a pokračovat v opakování. Uživatel zvolí tuto možnost. Aplikace mu dané cvičení již znovu zobrazovat nebude a zobrazí uživateli následující blok opakování.

2.1.2.22 UC-22 Vyplnění cvičení pomocí editoru kódu

Popis:	Uživateli se v rámci průchodu lekcí zobrazí cvičení s editorem kódu. Uživatel použije editor kódu k vyplnění cvičení.
Požadavek:	FP-14 Editor kódu
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel spustí lekci. Aplikace mu zobrazí blok cvičení s editorem kódu. Uživatel správně vyplní všechna cvičení v záložkách editoru a zvolí možnost zkontrolovat řešení. Aplikace zobrazí informaci, že byla odpověď správná, a ukáže uživateli výstup daného kódu, pokud je k dispozici. Uživatel zvolí možnost pokračovat a tím dokončí cvičení.
Alternativní scénář 1:	Uživatel při vyplňování cvičení odpoví špatně. V takovém případě aplikace nezobrazí výstup daného kódu a zobrazí pouze negativní zpětnou vazbu. Uživatel opraví svou odpověď a dokončí cvičení.

2.1.2.23 UC-23 Zobrazení vysvětlení odpovědi při učení

Popis:	Uživatel v rámci učení nebo opakování vyplní cvičení a zvolí možnost vysvětlení, proč byla jeho odpověď správná nebo špatná.
Požadavek:	FP-16 Vysvětlení odpovědí ve cvičeních
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel v rámci učení nebo opakování vyplní cvičení a zvolí možnost zkontrolovat jeho řešení. Aplikace mu zobrazí možnost si zobrazit vysvětlení, proč byla jeho odpověď správná nebo špatná. Uživatel zvolí tuto možnost a zobrazí si tak vysvětlení odpovědi.

2.1.2.24 UC-24 Nahlášení problému při učení

Popis:	Uživatel nebo korektor nahlásí problém v bloku cvičení nebo v bloku vysvětlení látky.
Požadavek:	FP-17 Nahlašování problému při učení
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel při učení nebo opakování vyplní cvičení a zvolí možnost nahlásit problém. Aplikace zobrazí formulář pro nahlášení problému. Uživatel zvolí typ problému, popíše slovy problém a odešle formulář. Aplikace přesměruje uživatele zpět na stránku se cvičením.
Alternativní scénář 1:	Uživatel při učení nebo opakování na stránce bloku vysvětlení látky zvolí možnost nahlásit problém. Aplikace zobrazí formulář pro nahlášení problému. Uživatel zvolí typ problému, popíše slovy problém a odešle formulář. Aplikace přesměruje uživatele zpět na stránku s blokem vysvětlení látky.
Alternativní scénář 2:	Korektor při nahlášení využije rozšířené typy problémů a provede nahlášení problému podle předchozích scénářů.

2.1.2.25 UC-25 Zobrazení statistik po dokončení učení nebo opakování

Popis:	Po dokončení učení nebo opakování jsou uživateli zobrazeny souhrnné statistiky o učení resp. opakování.
Požadavek:	FP-18 Statistika o učení
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel dokončí poslední blok učení nebo opakování. Aplikace zobrazí uživateli stránku se statistikami. Na této stránce aplikace zobrazí délku učení nebo opakování, počet naučených konceptů, úspěšnost uživatele při odpovědích a počet bodů XP, které uživatel získal.

2.1.2.26 UC-26 Zobrazení žebříčku uživatelů v lize učení

Popis:	Uživatel si zobrazí žebříček uživatelů ve své aktuální lize učení.
Požadavek:	FP-34 Žebříček uživatelů
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel si zobrazí stránku s žebříčkem uživatelů. Aplikace zobrazí jeho aktuální ligu a pořadí uživatelů v dané skupině podle bodů XP získaných v aktuálním týdnu, včetně pozice daného uživatele.
Alternativní scénář 1:	Uživatel se v aktuálním týdnu ještě neučil. Aplikace zobrazí informaci, že bude uživatel zařazen do žebříčku po dokončení libovolné lekce nebo opakování.

2.1.2.27 UC-27 Zobrazení stavu energie uživatele

Popis:	Uživatel si zobrazí jeho aktuální stav energie, který určuje, kolik učení nebo opakování může ještě v daný den spustit.
Požadavek:	FP-19 Stav energie, FP-26 Nový systém předplatného
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel na hlavní stránce aplikace zvolí možnost zobrazení stavu energie. Aplikace zobrazí aktuální počet dostupných bodů energie pro daný den a také možnost odemknutí neomezeného množství energie.
Alternativní scénář 1:	Uživatel s premium účtem zvolí možnost zobrazení stavu energie. Aplikace zobrazí informaci, že učení a opakování nejsou pro premium účet omezeny systémem energie.

2.1.2.28 UC-28 Spuštění učení nebo opakování s nedostatkem bodů energie

Popis:	Uživatel se z libovolného místa aplikace pokusí spustit učení nebo opakování, přestože nemá dostatek bodů energie. Aplikace ho přesměruje na stránku s přehledem výhod premium účtu.
Požadavek:	FP-19 Stav energie, FP-26 Nový systém předplatného
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel bez aktivního předplatného, který již nemá žádné body energie, z libovolného místa aplikace spustí učení nebo opakování. Aplikace uživatele přesměruje na stránku s přehledem výhod premium účtu a informuje ho, že může získat neomezené množství energie, pokud si zakoupí premium účet.

2.1.2.29 UC-29 Nákup položky v obchodě s virtuální měnou

Popis:	Uživatel si na stránce obchodu zakoupí položku za virtuální měnu.
Požadavek:	FP-29 Rozšíření obchodu a odemykání položek
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře obchod, vybere odemčenou položku. Aplikace mu zobrazí detaily dané položky. Uživatel vybere možnost zakoupit položku. Aplikace odečte odpovídající množství virtuální měny a označí položku jako zakoupenou.
Alternativní scénář 1:	Uživatel vybere zamčenou položku. Aplikace zobrazí informaci, jaké podmínky musí uživatel splnit, aby položku odemknul a mohl ji zakoupit.

2.1.2.30 UC-30 Přizpůsobení avatara uživatele

- Popis:** Uživatel si zobrazí stránku pro přizpůsobení avatara, upraví vzhled avatara a uloží změny.
- Požadavek:** FP-28 Přizpůsobení avatara uživatele
- Hlavní aktér:** Uživatel
- Hlavní scénář:** Uživatel otevře stránku pro přizpůsobení avatara. Aplikace zobrazí aktuální vzhled avatara a dostupné položky, které si uživatel již zakoupil. Uživatel provede úpravy vzhledu avatara a zvolí možnost uložení změn. Aplikace změny uloží a přesměruje uživatele na předchozí stránku.
- Alternativní scénář 1:** Uživatel otevře stránku pro přizpůsobení avatara a provede úpravy, ale před uložení se rozhodne změny zahodit a zvolí možnost opustit stránku. Aplikace se zeptá uživatele, zda chce skutečně opustit stránku. Uživatel zvolí možnost opustit stránku. Aplikace změny neuloží a přesune uživatele na předchozí stránku.

2.1.2.31 UC-31 Zobrazení uživatelského profilu

- Popis:** Uživatel si zobrazí stránku svého profilu nebo profilu jiného uživatele.
- Požadavek:** FP-30 Uživatelský profil
- Hlavní aktér:** Uživatel
- Hlavní scénář:** Uživatel otevře stránku profilu. Aplikace zobrazí základní informace o uživateli, jeho avatar, aktuální ligu, stručný seznam zapsaných kurzů včetně jejich pokroku a stručný seznam získaných ocenění.
- Alternativní scénář 1:** Uživatel na stránce profilu zvolí možnost zobrazení kompletního seznamu ocenění. Aplikace mu zobrazí přehled všech získaných ocenění.
- Alternativní scénář 2:** Uživatel na stránce profilu zvolí možnost zobrazení kompletního seznamu zapsaných kurzů. Aplikace mu zobrazí přehled všech zapsaných kurzů včetně pokroku.

2.1.2.32 UC-32 Zapnutí tmavého režimu

Popis:	Uživatel otevře nastavení aplikace, změní režim vzhledu a rozhraní aplikace se okamžitě zobrazí v nově zvoleném režimu.
Požadavek:	FP-22 Tmavý režim
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře stránku nastavení. Aplikace mu zobrazí dostupné sekce nastavení. Uživatel zvolí sekci preferencí a aplikace mu zobrazí možnost přizpůsobit své preference. Uživatel zvolí jiný režim vzhledu než dosud používaný. Aplikace nastavení uloží a všechny obrazovky začne zobrazovat v nově zvoleném režimu.

2.1.2.33 UC-33 Nákup zmrazení streaku v obchodě

Popis:	Uživatel si v obchodě zakoupí položku zmrazení streaku za virtuální měnu.
Požadavek:	FP-31 Zmrazení streaku
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře obchod a vybere položku zmrazení streaku. Aplikace se ho zeptá, zda chce položku skutečně zakoupit. Uživatel nákup potvrdí. Aplikace odečte virtuální měnu a navýší počet dostupných zmrazení streaku u uživatele.

2.1.2.34 UC-34 Změna nastavení aplikace

Popis:	Uživatel otevře stránku nastavení, upraví své preference a aplikace změny automaticky uloží.
Požadavek:	FP-21 Stránka nastavení
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře stránku nastavení aplikace a změní jednu nebo více svých preferencí. Ihned po úpravě dané preference aplikace změnu automaticky uloží.

2.1.2.35 UC-35 Zobrazení nových oznámení na hlavní stránce

Popis:	Uživatel otevře hlavní stránku aplikace a aplikace mu zobrazí nová oznámení o důležitých událostech v aplikaci.
Požadavek:	FP-32 Rozšířená oznámení v aplikaci
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře hlavní stránku aplikace. Aplikace mu zobrazí nová oznámení na základě událostí, které v aplikaci nastaly. Uživatel si přečte oznámení a zvolí možnost pokračovat na hlavní stránku aplikace. Aplikace ho přesměruje na hlavní stránku.

2.1.2.36 UC-36 Zobrazení reklamy uživateli po ukončení učení nebo opakování

Popis:	Po ukončení učení nebo opakování aplikace zobrazí uživateli reklamu.
Požadavek:	FP-25 Systém zobrazování reklam, FP-26 Nový systém předplatného
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel dokončí nebo předčasně ukončí učení či opakování. Aplikace uživateli zobrazí reklamu v závislosti dostupném místě na obrazovce. Uživatel zhlédne reklamu a zvolí možnost pokračovat. Aplikace ho přesměruje na následující stránku.
Alternativní scénář 1:	Uživatel s premium účtem dokončí nebo předčasně ukončí učení či opakování. Aplikace reklamu nezobrazí a uživatele přesměruje přímo na následující stránku.

2.1.2.37 UC-37 Otevření truhly s odměnou po dokončení učení

Popis:	Uživatel po dokončení učení otevře truhlu s odměnou.
Požadavek:	FP-33 Truhla s odměnou, FP-25 Systém zobrazování reklam
Hlavní aktér:	Uživatel
Hlavní scénář:	Po dokončení učení nebo opakování aplikace zobrazí uživateli stránku s truhlou, která se sama otevře. Aplikace zobrazí uživateli množství obdržené virtuální měny. Uživatel zvolí možnost pokračovat. Aplikace ho přesune na následující stránku.
Alternativní scénář 1:	Po otevření truhly uživatel zvolí možnost získat dvojnásobek odměny. Aplikace zobrazí uživateli reklamu. Uživatel reklamu zhlédne. Aplikace po jejím úspěšném přehrání zobrazí uživateli informaci, že získal dvojnásobek odměny. Uživatel zvolí možnost pokračovat. Aplikace přesune uživatele na následující stránku.

2.1.2.38 UC-38 Zobrazení zpráv o pokroku sledovaných uživatelů

Popis:	Uživatel si zobrazí stránku s novinkami o pokroku sledovaných uživatelů.
Požadavek:	FP-35 Sdílení pokroku mezi uživateli
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře stránku s novinkami. Aplikace zobrazí zprávy o pokroku uživatelů, které daný uživatel sleduje. Uživatel následně na zprávy zareaguje pomocí dostupných emotikonů.
Alternativní scénář 1:	Uživatel otevře stránku s novinkami, ale aplikace nenalezne žádné nové zprávy o sledovaných uživatelích. Aplikace zobrazí informaci, že momentálně nejsou dostupné žádné nové zprávy o pokroku sledovaných uživatelů.

2.1.2.39 UC-39 Registrace pomocí účtu Google

Popis:	Nepřihlášený uživatel se zaregistruje do aplikace pomocí účtu Google.
Požadavek:	FP-20 Registrace a přihlášení pomocí účtu Google
Hlavní aktér:	Nepřihlášený uživatel
Hlavní scénář:	Nepřihlášený uživatel na stránce pro registraci zvolí možnost registrace pomocí účtu Google. Aplikace přesměruje uživatele na stránku, kde ověří svoji totožnost pomocí účtu Google. Po úspěšném ověření uživatele aplikace vytvoří nový účet, uživatele přihlásí a přesměruje ho na úvodní dotazník.
Alternativní scénář 1:	Nepřihlášený uživatel zahájí svoji registraci pomocí účtu Google, přičemž účet se stejnou e-mailovou adresou již v aplikaci existuje. Aplikace registraci neprovede, oznámí uživateli chybovou hlášku a nabídne uživateli přihlášení k existujícímu účtu.
Alternativní scénář 2:	Nepřihlášený uživatel při ověření přes Google zruší registraci. Aplikace ho přesměruje zpět na stránku registrace.

2.1.2.40 UC-40 Přihlášení pomocí účtu Google

Popis:	Nepřihlášený uživatel se přihlásí do aplikace pomocí svého účtu Google.
Požadavek:	FP-20 Registrace a přihlášení pomocí účtu Google
Hlavní aktér:	Nepřihlášený uživatel
Hlavní scénář:	Nepřihlášený uživatel na stránce pro přihlášení zvolí možnost přihlásit se pomocí účtu Google. Aplikace přesměruje uživatele na stránku, kde ověří svoji totožnost pomocí účtu Google. Po úspěšném ověření pomocí Google účtu aplikace uživatele přihlásí a přesměruje ho na hlavní stránku aplikace.
Alternativní scénář 1:	Nepřihlášený uživatel zvolí možnost přihlášení pomocí účtu Google, přičemž v aplikaci neexistuje účet propojený s daným Google účtem. Aplikace přihlášení neprovede, oznámí uživateli chybovou hlášku a nabídne uživateli registraci pomocí Google účtu.
Alternativní scénář 2:	Nepřihlášený uživatel při ověření přes Google zruší přihlášení nebo nepotvrdí souhlas. Aplikace ho vrátí na stránku přihlášení.

2.1.2.41 UC-41 Aktivace zkušební verze předplatného

Popis:	Uživatel bez aktivního premium účtu aktivuje zkušební verzi předplatného.
Požadavek:	FP-27 Zkušební verze předplatného
Hlavní aktér:	Uživatel
Hlavní scénář:	Uživatel otevře hlavní stránku aplikace. Aplikace mu zobrazí oznámení o dostupné zkušební verzi předplatného. Uživatel zvolí možnost vyzkoušet předplatné. Aplikace zobrazí stránku s informací, že bude uživateli zasláno oznámení dva dny před koncem zkušební doby. Uživatel zvolí možnost spustit zkušební dobu. Aplikace zobrazí uživateli formulář pro vyplnění platebních údajů. Uživatel vyplní platební údaje a potvrdí spuštění zkušební doby. Aplikace zkušební verzi aktivuje a zobrazí potvrzení o aktivaci.
Alternativní scénář 1:	Uživatel otevře stránku s předplatným. Na stránce předplatného zvolí možnost vyzkoušet předplatné. Scénář následně pokračuje stejným způsobem jako v hlavním scénáři krokem zobrazení informace, že bude uživateli zasláno oznámení dva dny před koncem zkušební doby.

2.1.2.42 UC-42 Zrušení zkušební verze předplatného

Popis:	Uživatel během zkušební doby zruší předplatné na stránce správy předplatného.
Požadavek:	FP-27 Zkušební verze předplatného
Hlavní aktér:	Premium uživatel
Hlavní scénář:	Uživatel s aktivní zkušební dobou předplatného otevře stránku pro správu předplatného a zvolí možnost zrušit předplatné. Aplikace se uživatele zeptá na důvod zrušení, uživatel vybere jednu z nabízených možností a zvolí možnost pokračovat. Aplikace se uživatele zeptá, zda chce skutečně zrušit předplatné, uživatel zrušení potvrdí a aplikace zobrazí informaci, že předplatné bylo zrušeno.

2.1.3 Specifikace funkcí pomocí jazyka Gherkin

Po sepsání případů užití jsem se rozhodl nové funkce přesněji specifikovat pomocí jazyka Gherkin [19]. Gherkin je jednoduchý strukturovaný jazyk, který

umožňuje pomocí přirozeného jazyka popsat chování systému v podobě textových scénářů. Díky přesně dané struktuře je následně možné tyto scénáře strojově zpracovat pomocí nástroje Cucumber¹ a ověřit, že se aplikace chová v souladu se specifikací pomocí automatizovaných testů. Celý proces je postavený na přístupu *Behavior-Driven Development* (BDD) [18], tedy na metodice vývoje, při které se přesně specifikuje očekávané chování systému a následně se vytvoří automatizované testy, které ověří, že se aplikace skutečně podle specifikace chová. Tento přístup vznikl jako reakce na rozšířenou metodiku *Test-Driven Development* (TDD) s cílem pomoci vývojářům a testerům nahlížet na testování více z pohledu samotného chování aplikace, přesněji pochopit a definovat požadované chování a minimalizovat tak nejednoznačnosti v zadání a implementaci jednotlivých funkcí [22].

Tento přístup má několik zásadních výhod. Zaprvé samotná specifikace slouží jako přesná dokumentace toho, jak se současný systém skutečně chová. Díky přirozenému jazyku může být užitečná nejen testerům a vývojářům, ale i lidem, kteří nejsou technicky zaměřeni. Zadruhé díky automatizovaným testům, které jsou spouštěny dle jednotlivých scénářů, je možné průběžně ověřovat, že se aplikace skutečně chová přesně v souladu se specifikací. Zatřetí přesné specifikace funkcí mohou sloužit jako podklady pro asistenty umělé inteligence, kteří pomáhají vývojářům a testerům s implementací kódu. Používání nástrojů umělé inteligence tak může být díky přesné specifikaci a automatizovaným testům výrazně efektivnější. [19]

Z výše zmíněných důvodů jsem se rozhodl využít jazyk Gherkin a nástroj Cucumber pro specifikaci funkcí a automatizaci testů s cílem zajistit efektivnější vývoj a kvalitnější implementaci samotných funkcí.

2.1.3.1 Postup při specifikaci

Funkce jsem specifikoval na základě definovaných případů užití a v souladu s dokumentací nástroje Cucumber [19], tedy přímo v repozitáři frontendu aplikace v souborech s příponou `.feature`. Tyto soubory jsem následně využil jako podklad pro implementaci a dále pro automatizaci testů, kterou jsem popsal v kapitole testování.

Soubory v jazyce Gherkin obsahují definici funkce **Feature** a jeden nebo více scénářů **Scenario** s jednotlivými kroky. Kroky scénáře jsou definovány pomocí klíčových slov **Given** a **When**, které definují předpoklady a dále

Soubor ve formátu Gherkin typicky obsahuje definici funkce (*Feature*), jeden nebo více scénářů (*Scenario*) a jednotlivé kroky. Kroky jsou definovány pomocí klíčových slov, kterými jsou například slova **Given** definující předpoklady, **When** definující událost v systému, **Then** definující očekávaný výsledek a **And** umožňující definovat více kroků stejného typu. Příklad definice funkce pomocí jazyka Gherkin lze nalézt v ukázce kódu 1.

¹Dostupné na: <https://cucumber.io/>

Feature: Vyhodnocení odpovědi ve cvičení

Scenario: Uživatel odpoví správně

Given uživatel je přihlášen

And má otevřené cvičení

When odešle správnou odpověď

Then aplikace zobrazí pozitivní zpětnou vazbu

And přičte uživateli body

■ **Výpis kódu 1** Ukázka specifikace funkce pomocí jazyka Gherkin

2.2 Návrh vzhledu uživatelského rozhraní

V této sekci se zaměřím na samotný vzhled uživatelského rozhraní a na volbu barev, písem, ikon a vhodných animací. Cílem je vytvořit konzistentní vzhled, který bude následně možné použít v prototypu a samotné aplikaci.

Jelikož stávající aplikace využívá knihovnu Material UI, která implementuje standard Material Design 2 [9], budu celý návrh zakládat na tomto standardu. Zmíněný standard klade důraz na konzistentní využívání palety barev, typografické hierarchie a komponentů. Dále se také zaměřuje na přístupnost uživatelského rozhraní, podporu světlého i tmavého režimu a dodržování doporučených osvědčených při návrhu uživatelského rozhraní [23].

Na tomto místě je vhodné zmínit, že v současné době již existuje nová verze tohoto standardu Material Design 3, nicméně v době psaní tohoto textu nejsou k dispozici žádné knihovny, které by tento standard implementovaly v takové míře, aby je bylo možné pro tento projekt vhodně využít. Proto budu nadále pro aplikaci využívat knihovnu Material UI, která se řídí standardem Material Design 2.

2.2.1 Principy návrhu

Před samotnou tvorbou návrhu je třeba stanovit základní principy, kterými se bude nový vzhled uživatelského rozhraní řídit. Při sestavování těchto principů budu vycházet z několika zdrojů. Zprvce budu vycházet z analýzy existujících řešení, které jsem se věnoval v předchozí kapitole.

Zadruhé budu vycházet z toho, jakým způsobem by měla aplikace působit na uživatele. Aplikace by například měla působit jako důvěryhodný zdroj informací, aby uživatel byl ochotný se pomocí ní vzdělávat.

Zatřetí jsem se rozhodl zaměřit na návrh s ohledem na emocionální potřeby uživatelů [24, s. 385]. Emoční potřeby uživatelů mohou být při návrhu určitých typů systémů klíčové a mohou aplikaci zásadně odlišit od konkurenčních řešení [24, s. 387]. Pokud například navrhujeme aplikaci pro děti, můžeme se cíleně zaměřit na návrh uživatelského rozhraní, které bude barevné, interak-

tivní a zábavné na používání. U jiných typů systémů, jako například v aplikaci obchodní burzy, by podobný způsob designu pravděpodobně působil dětinsky, neprofesionálně a nedůvěryhodně. Jelikož se v rámci této práce zaměřuji na tvorbu vzdělávací aplikace, mohou být emoční potřeby, jako motivace, pocit pokroku a úspěchu klíčové a proto na nich budu zakládat návrh uživatelského rozhraní.

Na základě zmíněných zdrojů jsem identifikoval tyto základní principy, kterými se budu při návrhu řídit.

- **Minimalismus** – Hlavním cílem aplikace je učit uživatele programování. Při takové činnosti je ovšem po uživateli vyžadována vysoká úroveň pozornosti a koncentrace. Ve srovnání s ostatními aplikacemi tak může být používání této aplikace značně kognitivně náročné. Proto jsem se rozhodl při návrhu zaměřit na minimalistický design. Design by tak měl co nejvíce omezit zbytečné informace a zjednodušit prvky uživatelského rozhraní tak, aby nepřehlcovaly uživatele a neodváděly jeho pozornost od samotného učení. Uživatelské rozhraní by tak mělo svým minimalismem pomoci uživateli při učení a používání aplikace.
- **Důvěryhodnost** – Jelikož se aplikace zaměřuje na výuku a předávání informací uživatelům, je naprosto klíčové, aby působila jako důvěryhodný zdroj informací. Uživatelské rozhraní by tak mělo působit moderně, důvěryhodně a profesionálně.
- **Zábava** – Jedním z cílů aplikace je motivovat uživatele k dlouhodobému učení a používání aplikace. Proto by pro uživatele by mělo být učení a používání aplikace zábavné a měl by se chtít k aplikaci dlouhodobě vracet.
- **Pokrok a úspěch** – Při používání aplikace a zejména při učení by měl mít uživatel pocit, že dělá pravidelně pokrok a že se posouvá blíže ke svému cíli. Aplikace by tak měla uživateli často ukazovat jeho pokroky a úspěchy.

V následujících sekcích na základě těchto charakteristik vyberu konkrétní barvy, typografii, ikony a animace.

2.2.2 Výběr palety barev

Jako první krok návrhu vzhledu jsem se rozhodl zvolit paletu barev². Vybrané barvy pak budu konzistentně využívat napříč prototypem a celou aplikací. Barvy budu volit v souladu se standardem Material Design 2 a budu volit barvy jak pro světlý, tak pro tmavý režim uživatelského rozhraní. Při volbě palety barev se také budu řídit standardem přístupnosti *Web Content Accessibility Guidelines 2.2* [25], díky kterému by mělo být uživatelské rozhraní dobře použitelné napříč různými zařízeními a také použitelné osobami se zdravotním postižením, jako například osobami se sníženým barvocitem.

²Dostupné na: <https://www.figma.com/design/VZFuqWojrXq8yWjrBIq4OH>

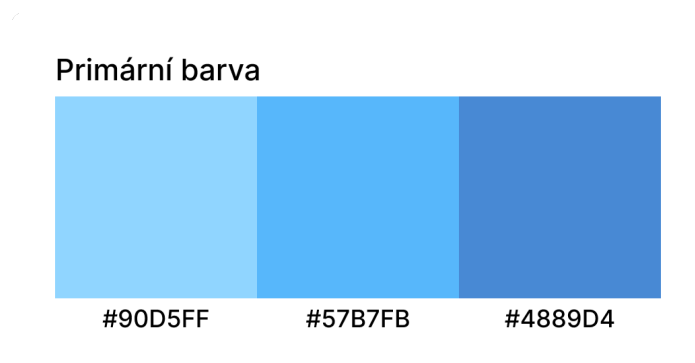
2.2.2.1 Primární barva

První barvu, kterou budu volit je primární barva. Tato barva reprezentuje samotnou značku aplikace a je napříč aplikací nejčastěji používána [26]. Využívá se zejména ke zvýraznění důležitých prvků, kterými jsou například tlačítka nebo navigační lišty.

Barvu jsem se rozhodl zvolit v souladu se stanovenými principy tak, aby aplikace působila důvěryhodně a profesionálně. Proto jsem jako primární barvu zvolil odstín modré, protože bývá tato barva považována za barvu důvěryhodnosti, inteligence a přesnosti [27].

Jako základ pro volbu primární barvy jsem vybral světle modrou s kódem #90D5FF. Tato barva by měla evokovat pocit důvěryhodnosti a klidu [28]. Jelikož se jedná o světlou barvu, zvolil jsem následně její dvě tmavší varianty #57B7FB a #4889D4. Pro volbu těchto variant jsem využil oficiální nástroj Material palette generator³, který umožňuje pro danou barvu vygenerovat různé světlé varianty. Dále jsem pomocí tohoto nástroje vybral kontrastní barvu textu #000000, která bude používána pro text zobrazovaný na primární barvě. Vybraná černá barva textu dodržuje s primární barvou kontrastní poměr 4.5:1 dle AA standardu přístupnosti *Web Content Accessibility Guidelines 2.2* [25]. Díky tomu bude text na primární barvě dostatečně kontrastní a bude tedy snadno čitelný.

Všechny tyto barvy generovány zmíněným nástrojem jsou v souladu se standardy přístupnosti UI [26]. Výslednou volbu primární barvy a jednotlivých variant lze nalézt na obrázku 2.1.



■ Obrázek 2.1 Volba primární barvy

2.2.2.2 Sekundární barva

Material Design 2 umožňuje kromě primární barvy také definovat sekundární barvu, která slouží k doplnění primární barvy, vytvoření vizuálního kontrastu

³Dostupné na: <https://m2.material.io/design/color/the-color-system.html>

a zvýraznění určitých prvků rozhraní. Na rozdíl od primární barvy, která typicky reprezentuje samotnou značku aplikace a je v rozhraní přítomna na většině míst, je sekundární barva používána střídměji a pouze u některých prvků UI. Prakticky se tak využívá například na zvýraznění odkazů, v přepínačích, indikátorech stavů aplikace nebo v tzv. *floating action buttons*. Přestože Material Design 2 definuje tuto sekundární barvu, její volba a použití jsou volitelné. [26]

Při volbě sekundární barvy jsem vycházel z principů teorie barev a ze standardně používaných barevných schémat, která definují rozložení barev na barevném kruhu [29]. Tato schémata slouží jako praktický nástroj pro posouzení, které barvy se vhodně doplňují tak, aby bylo možné je spolu využít například v uživatelském rozhraní. Na základě těchto schémat je tak možné zvolit sekundární barvu, která bude vhodně doplňovat primární barvu a vytvářet tak vizuálně příjemný vzhled uživatelského rozhraní. Při výběru sekundární barvy jsem volil z následujících schémat.

Monochromatické schéma Monochromatické schéma využívá jeden odstín a jeho světlejší či tmavší varianty. V rozhraní obvykle působí klidně a kompaktně. Je také snadné na použítí, protože designeři musí pracovat pouze s jedním hlavním odstínem. [29]

Analogické schéma Analogické schéma kombinuje sousední barvy na barevném kruhu. Toto schéma je často možné pozorovat v přírodě, kde se podobné odstíny barev často vyskytují blízko sebe. Výsledkem bývá harmonický, méně kontrastní vzhled a přírodně vypadající vzhled. [30]

Komplementární schéma Komplementární schéma využívá dvojici protilehlých barev na barevném kruhu. Vytváří tak vysoký kontrast, což je přínosné pro uchycení pozornosti uživatele na důležité prvky UI. Toto schéma tak umožňuje využití teplejších a chladnějších odstínů v UI. [30]

Rozděleně komplementární schéma Rozděleně komplementární schéma vychází z komplementární dvojice odstínů, ale místo přímého protikladu jsou využity dvě sousední barvy k doplňkové barvě. V praxi tak designeři mají možnost využít více barev a paleta tak může působit vyváženěji. [30] [29]

Triadické schéma Triadické schéma pracuje se třemi barvami rovnoměrně rozmístěnými na barevném kruhu do tvaru rovnostranného trojúhelníku. Schéma může působit živěji a dynamičtěji, nicméně stále je třeba určit která ze tří barev bude v UI dominantní a které budou používány střídměji. [30]

Tetradické schéma Tetradické schéma využívá čtyři barvy tvořící na barevném kruhu obdélník, často ve formě dvou komplementárních dvojic. Nabízí

široké možnosti kombinací, avšak jeho použití v designu UI může být náročné, zejména kvůli udržení konzistence ve využití jednotlivých barev. [30]

Čtvercové schéma Čtvercové schéma pracuje, jako tetradické schéma, se čtyřmi barvami rovnoměrně rozmístěnými na kruhu. Podobně jako tetradické schéma poskytuje velkou variabilitu, nicméně zvyšuje riziko vizuální roztržitosti, pokud nejsou stanoveny a dodržována jasná pravidla pro použití jednotlivých barev. Oproti tetradickému schématu umožňuje čtvercové schéma sestavit vyrovnanější vzhled ve srovnání s vysokým kontrastem, který tetradické schéma často vytváří. [30]

V rámci výběru sekundární barvy jsem se nakonec rozhodl využít monochromatické schéma, tedy pracovat primárně s jedním odstínem modré a jeho variantami, a samostatnou sekundární barvu pro rozhraní nedefinovat. Toto rozhodnutí přímo plyne ze stanovených principů minimalismu a důvěryhodnosti. Díky omezení počtu barev by tak mělo uživatelské rozhraní působit minimalističtěji a mělo by být snadnější dodržet konzistentní využití barev napříč návrhem a aplikací. Celý design by tak měl působit profesionálnějším a čistším dojmem. Zároveň, jelikož jsem jako primární barvu zvolil světle modrou, mělo by výsledné rozhraní působit klidným a méně rušivým dojmem [28]. To bude podstatné zejména při samotném učení, kdy je uživatel vystaven vyšší kognitivní zátěži a příliš agresivní barvy by mohly být rozptylující.

Přestože jsem se rozhodl pro monochromatické schéma a využívat tak hlavně primární barvu, bude aplikace obsahovat řadu dalších barev, které budou používány například pro indikaci stavů v aplikaci, jako například pro indikaci správných a špatných odpovědí uživatele. Tomu se budu věnovat v následujících sekcích.

2.2.2.3 Stavové barvy

Kromě zvolení primární a sekundární barvy je třeba také zvolit barvy pro indikaci různých stavů, které mohou v aplikaci nastat. Tyto barvy budou například použity pro indikaci správných a špatných odpovědí uživatele, nebo pro indikaci chyby a varování v aplikaci. Jako základní odstíny jsem zvolil zelenou pro indikaci úspěšných stavů, oranžovou pro indikaci varování a červenou pro indikaci chyb. Jelikož budou tyto barvy často používány při učení, rozhodl jsem se vybrat pastelové varianty těchto barev. Podobně jako primární barva by tak měly působit klidně, příjemně a neměly by příliš rozptylovat uživatele při učení.

Podobně jako u primární barvy, umožňuje Material Design 2 definovat hlavní variantu `main`, světlou variantu `light` a tmavou variantu `dark` pro každou barvu. Dále knihovna Material UI umožňuje definovat také odpovídající barvu textu `contrastText` [31], která je dostatečně kontrastní vůči dané barvě a dodržuje tak standard přístupnosti *Web Content Accessibility Guidelines 2.2*, dále WCAG 2.2 [25]. Pro každý stav jsem tak vybral varianty barev, které lze

nalézt na obrázku 2.2.



■ **Obrázek 2.2** Volba barev pro indikaci stavů

Podobně jako u primární barvy jsem pro volbu jednotlivých variant využil nástroj Material palette generator.

2.2.2.4 Barvy pozadí

Co se týče tmavého režimu, je třeba tuto barvu upravit tak, aby byla dostatečně kontrastní vůči pozadí dle standardu přístupnosti AA *Web Content Accessibility Guidelines 2.2* [25]. U primární barvy je tak třeba snížit její saturaci, aby byl dodržen kontrastní poměr 4.5:1 vůči pozadí při používání této barvy u textových prvků.

Jako barvu pozadí **background** jsem se rozhodl využít výchozí barvu Material Design 2, tedy #FFFFFF. Tato barva je používána zejména pro pozadí jednotlivých stránek. Jako barvu povrchů, neboli barvu pozadí některých komponentů, jsem dále zvolil světle šedou barvu #F3F3F3 označovanou jako **paper**, díky které bude možné komponenty lépe odlišit od pozadí stránky. Tyto barvy budou používány ve světlém režimu aplikace.

Ukázky barev pozadí a povrchů lze nalézt na obrázku 2.3.



■ **Obrázek 2.3** Volba barev pozadí a povrchů

2.2.2.5 Barvy textu

Pro textové prvky jsem se rozhodl využít výchozí barvu Material Design 2, tedy černou barvu #000000. Kromě této primární barvy textu knihovna Material UI ještě umožňuje definovat sekundární barvu textu, kterou je možné využít například pro odstavce a vytvořit tak silnější vizuální kontrast mezi nadpisy a normálním textem. Jako sekundární barvu jsem se tak rozhodl využít barvu #6f6f6f, která je výrazně světlejší než černá barva, nicméně i pro běžný text stále splňuje kontrastní poměr mezi textem a barvami pozadí 4.5:1 dle AA standardu WCAG 2.2 [25].

2.2.2.6 Tmavý režim

Jedním z definovaných požadavků je dostupnost tmavého režimu aplikace. Pro vytvoření tmavého režimu je třeba jednotlivé barvy upravit tak, aby je bylo možné využívat na tmavých pozadích a stále odpovídaly standardu WCAG 2.2 [25]. Při tvorbě tmavého režimu jsem postupoval dle doporučení oficiální dokumentace Material Design 2 [32].

Jako první krok jsem se rozhodl zvolit samotné barvy pozadí a povrchů pro tmavý režim. Jako barvu pozadí jsem zvolil tmavě šedou barvu #1A181D a jako barvu povrchů jsem dále zvolil tmavě šedou barvu #292834. Hlavním důvodem pro použití tmavě šedých barev, místo čistě černé barvy, je snaha dosáhnout nižšího kontrastu mezi světlým textem a tmavým pozadím. Text je

pak lépe čitelný a méně namáhá oči uživatele. [32]

Po vybrání barvy pozadí je třeba přizpůsobit saturaci primární barvy, barvy stavů a barvy textu tak, aby byl dodržen kontrastní poměr 4.5:1 mezi textem a tmavým pozadím [25]. Pro nalezení odpovídajících desaturovaných barev jsem využil nástroj Colour contrast checker⁴, který mi umožnil vybrat k jednotlivým barvám jejich desaturované varianty a zkontrolovat dodržení kontrastního poměru.

Nakonec jsem vybral barvy textu pro tmavý režim. Jako primární barvu jsem zvolil výchozí bílou barvu #FFFFFF a jako sekundární barvu jsem zvolil #989898. Obě barvy opět splňují požadovaný kontrastní poměr.

2.2.3 Typografie

V této sekci se zaměřím na volbu písma. Podobně jako u volby palety barev budu vycházet ze standardu Material Design 2 a budu volit vlastnosti písma v souladu se standardem přístupnosti WCAG 2.2 [25].

2.2.3.1 Výběr písma

Jako základní písmo jsem se rozhodl využít rodinu písem Inter. Tato rodina písem byla publikována v roce 2017 zaměřuje se na moderní a čistý vzhled, podporuje přes 147 jazyků a je široce využívána napříč různými obory. Pro tuto rodinu písem jsem se rozhodl zejména z důvodu moderního a profesionálního vzhledu a její široké podpory a rozšířenosti. [33]

Při volbě písma je možné dále využít tzv. párování fontů [14, s. 271-272], při kterém se pro nadpisy využije jiná rodina písma, než pro běžný text. Díky tomu lze dosáhnout výraznějšího rozlišení mezi nadpisy a běžným textem a také zajímavějšího vzhledu rozhraní. Při tomto párování je ovšem třeba použít písma, která jsou od sebe dostatečně odlišná a využít tak například patková a bezpatková písma. Pro zachování minimalismu jsem se rozhodl toto párování nevyužít a použít tak rodinu písem Inter pro nadpisy i běžný text.

Dále, jelikož budou v aplikaci často zobrazovány ukázky kódu, bylo potřeba zvolit písmo pro zobrazování kódu. Pro tento účel jsem se rozhodl využít rodinu písem JetBrains Mono⁵, která je speciálně navržena tak, aby byla dobře použitelná v editorech kódu a aby byl samotný kód dobře čitelný. [34]

2.2.3.2 Hierarchie a škálování

Po volbě rodiny písma je třeba zvolit typografickou škálu, která určuje velikosti jednotlivých textových prvků rozhraní, jako například velikosti nadpisů, podnadpisů a odstavců. Prvním krokem při volbě škály je zvolení základní velikosti písma, například 16 pixelů. Tato velikost reprezentuje velikost běžného textu

⁴Dostupné na: <https://colourcontrast.cc/>

⁵Dostupné na: <https://www.jetbrains.com/lp/mono/>

v odstavcích. Následně, na základě zvolené škály, se toto číslo násobí daným poměrem, čímž se získají velikosti nadpisů, podnadpisů a dalších textových prvků.

Volba typografické škály tak určuje, jaký je rozdíl mezi velikostmi jednotlivých textových prvků. Tento rozdíl pak ovlivňuje, jak velký bude vizuální kontrast mezi jednotlivými prvky. Škály s vysokým kontrastem, jako například *Golden ratio* s poměrem 1,618, jsou vhodné pro velké obrazovky [35]. Naopak škály s nízkým kontrastem, jako například *Major Second* s poměrem 1,125, umožňují zobrazit větší množství textu na jedné stránce, jelikož jednotlivé nadpisy nezabírají tolik prostoru. Takové škály se často používají například na mobilních zařízeních, kde je prostor na obrazovce omezený.

Při volbě škály jsem nejprve použil oficiální nástroj pro generaci typografické škály dle standardu Material Design 2 [36]. Po aplikaci této škály při návrhu uživatelského rozhraní jsem ovšem došel k názoru, že tato škála vytváří příliš vysoký kontrast mezi textovými prvky. Velké nadpisy zabírají příliš mnoho prostoru a jsou při návrhu rozhraní pro tento typ aplikace takřka nepoužitelné. Nedostatek menších nadpisů tak neumožňuje dosáhnout detailnějšího odlišení jednotlivých sekcí a podsekcí, což značně znepráhledňuje například stránky se cvičeními a vysvětlením látky. Došel jsem tak k závěru, že výchozí oficiální typografická škála Material Design 2 není pro tuto aplikaci vhodná.

Z tohoto důvodu jsem se rozhodl zvolit vlastní typografickou škálu, která bude vhodnější pro tento typ aplikace. Rozhodl jsem se vybrat dvě škály, jednu pro mobilní zařízení a jednu pro desktopová zařízení. Pro mobilní zařízení jsem se rozhodl využít škálu s nižším kontrastem *Minor Third* s poměrem 1.200 a pro desktopová zařízení škálu *Major Third* s poměrem 1,250. Díky těmto škálám tak bude možné plně využít všechny velikosti nadpisů a text tak bude přizpůsoben jednotlivým zařízením. [35]

Pro generaci jednotlivých velikostí písma jsem následně využil nástroj Typescale⁶. Při aplikaci vygenerované škály v návrhu rozhraní jsem následně dodržoval standardy Material Design 2, které se týkají vlastností jednotlivých textových prvků a definují například doporučenou velikost písma.

2.2.4 Ikony a ilustrace

V rámci návrhu rozhraní je dále třeba zvolit vhodnou kolekci ikon a ilustrací. Pro vzhled ikon jsem se rozhodl využít kolekci Material Icons⁷. Pro návrh jsem se rozhodl využít zaoblenou variantu těchto ikon, aby rozhraní působilo hravějším a zábavnějším dojmem.

Jelikož bude aplikace obsahovat různé formy gamifikace a dalších motivačních prvků, je třeba do návrhu zahrnout kromě ikon i různé ilustrace. Jako zdroj ilustrací jsem se rozhodl využít repozitář ilustrací publikovaných pod

⁶Dostupné na: <https://typescale.com/>

⁷Dostupné na: <https://fonts.google.com/icons>

svobodnými licencemi SVG Repo⁸. Jednotlivé ilustrace jsem se pak snažil volit tak, aby měly vzájemně konzistentní styl.

2.2.5 Animace a zvukové efekty

V rámci návrhu jsem se rozhodl v souladu s nefunkčními požadavky navrhnout i animace a efekty, které by měly být při implementaci použity. Příslušné animace a efekty jsem vyznačil poznámkami u jednotlivých stránek v návrhu uživatelského rozhraní.

Mezi použité animace patří například animace přechodu mezi stránkami, animace zmáčknutí tlačítka, animace konfet při dokončení lekce nebo získání ocenění, animace indikátoru pokroku při učení a další.

Kuriozitou při návrhu rozhraní bylo využití zvukových efektů. Zvukové efekty bývají ve webových aplikacích využívány velice zřídka, zejména kvůli tomu, že nejsou pro interakci s uživatelem potřeba a zbytečně by uživatele rušily při používání aplikace. V jistých případech mohou být ovšem zvukové efekty vhodné pro podpoření určitých emocí u uživatele a pro zvýšení intenzity požitku z vykonání určitých akcí. V této aplikaci budu například využívat zvukové efekty při samotném učení uživatele pro indikaci správných a špatných odpovědí. Jako zdroj zvukových efektů jsem se rozhodl využít soubor efektů Material Product Sounds⁹. Podobně jako u animací jsem vyznačil zvukové efekty v návrhu uživatelského rozhraní na místech, kde by měly být použity. [37]

2.3 Návrh prototypu uživatelského rozhraní

V této sekci se zaměřím na návrh *high fidelity* prototypu uživatelského rozhraní¹⁰, tedy prototypu, který bude obsahovat detailní návrh uživatelského rozhraní včetně definovaných stylů, barev, písma, ilustrací a interakcí [21, s. 53-54]. Cílem je vytvořit návrh uživatelského rozhraní, které bude přesně specifikovat vzhled aplikace a přechody mezi jednotlivými stránkami. Prototyp tak bude sloužit jako podklad pro samotnou implementaci aplikace.

Pro tvorbu prototypu jsem zvolil nástroj Figma¹¹, protože umožňuje rychlé iterace návrhu, práci s komponentami a také propojení obrazovek do podoby interaktivního prototypu. Při návrhu jsem vycházel ze stylů definovaných v předchozí sekci.

⁸Dostupné na: <https://www.svgrepo.com/>

⁹Dostupné na: <https://m2.material.io/design/sound/sound-resources.html>

¹⁰Dostupné na: <https://www.figma.com/design/VZFuqWojrXq8yWjrBIq4OH>

¹¹Dostupné na: <https://www.figma.com/>

2.3.1 Tvorba prototypu

V rámci mé bakalářské práce jsem se mimo jiné věnoval vytvoření návrhu uživatelského rozhraní pro stávající aplikaci na výuku angličtiny [2]. V průběhu let jsem ovšem zpozoroval několik nedostatků tohoto prototypu. Použitá knihovna předdefinovaných komponentů je zbytečně složitá a komplikuje vytváření a používání nových komponentů. Prototyp také nevyužívá moderní funkce programu Figma, jako například definice proměnných komponentů a globálních proměnných. S ohledem na nově definované styly, nové funkce a na nový účel aplikace jsem se tak rozhodl vytvořit celý návrh uživatelského rozhraní od začátku a bez návaznosti na existující prototyp.

Při tvorbě prototypu jsem postupoval následovně. Nejprve jsem zdefinoval pomocí proměnných jednotlivé styly dle návrhu v předchozí sekci. Následně jsem provedl hrubý návrh základních komponent a stránek aplikace. Poté jsem se v rámci agilního procesu vývoje vždy zaměřil na jednu funkci, kterou jsem navrhl podrobněji a zprovoznil její interakce v prototypu. Po dokončení návrhu dané funkce jsem provedl její implementaci a testování, které jsou popsány v následujících kapitolách. Po dokončení dané funkce jsem se přesunul na následující funkci, dle její priority. Po dokončení návrhu všech funkcí jsem provedl závěrečné úpravy návrhu a organizaci stránek a komponentů.

Při návrhu jsem kladl důraz na znovupoužitelnost komponent a na využití pokročilejších možností Figmy, díky kterým je možné prototyp v budoucnu jednodušeji rozšiřovat a upravovat. Konkrétně jsem široce využíval komponenty a jejich varianty, díky kterým je zajištěna vyšší konzistence samotného návrhu. Dále jsem využíval lokální proměnné komponentů a globální kolekce proměnných, díky kterým je možné kdykoliv měnit styly v návrhu, aniž by bylo nutné modifikovat samotné stránky a komponenty [38].

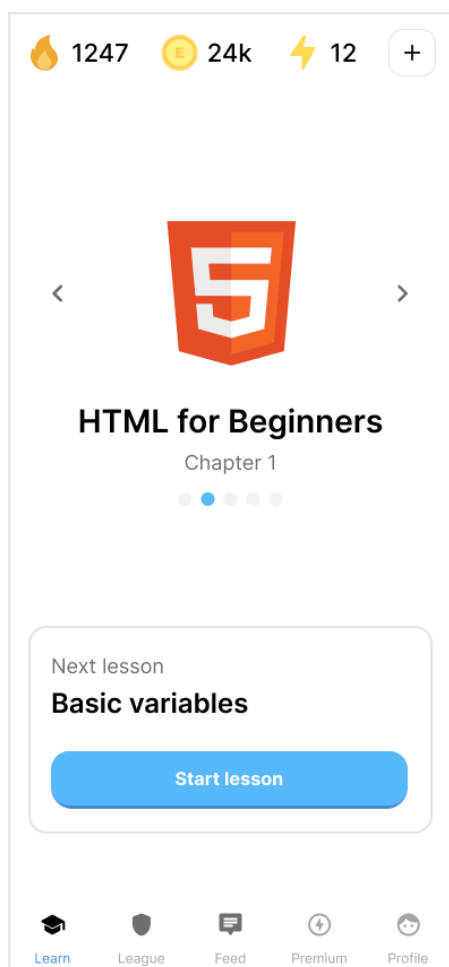
Jednotlivé komponenty jsem dále strukturoval tak, aby bylo možné je přímo implementovat s využitím jejich zadaných parametrů v prototypu. Při návrhu jsem také využíval funkci *auto layout*, díky které je možné přesně specifikovat rozložení komponentů tak, aby byly responzivní a použitelné v různých částech UI [39].

Součástí návrhu jsou také dva režimy aplikace, světlý a tmavý. Oba režimy sdílí stejné rozvržení stránek a komponentů a liší se pouze aplikovanými styly, zejména barvami pozadí, textu a stavů. Pro definici světlého a tmavého režimu jsem využil funkci módů proměnných [40], díky které je možné pro každý režim definovat jiné hodnoty proměnných a následně snadno mezi tmavým a světlým režimem přepínat. Díky tomu je opět zajištěna vyšší konzistence návrhu, jelikož není třeba pro světlý a tmavý režim navrhovat samostatné stránky, ale je možné pouze přepínat mezi aplikovanými styly.

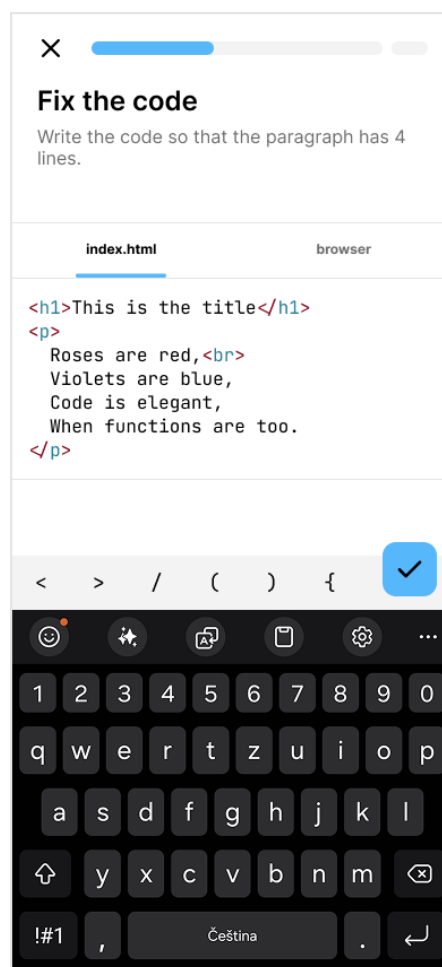
Vytvořený návrh vzhledu jsem také doplnil o interakce reprezentující reálné používání aplikace. Zaměřil jsem se zejména na navigaci mezi obrazovkami tak, aby bylo možné procházet základní scénáře používání aplikace. Díky tomu bylo možné prototyp interaktivně procházet bez samotné implementace a provádět

na něm základní testování.

Pro ukázkou návrhu jsem vybral několik stránek z prototypu. Tyto stránky lze najít na obrázcích 2.4, 2.5, 2.6 a 2.7.



■ **Obrázek 2.4** Ukázka návrhu hlavní stránky aplikace

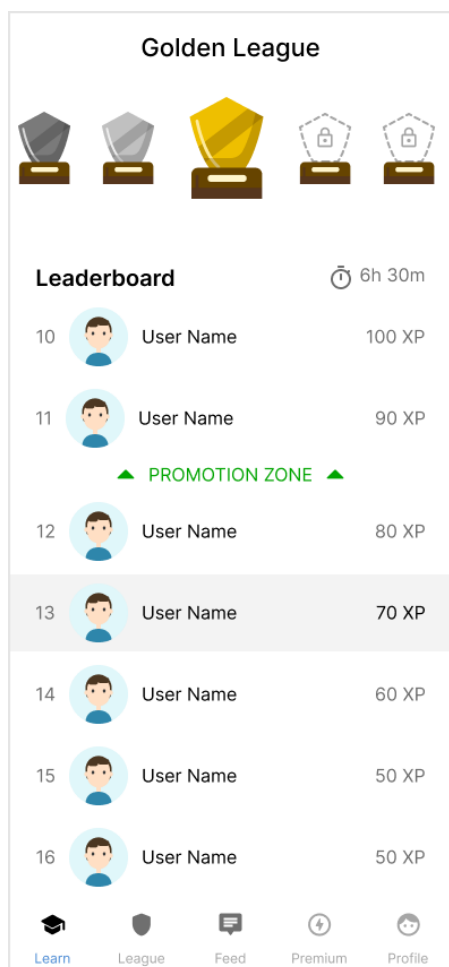


■ **Obrázek 2.5** Ukázka návrhu cvičení programování

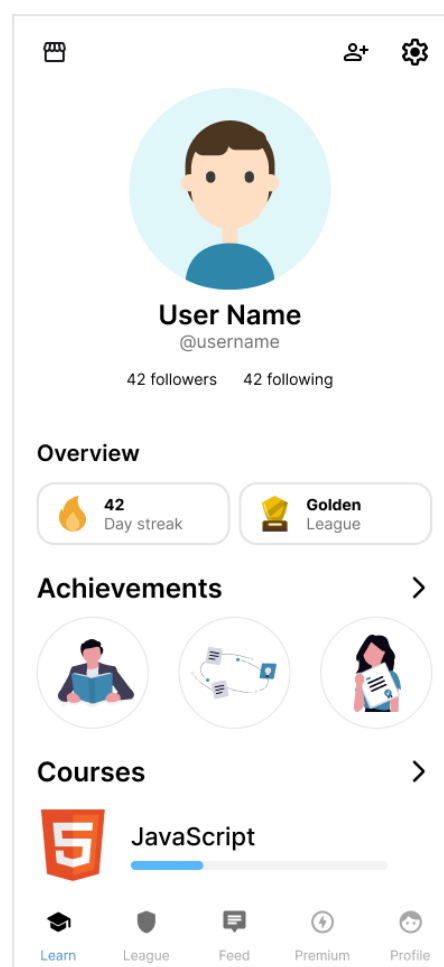
2.3.2 Responsivita

Návrh prototypu jsem realizoval přístupem *mobile-first* [41]. Primární cílovou platformou této aplikace jsou mobilní zařízení, proto jsem nejprve navrhl vzhled stránek pro menší rozlišení a teprve následně jsem řešil úpravy pro větší displeje. Tento postup mi umožnil soustředit se na efektivní využití prostoru a na jednoduché rozvržení stránek, které je na malých obrazovkách potřeba.

Po dokončení návrhu prototypu bylo třeba zvolit, jakým způsobem bude rozhraní responzivní, tedy jak se bude rozložení stránek přizpůsobovat různým



■ **Obrázek 2.6** Ukázka návrhu žebříčku uživatelů



■ **Obrázek 2.7** Ukázka návrhu uživatelského profilu

velikostem displejů. Při návrhu responzivity je možné využít několik standardních rozložení stránek.

Rozložení *mostly fluid* například typicky pracuje s bloky obsahu, které se přesouvají pod sebe se zmenšujícím se rozlišením [42, s. 7]. Dále je možné využít rozložení *column drop*, které využívá několik sloupců s obsahem a se zmenšujícím se rozlišením se sloupce opět přesouvají pod sebe [42, s. 8]. Také je možné využít rozložení *layout shifter*, které představuje nejkomplexnější ze zmíněných přístupů a spočívá v navržení jiné struktury stránek pro každou velikost displeje [42, s. 9].

V tomto projektu jsem se rozhodl využít přístup *tiny tweaks*, který spočívá v uspořádání veškerého obsahu stránky do jediného sloupce a zachování tohoto rozložení stránky napříč všemi zařízeními [42, s. 10]. Na stránkách jsou tak při změně velikosti displeje provedeny pouze drobné úpravy. Implemen-

tace se tak bude řídit primárně návrhem pro mobilní zařízení a na větších displejích se budou měnit především maximální šířka obsahu, odsazení a práce s volným prostorem. Kromě toho bude struktura stránek zachována a bude napříč zařízeními totožná.

Jako důsledek zvolení tohoto přístupu je tak možné udržet návrh jednoduchý a konzistentní. Zároveň díky totožnému rozložení stránek je snížené riziko rozdílného chování napříč různými velikostmi displejů. Aplikaci by tak mělo být snazší rozšiřovat, udržovat a testovat.

2.3.3 Testování prototypu

Po dokončení počáteční verze návrhu prototypu jsem provedl heuristickou analýzu, jejímž cílem bylo zachytit základní problémy v použitelnosti a konzistenci návrhu. Díky této analýze bylo možné identifikovat nedostatky návrhu ještě před samotnou implementací a upravit návrh tak, aby lépe odpovídal očekávanému chování uživatelů a byl v souladu se základními principy a heuristikami návrhu. [43]

Heuristická analýza a její výsledky jsou podrobněji popsány v kapitole testování. Na základě poznatků z této analýzy jsem následně upravil návrh tak, aby byl v souladu s heuristikami a byla tak vznikající aplikace pro uživatele lépe použitelná.

Implementace a nasazení

V předchozích kapitolách jsem se věnoval analýze a návrhu jednotlivých částí aplikace. V této kapitole navážu na předchozí kapitoly a popíšu, jakým způsobem jsem postupoval při implementaci a nasazení aplikace.

Nejprve popíšu úvodní úpravy aplikace, které jsem provedl s cílem připravit aplikaci na nadcházející vývoj. Tyto úpravy se týkaly zejména aktualizace používaných knihoven, aktualizace konvencí a linterů, refaktoringu kódu a zavedení nástrojů umělé inteligence pro asistenci při vývoji.

Dále popíšu, jakým způsobem jsem při vývoji postupoval a jakým způsobem jsem spolupracoval s kolegou Bc. Jakubem Vondráčkem, který se v jeho diplomové práci zabývá backendovou částí této aplikace.

Dále se budu věnovat implementaci nejvýznamnějších funkcí aplikace. Mezi tyto funkce patří například nová struktura učení, notificační systém, autentizace pomocí účtu Google nebo přizpůsobení aplikace tak, aby splňovala požadavky progresivních webových aplikací a bylo ji tak možné stáhnout a nainstalovat na zařízení uživatele.

Nakonec popíšu, jakým způsobem jsem nasadil frontendovou část aplikace do jednotlivých prostředí. Nejprve popíšu testovací prostředí využívající mock API, které simuluje reálné API a umožňuje testování nezávisle na stavu backendové části aplikace. Dále popíšu druhé testovací prostředí, které již využívá backendové API nasazené v testovacím režimu. Nakonec popíšu nasazení do produkčního prostředí, kde aplikace využívá produkční verzi backendového API.

3.1 Postup při realizaci projektu

V této části textu nejprve popíšu úvodní přípravu projektu a jednotlivé kroky, které jsem provedl před zahájením samotné implementace. Následně popíšu, jakým způsobem jsem postupoval při implementaci a jak jsem spolupracoval s kolegou Bc. Jakubem Vondráčkem, který se zabývá backendovou částí této

aplikace.

3.1.1 Příprava implementace

Před zahájením samotné implementace bylo třeba provést úvodní přípravu projektu. Tuto přípravu bylo třeba provést hned z několika důvodů. Původní aplikace byla naposledy aktivně vyvíjena v roce 2024, kdy jsem ji společně s kolegou Bc. Janem Hlaváčem vyvíjel v rámci našich bakalářských prací. Přestože jsme se projektu věnovali i po dokončení bakalářských prací, naše zaměření bylo především na části projektu které se netýkaly samotného vývoje. Vývoj tak ustál a aplikace nedostávala žádné nové aktualizace. Od roku 2024 došlo k řadě změn, kterým je nyní třeba projekt přizpůsobit.

První změnou jsou nové verze používaných knihoven. Většina používaných knihoven od té doby vydala nové verze, které obsahují podstatné opravy a vylepšení. Jejich aktualizace tak může přinést řadu výhod, jako například nové funkce, efektivnější vývoj, větší bezpečnost a stabilitu.

Další zásadní změnou je nástup nástrojů umělé inteligence v oblasti softwarového inženýrství. Tyto nástroje zásadně mění, jakým způsobem lze postupovat při vývoji softwaru a umožňují vyvíjet aplikace efektivnějším způsobem. Proto je třeba projekt přizpůsobit tak, aby byl připraven pro práci s těmito nástroji.

Nakonec, došlo k zásadní změně v doméně, na kterou se tato vzdělávací aplikace zaměřuje. Původní aplikace byla zaměřena výhradně na výuku jazyků. Nová aplikace by měla fungovat jako obecnější rozšiřitelná vzdělávací aplikace se zaměřením na výuku programování. Z tohoto důvodu je třeba změnit strukturu některých částí aplikace tak, aby bylo možné ji v budoucnu lépe rozšiřovat a udržovat. Se změnou domény se také pojí odstranění některých zastaralých funkcí, které neodpovídají novému zaměření aplikace.

V následujících částech textu tak podrobněji popíšu jednotlivé kroky přípravy projektu. Tuto přípravu jsem prováděl současně s předchozími fázemi analýzy a návrhu.

3.1.1.1 Příprava knihoven komponentů

V rámci přípravy aplikace jsem se rozhodl od aplikace oddělit některé UI komponenty a přesunout je do samostatných knihoven, které bude tato aplikace využívat. Komponenty jsem rozdělil do dvou knihoven `@eduriam/ui-core` a `@eduriam/ui-x`. První zmíněná knihovna bude obsahovat základní UI komponenty, jako tlačítka, textová pole, karty apod. Druhá zmíněná knihovna pak bude obsahovat komplexnější komponenty, které budou využívány jen ve specifických případech. Toto rozdělení tak reflektuje strukturu knihovny Material UI [9], která je podobně rozdělena na knihovnu základních komponentů MUI Core a na knihovnu komplexnějších komponentů MUI X.

Důvodů pro toto rozdělení je několik. Vytvoření knihovny `@eduriam/ui-core`

umožní postupně vytvořit jednotný design systém a knihovnu základních komponentů, kterou bude možné využívat jak v samotné aplikaci, tak v dalších částech projektu, jako například na webových stránkách, nebo v dalších interních programech, které nejsou součástí této diplomové práce. Díky tomu bude možné dodržet konzistentní design napříč všemi částmi projektu. Dalším důvodem je dodržení design principu *Separation of Concerns*, který říká, že bychom při vytváření systémů měli jasně oddělit zodpovědnosti jednotlivých částí systému. Díky tomu bude možné docílit přehlednější struktury kódu a lepší testovatelnosti a udržitelnosti celého systému. V případě této aplikace jsem se tak snažil oddělit základní UI komponenty od logiky samotné aplikace.

Cílem zmíněné knihovny tak bylo oddělit základní UI komponenty od zbytku projektu. Kromě základních komponentů ovšem vznikla potřeba sdílet určité složitější komponenty napříč různými částmi projektu. Zejména se jedná o komponenty, které vizualizují průběh učení dané lekce, tedy komponenty zobrazující jednotlivá cvičení a vysvětlení látky. Tyto komponenty budou využívány jednak v samotné aplikaci a jednak v interním nástroji pro tvorbu obsahu, ve kterém bude možné například zobrazit náhled učení se dané lekce. Zmíněný nástroj již není součástí této práce. Jelikož bude knihovna `@eduriam/ui-core` se základními komponenty využívána ve více programech, snažil jsem se tuto knihovnu zachovat co nejjednodušší a nejmenší. Z tohoto důvodu jsem pro komplexnější komponenty vytvořil samostatnou knihovnu `@eduriam/ui-x`, kterou bude možné zavést do programů ve kterých tyto specifické komponenty budou skutečně potřeba.

Další výhodou tohoto přístupu je možnost verzování knihoven pomocí sémantického verzování [44]. Díky tomuto způsobu verzování je možné snadno rozšiřovat a vylepšovat komponenty nezávisle na ostatních částech systému. Celý vývoj komponentů je tak strukturovanější a přehlednější.

Frontend aplikace tak bude využívat obě zmíněné knihovny. Do obou knihoven jsem také zavedl nástroj Storybook, který umožňuje snadno vyvíjet, dokumentovat a testovat UI komponenty v izolovaném prostředí.

3.1.1.2 Aktualizace knihoven a nástrojů

Dalším krokem přípravy aplikace byla aktualizace používaných knihoven. Jak jsem již zmínil, v průběhu let byla vydána řada nových verzí používaných knihoven, které mohou přinést řadu dříve zmíněných výhod. Proto jsem se rozhodl aktualizovat nejpodstatnější knihovny a nástroje, které aplikace využívá.

Mezi aktualizované knihovny patří například Next.js, Material UI, Storybook, ESLint, Prettier, Yarn, Node.js a další. Kromě těchto knihoven jsem dále v průběhu vývoje aktualizoval knihovny, u kterých bylo třeba použít jejich novější verze.

Celkově aktualizace knihoven skutečně přinesla řadu výhod. Novější knihovny

zefektivnily vývoj aplikace, což bylo možné pozorovat například u nástroje Storybook, jehož novější verze přinesla přehlednější a výrazně rychlejší tvorbu UI komponentů díky novým konvencím a rychlejšímu sestavování jednotlivých komponentů při spouštění Storybook serveru.

3.1.1.3 Refaktoring kódu

Po aktualizaci knihoven jsem dále provedl refaktoring stávajícího kódu. Nejprve jsem provedl úvodní refaktoring před fází implementace, který zahrnoval zejména přesun příslušných komponentů do dříve zmíněných knihoven `@eduriam/ui-core` a `@eduriam/ui-x`. Úvodní refaktoring dále zahrnoval zásadní změnu struktury komponentů pro učení, v rámci které jsem komponenty změnil tak, aby byly rozšiřitelnější a lépe udržitelné. Komponenty jsem tak připravil pro budoucí úpravy a vývoj.

V rámci úvodního refaktoringu jsem dále vytvořil nové komponenty, které usnadňují vývoj a sjednocují rozložení prvků UI na jednotlivých stránkách aplikace. Mezi tyto komponenty patří například komponenty `PageRoot`, `ContentContainer` a `PageNavigation`, díky kterým jsou rozložení a navigace napříč všemi stránkami konzistentní a snadno konfigurovatelné.

V průběhu vývoje jsem pak průběžně prováděl další refaktoring, který se týkal převážně průběžného odstraňování a aktualizace zastaralých komponentů a stránek. Tyto změny jsem prováděl průběžně zejména kvůli tomu, že bylo snazší staré funkce postupně nahrazovat novými a aktualizovat je, než všechny odstranit najednou.

3.1.1.4 Aktualizace linterů a konvencí

V rámci úvodní přípravy projektu jsem také provedl aktualizaci konvencí a konfigurací nástrojů pro formátování a kontrolu kvality kódu ESLint a Prettier. Jak jsem již zmínil výše, tyto nástroje jsem aktualizoval na jejich novější verze a dále jsem zavedl přísnější pravidla pro kontrolu kódu. Jedním z nově zavedených pravidel je například pravidlo `eqeqeq`¹, které vynucuje typově bezpečné porovnávání hodnot pomocí `===` a `!==` namísto porovnávání pomocí `==` a `!=`.

Zmíněná pravidla jsem zavedl s cílem snížit riziko vzniku nejednoznačného kódu a chyb, které mohou nastávat při nesprávném použití jazyků JavaScript a TypeScript, ve kterých je frontendová část aplikace naprogramována.

3.1.1.5 Zavedení nástrojů umělé inteligence

Další částí přípravy bylo zavedení konvencí pro nástroje umělé inteligence, které mohou vývojářům pomoci s vytvářením kódu. Vycházel jsem ze standardu Agent Skills² definovaným společností Anthropic. Tento standard umož-

¹Dokumentace dostupná na: <https://eslint.org/docs/latest/rules/eqeqeq>

²Dostupné na: <https://agentskills.io/>

ňuje s pomocí souborů ve formátu Markdown definovat instrukce a pravidla pro AI agenty, kteří na základě nich mohou v kódu vykonávat různé úkony.

Zdefinoval jsem tak pravidla a instrukce pro tvorbu nových UI komponentů, programování stránek aplikace, tvorbu automatizovaných testů a práci s `.feature` soubory.

Výhodou využití zmíněného standardu je vysoká kompatibilita s nejpoužívanějšími nástroji umělé inteligence pro tvorbu kódu, kterými jsou například Claude Code, Cursor, Codex, Gemini CLI a další. Instrukce tak bude možné využívat napříč různými IDE, nástroji a AI modely.

3.1.1.6 Příprava nového prototypu v nástroji Figma

V rámci úvodních příprav projektu, které probíhaly současně s analýzou a návrhem, jsem se rozhodl změnit způsob, kterým je tvořen prototyp uživatelského rozhraní v nástroji Figma. Původní návrh uživatelského rozhraní, který jsem vytvořil v rámci mé bakalářské práce, využíval externí knihovnu základních komponentů, které jsem upravil a následně využil v návrhu jednotlivých stránek aplikace.

Využití knihovny s již připravenými komponenty zpočátku umožnilo rychle a efektivně vytvořit úvodní prototyp uživatelského rozhraní. Časem se však ukázalo, že má tento přístup zásadní nevýhody při tvorbě *high fidelity* prototypu. Komponenty externí knihovny často používaly jinou strukturu a parametry, než bylo potřeba. Struktura komponentů a jejich parametrů se tak postupnými úpravami stávala zbytečně složitou, což značně snižovalo efektivitu tvorby prototypu.

Jelikož jsem v rámci této práce měl za cíl vytvořit kompletně nový a propracovanější vzhled všech komponentů a stránek aplikace, rozhodl jsem se zmíněný přístup změnit a místo externí knihovny komponentů vytvořit všechny komponenty ručně. Díky tomu jsem měl vyšší kontrolu nad tvorbou jednotlivých komponentů a byl jsem schopen je navrhovat jednodušeji a přesněji. Důsledkem toho byl přesnější návrh a snížená složitost komponentů. To vedlo k rychlejšímu úpravám prototypu a následně k efektivnější implementaci.

V rámci úpravy prototypu jsem také provedl organizaci návrhů stránek a komponentů, aby odpovídaly nové struktuře aplikace a dříve zmíněným knihovnám `@eduriam/ui-core` a `@eduriam/ui-x`.

Díky těmto změnám by tak měl být prototyp v budoucnu snadno rozšiřitelný a upravitelný.

3.1.2 Postup při implementaci

Po dokončení přípravy projektu jsem začal se samotnou implementací. Před samotnou implementací již byla dokončena většina kroků analýzy a společně s ní také úvodní verze návrhu uživatelského rozhraní. Měl jsem tak veškeré podklady pro zahájení implementace.

Současně s implementací frontendové části projektu také probíhala implementace backendové části, kterou prováděl kolega Bc. Jakub Vondráček v rámci jeho diplomové práce. Z důvodu výrazných změn API bylo třeba zajistit vhodnou formu spolupráce tak, aby probíhal vývoj plynule a efektivně. Rozhodli jsme se tak využít agilní metodiku Kanban, která se osvědčila při vývoji původní aplikace [2], kterou jsem vyvíjel s kolegou Bc. Janem Hlaváčem v rámci našich bakalářských prací. Tato metodika nám umožnila pracovat na jednotlivých funkcích takřka nezávisle na sobě a efektivně implementovat jednotlivé funkce. Důvody pro zvolení zmíněné metodiky jsem popsal v rámci mé bakalářské práce.

Implementaci funkcí jsem tak rozdělil do dvou fází. V první fázi jsem vždy implementoval danou funkci s využitím existujícího mock API, které jsem upravil tak, aby reflektovalo očekávanou strukturu nového API. V druhé fázi, poté co kolega Bc. Jakub Vondráček dokončil implementaci příslušných koncových bodů API, jsem napojil funkci na nové API a provedl případné úpravy v kódu frontendu. Jednotlivé fáze podrobněji popíšu v následujících částech textu.

3.1.2.1 Implementace nových funkcí

Při implementaci nových funkcí jsem postupoval iterativním způsobem dle priorit jednotlivých požadavků. V rámci iterativního vývoje jsem postupoval v následujících krocích.

Nejprve jsem na základě případů užití a požadavků aplikace provedl podrobnou specifikaci scénářů v jazyce Gherkin. Tuto specifikaci jsem prováděl již přímo v repozitáři aplikace v souborech s příponou `.feature`. Současně s tím jsem pro danou funkci kompletně dokončil návrh uživatelského rozhraní.

Na základě přesné specifikace scénářů jsem následně provedl implementaci samotné funkce společně s automatizovanými testy, které s pomocí nástroje Cucumber využívaly specifikované `.feature` soubory. Součástí implementace bylo také provedení změn v existujícím mocku backend API.

Po dokončení implementace jsem otestoval nově vytvořenou funkci spuštěním automatizovaných testů. Následně jsem také provedl manuální testování dané funkce.

3.1.2.2 Napojení funkcí na API

Po tom, co kolega Bc. Jakub Vondráček dokončil implementaci příslušných koncových bodů API, jsem provedl napojení funkce na nové API. V rámci této fáze jsem postupoval následujícím způsobem.

Nejprve jsem na základě podrobné OpenAPI specifikace backend API vygeneroval ve frontendové části aplikace třídy a typy, které reflektují novou strukturu API. Díky automatické generaci typů na základě OpenAPI specifikace jsme tak zajistili konzistentnější a přesnější komunikaci mezi fronten-

dovou a backendovou částí aplikace. Následně jsem provedl případné úpravy dané funkce a mocku API tak, aby odpovídaly nové struktuře API.

Po napojení funkce na nové API jsem znovu provedl testování funkce spuštěním automatizovaných testů. Dále jsem provedl manuální testování dané funkce, tentokrát s využitím testovacího backend API.

Po provedení všech výše zmíněných kroků tak byla funkce dokončená a připravená k nasazení do produkčního prostředí.

3.2 Implementace funkcí

V této podkapitole podrobněji popíšu implementaci vybraných funkcí této aplikace. Nejprve popíšu implementaci nové struktury učení, která zásadně mění způsob učení a opakování látky v aplikaci. Této funkci budu věnovat největší pozornost, protože se jedná o nejvýznamnější část celé aplikace. Dále popíšu implementaci registrace a přihlašování pomocí účtu Google, díky čemuž mohou uživatelé snadněji přistupovat do aplikace. Nakonec popíšu, jakým způsobem jsem z webové aplikace vytvořil progresivní webovou aplikaci, díky čemuž si mohou uživatelé aplikaci stáhnout a nainstalovat na svá zařízení.

3.2.1 Nová struktura učení

Se změnou domény původní aplikace bylo třeba zásadně změnit strukturu učení a opakování. Nejenže aplikace při učení musí důkladně zkoušet uživatele z dané látky, nyní musí uživateli také danou látku detailně a přehledně vysvětlit. V následujících částech textu popíšu nový model učení a jak nové učení vypadá z pohledu uživatele.

3.2.1.1 Lekce, atomy a bloky učení

Základním prvkem učení je **Atom**, který reprezentuje konkrétní znalost nebo koncept, který se uživatel může naučit. Každá lekce je tvořena jedním nebo více atomy. Ke každému atomu je pak přiřazen jeden nebo více tzv. **StudyBlock** bloků. Každý blok reprezentuje jednu stránku v učení, na které je uživateli buď vysvětlena nová látka, nebo je uživatel vyzkoušen z dané látky pomocí cvičení.

Tyto bloky se dělí do dvou hlavních typů.

- **ExplanationStudyBlock**, který slouží k vysvětlení nové látky,
- **ExerciseStudyBlock**, který slouží k procvičení dané látky a sběru dat o tom, jak uživatel dané látce rozumí.

Popsaný model je vyobrazen na diagramu ??.

3.2.1.2 Průběh učení a opakování

Při spuštění učení je uživateli zobrazena sekvence bloků učení, tedy objektů `ExplanationStudyBlock` a `ExerciseStudyBlock` s označením `learn` v předem stanoveném pořadí. Uživateli je tak pro každý atom v lekci nejprve vysvětlena látka pomocí `ExplanationStudyBlock`. Následně si uživatel látku procvičí pomocí `ExerciseStudyBlock`. V rámci každé lekce se tak uživatel postupně naučí několik nových atomů. V průběhu učení aplikace sbírá data o tom, jak uživatel odpovídá v jednotlivých cvičeních. Na základě těchto dat je následně po dokončení lekce naplánováno opakování dané látky v backendové části aplikace.

Pokud uživatel v průběhu učení odpoví špatně, aplikace mu nabídne možnost si zobrazit vysvětlení odpovědi a zkusit cvičení znovu. Pokud uživatel odpoví špatně třikrát, aplikace mu nabídne možnost cvičení přeskočit.

Při spuštění opakování je uživateli pouze zobrazena sekvence bloků `ExerciseStudyBlock` s označením `review`, které prověří, zda si uživatel danou látku pamatuje. Tyto bloky jsou vybrány v backendové části aplikace na základě toho, které atomy by si měl uživatel dle naplánovaného opakování zopakovat. Na základě odpovědi uživatele je látka opět po dokončení učení naplánována k dalšímu opakování v backendové části aplikace.

3.2.1.3 Vysvětlování látky pomocí `ExplanationStudyBlock`

Vysvětlování látky pomocí `ExplanationStudyBlock` je tvořeno vizualizacemi doplněnými o audio nahrávku vysvětlující danou látku. Stránky s vysvětlením látky tak připomínají krátká videa, díky kterým uživatel může pochopit danou látku. Klíčovým rozdílem oproti klasickým videím je způsob, jakým jsou tato videa vykreslována.

Na rozdíl od videí, která jsou například uložena ve formátu `mp4` a následně streamována přes síť, jsou tyto vizualizace animovány v prohlížeči za běhu samotné aplikace pomocí knihovny `Remotion` na základě JSON definice obsažené v daném `ExplanationStudyBlock` bloku. Tento přístup má několik zásadních výhod a také několik limitací.

Zaprvé, díky použité knihovně `Remotion`, která umožňuje snadno animovat UI komponenty za běhu aplikace, je tak možné vytvořit knihovnu UI komponentů, které mohou být využity při vytváření vysvětlovacích videí. Protože knihovna `Remotion` pracuje s React komponenty, je možné komponenty videí vytvářet analogickým způsobem, jako zbytek UI komponentů v aplikaci. Díky tomu lze v komponentech videí snadno aplikovat existující styly aplikace a zachovat tak naprosto konzistentní vzhled mezi samotnou aplikací a obsahem videí. Pokud bychom se například rozhodli změnit primární barvu nebo jiné styly aplikace, stejná změna bude automaticky aplikována ve všech vysvětlovacích videích. Vzhled videí tak bude vždy aktuální a konzistentní se vzhledem celé aplikace.

Se zmíněnými styly komponentů se pojí i další výhoda, která je spíše vedlejším efektem použitého přístupu. Protože komponenty videí dodržují stejný

styl jako zbytek aplikace, je ve videích reflektován i světlý či tmavý režim aplikace. Byť tento efekt nebyl primárním záměrem použitého přístupu, může výrazně zlepšit uživatelskou zkušenost při sledování videí. Pokud například uživatel preferuje používání aplikace v tmavém režimu při učení se večer, aplikace bude automaticky i všechna videa zobrazovat v tmavém režimu. Učení tak může být pro uživatele výrazně příjemnější.

Další výhodou je znovupoužitelnost komponentů uživatelského rozhraní. Pokud bychom například vytvořili v aplikaci komponentu vizualizující algoritmus Bubble Sort, můžeme tuto komponentu využít jak v samotných cvičeních, tak ve vysvětlovacích videích. Tato znovupoužitelnost tak může zásadně zefektivnit tvorbu obsahu a zvýšit jeho kvalitu. Při tvorbě obsahu tak bude možné investovat více času do tvorby samotných komponentů a následně je využívat na mnoha místech napříč cvičeními a videi.

Další kuriózní výhodou je responzivita samotných videí. Jelikož jsou videa tvořena responzivními UI komponenty, které jsou vykreslovány za běhu aplikace, jsou i samotná videa responzivní. Video tak lze sledovat na zařízeních s různě velkými displeji, tedy na počítačích, tabletech i mobilních zařízeních. Docílení této responzivity samozřejmě nebylo přímočaré a bylo třeba zavést řadu opatření, aby bylo této responzivity dosaženo. Tomuto tématu se budu věnovat v následující podkapitole tohoto textu.

Kromě výše zmíněných výhod je další zásadní výhodou snadná udržitelnost vysvětlovacích videí. Pokud například text ve videu obsahuje chybu nebo nepřesnost, stačí ji opravit v JSON definici `ExplanationStudyBlock` bloku v databázi a uživatelům se začne po načtení učení ihned zobrazovat opravená verze. Tyto opravy se netýkají jen obsahu, ale i samotných komponentů videí. Pokud se například některý prvek zobrazuje chybně, jeho oprava bude po nasazení frontendu ihned reflektována ve všech videích. Tyto změny se netýkají pouze chyb, ale i zdokonalování obsahu, resp. jednotlivých komponentů videí. Díky tomu, že mohou být všechny změny takřka okamžitě aplikovány, je tak možné velice rychle vyvíjet a vylepšovat kvalitu obsahu celé aplikace.

Poslední výhodou, kterou zde zmíním, je interaktivita samotných videí. Jelikož jsou komponenty videí vykreslovány dynamicky a jedná se o UI komponenty, je možné docílit různých interaktivních efektů. Ve videích je tak možné například vykreslit tlačítko, které bude reagovat na kliknutí uživatele. Jiný komponent může zase například vykreslit aktuální počasí na určitém místě. Byť jsem v rámci této práce do videí žádné interaktivní komponenty nezahrnoval, je možné tuto možnost využít v budoucnu a rozšířit tak videa, aby byly interaktivnější a zajímavější.

S výše zmíněnými výhodami se pojí několik limitací, které je třeba brát v úvahu. Jelikož jsou animace vytvářeny za běhu aplikace v prohlížeči, je třeba brát v potaz výkon prohlížečů, použitých programovacích jazyků a výkon samotných zařízení. Prohlížeče s jazykem JavaScript ze své podstaty nebyly vytvořeny primárně pro dynamické vykreslování komplexních videí. Uživatelé také mohou používat aplikaci na různě starých a výkonných zařízeních, což

může limitovat schopnost prohlížeče přesně animovat jednotlivé komponenty. Kvůli tomu by bylo nepraktické využívat tento způsob vykreslování pro složitější a graficky náročnější animace. Pokud by například bylo třeba vykreslit vizualizaci simulace, v níž by se pohybovalo přes deset tisíc prvků, byl by tento přístup takřka nepoužitelný.

Z výše zmíněného důvodu jsem se rozhodl do vysvětlovacích videí zavést komponent **Video**, díky kterému je možné do videa vložit video ve standardním formátu, tedy například ve formátu **mp4**. Díky tomu lze přístup popisovaný v této kapitole využít pro jednodušší a graficky méně náročné prvky, a pro složitější vizualizace využít zmíněný komponent **Video**.

Další limitací rozebíraného přístupu je responzivita samotných videí. Pokud chceme docílit responzivních videí, je třeba zásadně přizpůsobit rozložení prvků na obrazovce mobilním zařízením, které ve srovnání většími monitory obsahují výrazně méně prostoru pro zobrazení grafických prvků. Aby bylo dosaženo plně funkční responzivity, bylo třeba zásadně zjednodušit strukturu videí a zredukovat tak počet zobrazovaných komponentů, které lze v jednu chvíli ve videu zobrazit. Této responzivitě se budu věnovat v následující podkapitole tohoto textu.

Poslední limitací, kterou zde zmíním, je použitelnost videí v situacích, kdy uživatel nemá možnost poslouchat dostupnou audio nahrávku. Pokud například uživatel jede v autobuse nebo vlaku a nemá k dispozici sluchátka, jeho možnost sledovat vysvětlovací videa je značně limitována. Tato limitace vychází spíše z rozhodnutí vysvětlovat látku formou videí, než ze samotného přístupu dynamického vykreslování videí popisovaném v této kapitole. Byť je tato limitace zásadní z hlediska uživatelské zkušenosti, je cenou za získání potenciálně záživnější formy výuky a efektivnějšího vysvětlování látky. V rámci této práce jsem se tak rozhodl, že výhody tohoto přístupu převyšují zmíněnou nevýhodu a rozhodl jsem se tak tento přístup použít.

Výše zmíněnou limitaci jsem se snažil minimalizovat tím, že jsem k videím přidal možnost si zobrazit titulky, pokud uživatel nemá možnost poslouchat audio. Uživatelé, kteří nemají možnost poslouchat audio si tak aspoň mohou při sledování videa číst titulky a získat tak informace touto cestou.

Celkově tak tato forma vysvětlování látky má zásadní výhody a několik limitací. V rámci této práce jsem došel k názoru, že zmíněné výhody převažují nad nevýhodami a rozhodl jsem se tak tento přístup použít.

3.2.1.4 Responzivita videí s vysvětlením látky

V této podkapitole stručně popíšu, jakým způsobem jsem zajistil responzivitou vysvětlovacích videí, kterou jsem zmínil v předchozí kapitole.

Aby bylo docíleno responzivního chování videí, bylo třeba, aby videa dodržovala přesně danou a jednoduchou strukturu. Z tohoto důvodu jsem zavedl následující strukturu videí. Každý **ExplanationStudyBlock** se skládá z několika scén **Scene**. Každá scéna se pak skládá ze stránek **Slide**, které významově

odpovídají stránkám v prezentacích. Každý **Slide** je pak rozdělen do jednoho až tří sloupců **Column**, ve kterých jsou následně zadefinovány jednotlivé komponenty, které se mají zobrazit. Komponenty ve sloupcích se vždy zobrazují pod sebou v pořadí, ve kterém jsou ve sloupci zadefinovány.

Na širokých displejích, tedy například na monitorech nebo mobilních zařízeních orientovaných na šířku, jsou jednotlivé sloupce vykreslovány vedle sebe. Pokud uživatel ovšem sleduje video na zařízení s úzkým displejem, tedy typicky na mobilním zařízení otočeným na výšku, jsou pak sloupce vykreslovány pod sebou. Jednotlivé komponenty se dále přizpůsobují dostupnému místu, podobně jako responzivní komponenty na webových stránkách. Tento systém tak umožňuje snadno umisťovat komponenty na obrazovku tak, aby se jejich rozložení přizpůsobovalo velikosti a orientaci obrazovky.

Limitací tohoto přístupu je množství komponentů, které je možné ve videu na jedné stránce zobrazit. Do sloupců například není vhodné umisťovat přehršel komponentů s pevně danou výškou, protože by se z důvodu přesunu sloupců na menších displejích pod sebe nevešly. V důsledku tohoto systému by videa měla být tvořena tak, aby obsahovala více jednodušších stránek s méně komponenty, než méně stránek s mnoha komponenty. Dále, pokud je například třeba zobrazit komplexnější komponent, který obsahuje více detailů nebo textu, je vhodné na obrazovce zobrazit pouze tento komponent v jednom sloupci. Díky tomu bude mít sloupec na všech velikostech obrazovky maximální výšku a šířku a komponent tak bude možné dobře vidět.

Samotné komponenty bylo také třeba vytvářet s ohledem na responzivitu videí. Například komponenty zobrazující kód jsem implementoval tak, aby bylo možné v jejich definici určit, v jaké fázi videa se má zvýraznit jaká část kódu. Pokud by tak například bylo třeba ve videu ukázat kód s mnoha řádky a vysvětlovat různé části tohoto kódu, je možné v definici komponentu označit jednotlivé části, o kterých se bude ve videu ve vybraném čase mluvit. Aplikace pak bude automaticky v daném čase scrollovat na danou část kódu, díky čemuž bude uživatel vždy na obrazovce vidět, o které části kódu se mluví.

3.2.1.5 Plnění cvičení pomocí **ExerciseStudyBlock**

Po tom, co je uživateli vysvětlena látka pomocí dříve popsáných **ExplanationStudyBlock** bloků, je uživateli zobrazen blok **ExerciseStudyBlock**, který slouží k procvičení dané látky a sběru dat o tom, jak uživatel dané látce rozumí.

Každý tento blok je, podobně jako bloky s vysvětlením látky, vykreslen na samostatné stránce v rámci učení. Tento blok může obsahovat jeden nebo více komponentů, které jsou vykresleny pod sebou v jednom sloupci. Komponenty jsou buď pasivní, které pouze zobrazují informace, nebo aktivní, se kterými může uživatel interagovat. Pasivní komponenty jsou například nadpisy, odstavce nebo diagramy. Aktivními komponenty jsou různá cvičení, které musí uživatel vyplnit pro splnění daného bloku. Jeden **ExerciseStudyBlock** může obsahovat více aktivních bloků. Příkladem toho je editor kódu, který obsahuje

více oken s kódem. Každé okno je pak považováno za samostatné cvičení.

Po vyplnění všech aktivních komponentů v rámci daného bloku uživatel zvolí možnost zkontrolovat výsledky, aplikace odpověď ohodnotí a zaznamená. Uživatel pak má možnost pokračovat nebo odpověď opravit v případě, že byla špatná.

Podobně jako bloky s vysvětlením látky, mohou bloky se cvičeními obsahovat audio nahrávku, která doprovází uživatele při učení. Na rozdíl od bloků s vysvětlením látky, kde je audio nahrávka přehrávána v rámci celého videa, je v případě bloků se cvičeními audio nahrávka pouze připojena ke konkrétním komponentům. Každý komponent tak k sobě může mít připojenou audio nahrávku, která například vysvětluje zadání daného příkladu. Po načtení bloku se postupně přehrají všechny audio nahrávky, které jsou v daném bloku zadefinovány. Díky tomu například uživatel nemusí číst text se zadáním, ale může si ho poslechnout a následně hned vyplnit dané cvičení.

Díky audio nahrávkám a flexibilní struktuře bloků cvičení a vysvětlování látky je tak možné pro uživatele vytvářet vysoce interaktivní průchody lekcemi a opakováním. Přístup této aplikace se tak blíží vysoce interaktivním lekcím konkurenční aplikace Scrimba, kterou jsem analyzoval v rámci analýzy existujících řešení.

3.2.2 Registrace a přihlášení pomocí účtu Google

Pro externí autentizaci jsem implementoval OAuth workflow nad integrační vrstvou frontendu a backendových endpointů. Ve frontendu začíná tok funkcí `startGoogleAuth`, která uloží zdroj akce (`signin` nebo `signup`) do `sessionStorage`, získá autorizační URL z endpointu `external-auth/google/authorize` a přesměruje uživatele na autorizační stránku Google.

Po návratu z Google se zpracování provádí na callback stránce `/signin/callback`. Frontend načte autorizační kód z query parametru `code` a předá jej backendu přes endpoint `/authorize`. Po úspěšném dokončení frontend uloží autentizační data (`idToken`, `refreshToken`, uživatele), nastaví autorizační hlavičku pro další API volání a přesměruje uživatele buď na hlavní stránku, nebo na onboarding podle stavu účtu.

Součástí implementace je i explicitní mapování chybových stavů. Například stav 404 je mapován na Google účet nenalezen a stav 409 na Google účet již existuje. Uživateli se následně zobrazí odpovídající kontext na přihlašovací nebo registrační stránce.

3.2.3 Přizpůsobení aplikace pro PWA

V rámci nefunkčního požadavku jsem doplnil konfiguraci aplikace nutnou pro to, aby prohlížeče aplikaci klasifikovaly jako progresivní webovou aplikaci. V souboru `manifest.ts` jsem definoval metadata aplikace, jako například název, jazyk aplikace, základní barvy a ikony. V rámci této konfigurace jsem

také vytvořil několik základních ikon, které může progresivní webová aplikace využívat.

Dále jsem do aplikace doplnil registraci tzv. Service Worker služby. Service Worker je skript běžící v prohlížeči, jehož životní cyklus je takřka nezávislý na životním cyklu aplikace. Service Worker tak může běžet ve vlastním vlákne nezávisle na samotné aplikaci. Tato služba se využívá například pro správu cache aplikace, podporu offline funkcí a jako proxy mezi odchozími a příchozími požadavky a samotnou aplikací. V rámci této práce jsem se těmito funkcím nevěnoval a proto jsem pouze zaregistroval prázdný Service Worker skript, který je nutný pro vytvoření PWA. [45]

Nakonec bylo třeba zajistit, aby byla aplikace dostupná přes HTTPS protokol [46]. To jsem zajistil v rámci nasazení aplikace, kterému se věnuji v podkapitole Nasazení aplikace.

Díky výše zmíněné konfiguraci se tak aplikace stala progresivní webovou aplikací. Uživatelé si tak mohou aplikaci stáhnout a nainstalovat do svých zařízení přímo z prohlížeče na stránkách této aplikace.

Aplikaci tak bude možné v budoucnu po dalších konfiguracích, například s pomocí nástroje PWABuilder [46], zabalit a publikovat na platformách Google Play, App Store nebo například Microsoft Store. Publikace na tyto platformy je časově značně náročná a týká se spíše právních otázek, nežli softwarového inženýrství. Proto se samotné publikaci v rámci této práce nebudu věnovat.

3.3 Nasazení aplikace

V rámci nasazování aplikace jsem vytvořil tři samostatná prostředí `dev`, `stage` a `production`. Každé prostředí má odlišný účel, odlišné napojení na API a odlišný způsob nasazení. Kromě nasazení aplikace bylo třeba také zajistit, aby byly k dispozici pro stažení a instalaci vytvořené knihovny `eduriam/ui-core` a `eduriam/ui-x`. Vše podrobněji popíšu v této podkapitole.

3.3.1 Vývojové prostředí `dev`

Testovací prostředí `dev` je dostupné na adrese `dev.eduriam.com`. Hosting je zajištěn na platformě Vercel. Tento hosting jsem zvolil jednak protože původní aplikace tento hosting využívala a jednak protože společnost Vercel vyvíjí framework Next.js, ve kterém je tato aplikace naprogramována. Aplikace v tomto prostředí využívá mock backend API, který je nasazený na platformě Netlify.

Nasazení do tohoto prostředí probíhá automaticky při každém provedení `push` do větve `dev` v repozitáři frontendu aplikace. Při provedení této akce se provede jak nasazení aplikace, tak nasazení zmíněného mocku API.

Toto prostředí slouží k několika účelům. Jednak slouží k rychlému vyvíjení a testování funkcí aplikace, které ještě nejsou podporovány backend API. Dále lze toto prostředí snadno využít pro uživatelské testování a validaci nových funkcí.

3.3.2 Testovací prostředí stage

Testovací prostředí **stage** je dostupné na adrese `app.stage.eduriam.com` a rovněž běží na hostingu Vercel. Na rozdíl od prostředí **dev** je toto prostředí napojeno na stage backendové API, tedy na reálnou implementaci API určenou pro testování.

Nasazení frontendu do stage probíhá také automaticky při každém provedení **push** do větve **dev**. Nasazením backendu se zabývá kolega Bc. Jakub Vondráček v jeho diplomové práci.

Stage prostředí tak představuje hlavní testovací prostředí, ve kterém lze ověřovat chování aplikace se skutečnými koncovými body API.

3.3.3 Produkční prostředí production

Produkční prostředí **production** je dostupné na adrese `app.eduriam.com` a je napojeno na produkční backendové API. Podobně jako ostatní prostředí je toto prostředí nasazeno na platformě Vercel.

Nasazení probíhá při manuálním vytvoření release značky na větvi **master**, což odpovídá procesu nasazování, který byl zaveden v původní aplikaci. Nasazením produkčního API se opět zabývá kolega Bc. Jakub Vondráček v jeho diplomové práci.

Produkční prostředí tak představuje hlavní prostředí, ve kterém je aplikace veřejně dostupná uživatelům.

3.3.4 Nasazení knihoven @eduriam/ui-core a @eduriam/ui-x

Kromě nasazení frontendu bylo třeba zajistit, aby byly k dispozici pro stažení a instalaci nově vytvořené knihovny `@eduriam/ui-core` a `@eduriam/ui-x`. Knihovny jsem se rozhodl hostovat na platformě GitHub s využitím služby GitHub Packages³. Tato platforma umožňuje snadno hostovat a distribuovat knihovny sestavením balíčků přímo z GitHub repozitáře.

Nasazení nové verze probíhá automaticky při vytvoření **merge** z větve **dev** do větve **main** v repozitáři knihoven, pokud byla v rámci změn navýšena verze knihovny v souboru `package.json`. Po nasazení lze knihovny stáhnout a instalovat pomocí nástrojů **npm** nebo **yarn** podobně jako ostatní knihovny.

³Dostupné na: <https://docs.github.com/packages>

[illegible]

Testování

V této kapitole se budu zabývat testováním aplikace. Testování je klíčovou součástí vývoje aplikace a pomáhá zajistit kvalitu a stabilitu jednotlivých funkcí. V této kapitole se nejprve zaměřím na volbu vhodných forem testování s ohledem na rozsah a zaměření tohoto projektu. Dále popíšu jakým způsobem jsem prováděl manuální testování, end-to-end testování a uživatelské testování. Na závěr zhodnotím celou aplikaci a navrhnou možnosti jejího vylepšení v budoucnu.

4.1 Volba vhodných forem testování

V této sekci se budu věnovat volbě vhodných forem testování. Form testování existuje celá řada a každá forma testování je vhodná pro různé účely a projekty. Při volbě testování jsem zvažoval různé formy testování, včetně manuálního testování, automatizovaných testů a testů použitelnosti.

Jako první formu testování jsem zvolil manuální testování, které mi umožnilo v průběhu vývoje aplikace cíleně testovat části, které jsem implementoval. Způsob využití tohoto testování podrobněji popíšu v dalších částech tohoto textu.

Dále jsem se v rámci volby vhodných forem testování rozhodl zavést do aplikace automatizované testy. Automatizované testy umožní v aplikaci stanovit určitý standard kvality a stability funkcí, což je klíčové pro dlouhodobé fungování aplikace. Automatizované testy jsem se také rozhodl do aplikace zavést v souladu s návrhem úprav do budoucna původní aplikace, který jsem popsal v rámci mé bakalářské práce [2].

V rámci automatizovaných testů jsem se rozhodl zavést do aplikace automatizované end-to-end testy. Tyto testy umožní testovat funkčnost aplikace jako celku a ověřit tak, že se aplikace skutečně chová dle očekávání a specifikovaných požadavků. Tento typ automatizovaných testů jsem zvolil zejména kvůli tomu, že frontendová část aplikace cíleně neobsahuje velké množství byznys logiky a její komponenty jsou cíleně navrženy co nejjednodušším způsobem.

bem. End-to-end testy jsem tak vyhodnotil jako nejlepší přístup, protože v kombinaci se zvolenou metodikou BDD a nástrojem Cucumber popsanými v kapitole Specifikace funkcí pomocí jazyka Gherkin umožňují velice efektivně ověřit základní funkčnost všech implementovaných funkcí. S ohledem na přínos těchto testů a jejich efektivitu implementace jsem tak vyhodnotil, že tyto testy přinesou nejvyšší přínos v poměru k množství investovaného času a zdrojů.

V rámci zajištění kvality celé aplikace jsem se dále rozhodl provést testy použitelnosti. Konkrétně jsem se rozhodl provést kvalitativní uživatelské testování, které umožní ověřit, že jsou implementované funkce snadno a srozumitelně použitelné z pohledu uživatele [47]. Tomuto testování se opět budu podrobněji věnovat v následujících částech tohoto textu.

Kombinací zvolených testů tak vznikne ucelený systém testování, díky kterému bude možné zajistit vyšší kvalitu a stabilitu celé aplikace. Automatizované end-to-end testy zajistí základní funkčnost všech implementovaných funkcí a dále zajistí, že kriticky důležité funkce aplikace stále fungují před nasazením aplikace do produkčního prostředí. Manuální testování pak umožňuje se efektivně zaměřit na specifické problémy a chyby v aplikaci. Kvalitativní uživatelské testování dále umožní zajistit, že aplikace skutečně pokrývá potřeby uživatelů a že jsou funkce pro uživatele srozumitelné a snadno použitelné.

4.2 Manuální testování

Prvním prováděným testováním bylo manuální testování. Toto testování mi umožnilo v průběhu vývoje cíleně testovat části aplikace, které jsem implementoval. Testování jsem prováděl na několika úrovních.

Testování komponentů Při tvorbě knihoven aplikace @eduriam/ui-core a @eduriam/ui-x jsem každý UI komponent manuálně testoval v nástroji Storybook. V rámci testování jsem se zaměřoval zejména na testování responzivity a funkčnosti komponentů s různými parametry. Podobné testování jsem prováděl i u komponentů, které jsem implementoval přímo v kódu aplikace.

Testování funkcí Při implementaci každé funkce a po jejím dokončení jsem ji ručně testoval v aplikaci s využitím mocku API. Toto testování jsem prováděl lokálně nebo v testovacím prostředí dev.eduriam.com.

Testování funkcí s API Po tom, co kolega Bc. Jakub Vondráček dokončil v rámci jeho diplomové práce implementaci koncových bodů API týkajících se dané funkce, jsem funkci testoval v prostředí stage, které již využívalo reálné API. Po úspěšném dokončení tohoto testu pak byla funkce připravena k nasazení do produkčního prostředí.

4.3 End-to-end testování s mock API

End-to-end testování jsem prováděl pomocí nástrojů Cucumber¹ a Playwright². Cucumber je nástroj, který umožňuje specifikovat testovací scénáře pomocí jazyka Gherkin a následně testy dle scénářů automaticky spouštět. Playwright je dále framework, který umožňuje vytváření jednotlivých testů pomocí programovacího jazyka TypeScript a simulovat tak chování uživatele v aplikaci.

Díky zvoleným nástrojům jsem tak mohl v rámci sepisování případů užití specifikovat scénáře v jazyce Gherkin, které popisovaly, jak by se daná funkce měla chovat z pohledu uživatele. Tyto scénáře pak posloužily jednak jako podklad pro implementaci a jednak jako definice scénářů end-to-end testů. Díky tomu bylo možné na jednom místě přesně specifikovat, jak uživatel očekává, že se daná funkce bude chovat, a následně spustit testy, které ověří, že se tak daná funkce skutečně chová. Tento přístup tak vedl ke značnému zjednodušení tvorby end-to-end testů a k zajištění konzistence mezi požadavky definující chování aplikace, a testy, které toto chování ověřují.

Při tvorbě end-to-end testů jsem se rozhodl využívat mock API, pro simulaci reálného API. Jedná se tedy o zjednodušenou verzi end-to-end testů, protože testy ověřují chování pouze frontendové části aplikace. Pro tento přístup jsem se rozhodl z několika důvodů.

Zaprvé, díky využití mock API je možné testy vytvářet a spouštět nezávisle na stavu implementace backendové části aplikace. To mi umožnilo pracovat na testech nezávisle na stavu backendové části aplikace a vytvářet tak testy souběžně s implementací jednotlivých funkcí ve frontendové části aplikace.

Důležitým limitujícím faktorem end-to-end testů bývají jejich rychlost a náklady na spouštění, které jsou způsobeny komplexností celé aplikace [48]. Díky využití mock API jsem tak mohl značně zefektivnit a zrychlit spouštění těchto testů. Testy jsem tak mohl často spouštět lokálně v průběhu vývoje, což mi umožnilo rychleji implementovat nové funkce a ihned ověřovat, že fungují dle očekávání.

Celkem jsem tak v průběhu implementace vytvořil přes 120 testů, které mi umožnily rychle a automatizovaně ověřovat, že se frontend aplikace chová dle specifikovaných případů užití.

Testy momentálně ověřují funkčnost frontendové části aplikace. V budoucnu je však bude možné snadno napojit na reálné API upravením pomocných funkcí, které momentálně nastavují mock API prostředí. Testy pak bude možné nasadit do určitých kroků CI/CD a spouštět je tak například před nasazením do produkčního prostředí. Testy jsou tak připraveny na budoucí rozšíření, aby ověřovaly funkčnost i backendové části aplikace.

¹Dostupné na: <https://cucumber.io/>

²Dostupné na: <https://playwright.dev/>

4.4 Uživatelské testování

V rámci testování použitelnosti jsem se rozhodl provést kvalitativní uživatelské testování, při kterém budu postupovat dle doporučení Nielsen Norman Group [49]. Testování se bude skládat z následujících fází. První fází je příprava testování, v rámci které rozhodnu způsob testování, navrhnu testovací scénáře a dotazníky, a oslovím účastníky testování. V další fázi provedu samotné testování s jednotlivými účastníky. Po dokončení testování pak vyhodnotím výsledky a určím, jaké úpravy bude třeba na základě výsledků v aplikaci provést.

4.4.1 Příprava testování

Uživatelské testování lze provádět dvěma způsoby, online nebo prezenčně. V této práci jsem se rozhodl provést testování prezenčně z několika důvodů. Prezenční testování mi umožnilo se s účastníky testování osobně setkat a přímo sledovat jejich reakce při testování. Dále jsem byl, díky osobnějšimu přístupu, schopen od účastníků získat cennou zpětnou vazbu i po dokončení testování. Dalším důvodem bylo snazší nastavení testovacího prostředí. Díky tomu, že jsem se s účastníky testování setkal osobně, měl jsem plnou kontrolu nad testovacím prostředím a účastníci tak sami nemuseli stahovat ani nastavovat programy potřebné pro testování. Účastníci se tak naplno mohli soustředit na samotné testování.

V rámci přípravy testování bylo dále třeba rozhodnout, s kolika účastníky budu testování provádět. Dle doporučení Nielsen Norman Group jsem se rozhodl testování provést s 5 účastníky. S rostoucím počtem účastníků testování se překrývá jejich zpětná vazba a snižuje se tak hodnota, kterou testováním získáme. Testování s 5 účastníky je tak vhodný kompromis mezi získanou hodnotou a náklady na testování. [50]

Účastníky testování jsem vybíral na základě jejich charakteristik, které by měly odpovídat charakteristikám uživatelů, na které je aplikace zaměřena [49]. Tyto charakteristiky jsem se rozhodl ověřit úvodním dotazníkem před testováním, díky kterému jsem mohl vyřadit účastníky, kteří by nesplňovali požadované charakteristiky. Při výběru účastníků jsem se snažil cílit na osoby, kteří se věnují, nebo mají v zájmu se věnovat, programování. Mezi účastníky tak lze nalézt osoby, které se věnují programování profesionálně, ale i osoby, které takřka neprogramovaly a chtějí se pouze naučit základy programování. Přestože aplikaci mohou využívat lidé různých věkových kategorií, vybíral jsem převážně osoby v rozmezí 15 až 50 let, na které aplikace cílí.

Co se testovaných funkcí týče, rozhodl jsem se zaměřit na funkce, které jsou klíčové pro poskytnutí hodnoty uživatelům. Zaměřil jsem se tak na registraci, zapisování kurzů, proces učení, studijní plán a učení se konkrétních lekcí.

V rámci přípravy testování jsem dále sestavil dotazník po testování, který má za cíl zjistit zpětnou vazbu od účastníka testování a zjistit jeho dojmy z aplikace. Dotazník jsem podrobněji popsal v následujících částech textu.

4.4.2 Úvodní dotazník

Klíčovou součástí testování je úvodní dotazník, který má za cíl zjistit základní informace o účastníkovi testování a zhodnotit tak, zda je účastník vhodným kandidátem pro testování [47]. Vytvořil jsem proto dotazník skládající se z následujících otázek.

- Jaký je váš věk?
- Jaké jsou vaše zkušenosti s programováním? Jak dlouho se programování věnujete?
- Plánujete se programování věnovat v budoucnu? Pokud ano, plánujete se programování věnovat profesionálně nebo jako hobby?
- Jak často se v oblasti programování vzděláváte ve volném čase? Pokud se aktivně ve volném čase nevzděláváte, máte v plánu se v oblasti programování vzdělávat v budoucnu?
- Používali jste někdy aplikace na učení se programování? Pokud ano, které?

4.4.3 Testovací scénáře

V rámci přípravy testování jsem dále vytvořil následující testovací scénáře zaměřené na konkrétní funkce aplikace. Každý scénář se skládá ze zadání, které bude ukázáno účastníkovi testování, a popisu očekávaného průchodu testováním.

4.4.3.1 TS1 – Registrace a zapsání kurzu

Zadání Otevřel/a jste aplikaci na učení se programování. Založte si v aplikaci účet, vyplňte úvodní dotazník a запиšte si kurz *HTML*. Při registraci použijte uživatelské jméno *Test*, email *test@example.com* a heslo *Test1234*.

Očekávaný průchod aplikací

1. Uživatel začíná na úvodní stránce aplikace.
2. Uživatel se z úvodní stránky přesune na stránku registrace, vyplní registrační formulář a zvolí možnost zaregistrovat se. Aplikace přesune uživatele na stránku úvodního dotazníku.
3. V dotazníku uživatel vyplní svoji úroveň programování, témata, která se chce učit, a cíl, kterého chce dosáhnout.
4. Následně uživatel vybere jeden z doporučených kurzů.
5. Nakonec uživatel zvolí svůj denní cíl učení.

6. Aplikace oznámí uživateli, že je jeho účet připraven a uživatel zvolí možnost pokračovat, čímž se přesune na hlavní stránku aplikace.

4.4.3.2 TS2 – Učení se dle studijního plánu

Zadání Otevřel/a jste aplikaci a chcete se začít učit vámi zapsaný kurz. Spustíte nadcházející lekci kurzu a splňte v ní všechna cvičení.

Očekávaný průchod aplikací

1. Uživatel začíná na hlavní stránce aplikace.
2. Uživatel zvolí možnost spustit nadcházející lekci kurzu.
3. Aplikace přesune uživatele na stránku učení, kde mu zobrazí látku nadcházející lekce.
4. Uživatel splní všechna cvičení v lekci a aplikace mu oznámí, že lekce byla dokončena. Po dokončení učení aplikace nabídne uživateli možnost spustit opakování.
5. Uživatel zvolí možnost spustit opakování.
6. Aplikace přesune uživatele na stránku opakování, kde mu zobrazí látku k zopakování.
7. Uživatel splní všechna cvičení v rámci opakování a aplikace mu oznámí, že bylo opakování dokončeno.
8. Uživatel zvolí možnost pokračovat, čímž bude přesměrován na hlavní stránku aplikace.

4.4.3.3 TS3 – Zapsání nového kurzu

Zadání Chcete si zapsat nový kurz *CSS*. Zapište si tento kurz a spustíte v něm první lekci. Tuto lekci dokončete.

Očekávaný průchod aplikací

1. Uživatel začíná na hlavní stránce aplikace.
2. Uživatel se pomocí hlavní nabídky přesune na stránku výběru kurzů.
3. Na stránce výběru kurzů uživatel zvolí kurz *CSS*, čímž se přesune na stránku kurzu.
4. Na stránce kurzu pak uživatel zvolí možnost začít kurz, čímž mu bude kurz zapsán a uživatel bude přesměrován na stránku učení první lekce kurzu.

5. Uživatel splní všechna cvičení v lekci a aplikace mu oznámí, že lekce byla dokončena.
6. Uživatel zvolí možnost pokračovat, čímž bude přesměrován na hlavní stránku aplikace.

4.4.3.4 TS4 – Úprava studijního plánu

Zadání Máte zapsané kurzy *HTML* a *CSS*, které jste se nějakou dobu učili. Nyní se chcete zaměřit pouze na kurz *CSS* a z kurzu *HTML* se již nechcete učit nové lekce. Zařídte, aby vám aplikace na hlavní stránce již nezobrazovala nové lekce z kurzu *HTML*, ale abyste nepřišli o váš pokrok v kurzu *HTML*.

Očekávaný průchod aplikací

1. Uživatel začíná na hlavní stránce aplikace.
2. Uživatel zvolí možnost úpravy studijního plánu, čímž se přesune na stránku studijního plánu.
3. Na stránce studijního plánu uživatel přesune kurz *HTML* ze sekce Učení a opakování do sekce Opakování. Aplikace mu tak nebude zobrazovat nové lekce z kurzu *HTML*, ale stále mu bude zobrazovat látku z kurzu *HTML* při opakování.

4.4.3.5 TS5 – Spuštění konkrétní lekce

Zadání V rámci kurzu *HTML* se chcete naučit látku z lekce *Obrázky*. Spusťte tuto konkrétní lekci. Lekci nemusíte dokončovat.

Očekávaný průchod aplikací

1. Uživatel začíná na hlavní stránce aplikace.
2. Uživatel se pomocí hlavní nabídky přesune na stránku svého profilu, kde uvidí seznam zapsaných kurzů.
3. V seznamu kurzů uživatel zvolí kurz *HTML*, čímž se přesune na stránku kurzu.
4. Na stránce kurzu uživatel vybere první sekci kurzu, čímž se přesune na stránku sekce kurzu.
5. Na stránce sekce kurzu uživatel najde lekci *Obrázky* a zvolí možnost spuštění lekce.

4.4.4 Dotazník po testování

Dalším dotazníkem, který jsem v rámci přípravy testování vytvořil, byl dotazník po testování. Tento dotazník má za cíl získat zpětnou vazbu od účastníka testování, identifikovat problematická místa a získat podněty pro další zlepšení aplikace. Dotazník jsem sestavil z následujících otázek.

- Co se vám na aplikaci líbilo?
- Který z testovacích úkolů pro vás byl nejobtížnější a proč?
- Co vám v aplikaci vadilo? Co vás překvapilo nebo zmátlo?
- Máte pocit, že víte, kde co hledat v aplikaci? Proč?
- Co byste v aplikaci změnil/a nebo doplnil/a?
- Máte nějaké další připomínky nebo návrhy?

4.4.5 Průběh testování

Testování jsem prováděl v testovacím prostředí `dev.eduriam.com` popsaném v kapitole Nasazení aplikace. Díky tomu, že byla aplikace nasazená v tomto testovacím prostředí, bylo k ní možné snadno přistupovat skrze webový prohlížeč a nebylo třeba instalovat žádné další programy pro její spuštění.

Při testování jsem každému účastníkovi poskytl mobilní zařízení Samsung S24, na kterém následně testování probíhalo. Při testování jsem nahrával zvuk a obrazovku³ pomocí nativní aplikace Záznam obrazovky, která je výchozí aplikací pro záznam obrazovky na zařízeních Samsung. Tuto aplikaci jsem zvolil převážně z důvodu, že byla při nahrávání obrazovky nejspolehlivější ze všech aplikací, které jsem vyzkoušel.

Testování probíhalo následujícím způsobem. Nejprve jsem účastníkovi testování vysvětlil, jakým způsobem bude testování probíhat a zeptal jsem se ho, zda souhlasí s nahráváním zvuku a obrazovky. Poté účastník odpověděl na otázky z úvodního dotazníku, jehož výsledky jsou dostupné v příloze ???. Po dokončení úvodního dotazníku začalo samotné testování aplikace. Nejprve jsem účastníkovi vysvětlil, jak bude testování probíhat a umožnil jsem mu se zeptat na případné otázky. Poté jsem poskytl účastníkovi zadání úkolů, které měl vykonat. Účastník pak postupně vykonával všechny úkoly. Po vykonání každého úkolu měl účastník sám oznámit, že úkol dokončil, abych předešel předčasnému ukončení testu, pokud si uživatel například nebyl jistý, jestli úkol skutečně dokončil. Mezi jednotlivými úkoly jsem vždy nastavil aplikaci tak, aby uživatel začínal na správné stránce dle testovacího scénáře. Po dokončení všech úkolů jsem se účastníka zeptal na otázky z dotazníku po testování. Výsledky tohoto dotazníku jsou k dispozici v příloze ???.

³Dostupné na: <https://www.example.com>

Po dokončení testování jsem dále cíleně vedl neformální konverzaci s účastníkem testování o aplikaci, díky které jsem získal další cennou zpětnou vazbu ohledně celkového dojmu z aplikace a návrhů na její zlepšení.

4.4.6 Výsledky testování

4.5 Zhodnocení řešení a návrh úprav do budoucna

4.5.1 Zhodnocení řešení

4.5.2 Návrh úprav do budoucna

Závěr

..... Příloha A

Nějaká příloha

Sem přijde to, co nepatří do hlavní části.

Bibliografie

1. HLAVÁČ, Jan. *Backend webové aplikace na učení se angličtiny* [online]. 2024. [cit. 2026-01-14]. Dostupné z: <https://dspace.cvut.cz/entities/publication/ecf8a20f-9558-45d3-b91d-8ea9affc8d09>. Bakalářská práce. České vysoké učení technické, Fakulta informačních technologií.
2. JENÍČEK, Jan. *Frontend webové aplikace na učení se angličtiny* [online]. 2024. [cit. 2026-01-14]. Dostupné z: <https://dspace.cvut.cz/entities/publication/0b0498b3-d721-4ee3-9f61-83da8c90de4f>. Bakalářská práce. České vysoké učení technické, Fakulta informačních technologií.
3. MIMO. *Mimo* [online]. [cit. 2026-01-14]. Dostupné z: <https://mimo.org/>.
4. DETERDING, Sebastian; DIXON, Dan; KHALED, Rilla; NACKE, Lenart. From game design elements to gamefulness: defining “gamification”. In: *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*. Tampere, Finland: Association for Computing Machinery, 2011, s. 9–15. MindTrek '11. ISBN 9781450308168. Dostupné z DOI: 10.1145/2181037.2181040.
5. CODECADEMY. *Catalog* [online]. [cit. 2026-01-14]. Dostupné z: <https://www.codecademy.com/catalog>.
6. CODECADEMY. *Partnerships* [online]. [cit. 2026-01-14]. Dostupné z: <https://www.codecademy.com/pages/partnerships>.
7. SOLOLEARN. *Sololearn* [online]. [cit. 2026-01-14]. Dostupné z: <https://www.sololearn.com/>.
8. GOOGLE. *Applying density* [online]. [cit. 2026-01-14]. Dostupné z: <https://m2.material.io/design/layout/applying-density.html>.
9. MUI. *Material UI* [online]. [cit. 2026-01-14]. Dostupné z: <https://mui.com/material-ui/>.
10. SCRIMBA. *Courses* [online]. [cit. 2026-01-14]. Dostupné z: <https://scrimba.com/courses>.

11. GOOGLE. *System font or web-safe font* [online]. [cit. 2026-01-14]. Dostupné z: https://fonts.google.com/knowledge/glossary/system_font_web_safe_font.
12. HUDAIB, Amjad; MASADEH, Raja; HAJ QASEM, Mais; ALZAQEBAH, Abdullah. Requirements Prioritization Techniques Comparison. *Modern Applied Science*. 2018, roč. 12, č. 2, s. 64–65. ISSN 1913-1844. Dostupné z DOI: 10.5539/mas.v12n2p62.
13. MLEJNEK, Jiří. *Analýza a sběr požadavků* [online]. 2026. [cit. 2026-02-01]. Dostupné z: https://moodle-vyuka.cvut.cz/pluginfile.php/819387/mod_resource/content/7/03.prednaska.pdf.
14. TIDWELL, Jenifer; BREWER, Charles; VALENCIA, Aynne. *Designing Interfaces: Patterns for Effective Interaction Design*. 3. vyd. Sebastopol, CA: O'Reilly Media, Inc., 2020. ISBN 978-1-492-05196-1.
15. CHOU, Yu-kai. *Actionable Gamification: Beyond Points, Badges, and Leaderboards*. 1. vyd. Packt Publishing, 2019. ISBN 9781839210778.
16. BIRK, Max V.; MANDRYK, Regan L. Combating Attrition in Digital Self-Improvement Programs using Avatar Customization. In: *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. Montreal QC, Canada: Association for Computing Machinery, 2018, s. 1–15. CHI '18. ISBN 9781450356206. Dostupné z DOI: 10.1145/3173574.3174234.
17. SWAGGER. *OpenAPI Specification* [online]. [cit. 2026-01-14]. Dostupné z: <https://swagger.io/specification/>.
18. THE CUCUMBER OPEN SOURCE PROJECT. *Behaviour-Driven Development* [online]. 2025. [cit. 2026-02-01]. Dostupné z: <https://cucumber.io/docs/bdd/>.
19. THE CUCUMBER OPEN SOURCE PROJECT. *Introduction* [online]. 2026. [cit. 2026-02-01]. Dostupné z: <https://cucumber.io/docs/>.
20. COCKBURN, Alistair. *Writing Effective Use Cases*. 1. vyd. Addison-Wesley Professional, 2000. ISBN 9780321605795.
21. PAVLÍČEK, Josef. *UI design steps* [online]. 2025. [cit. 2026-01-29]. Dostupné z: <https://docs.google.com/presentation/d/1ClSHyC8D8cN2LGrqUVJ2U5eEKn5z9MpkFPmzwZX4Zc>.
22. THE CUCUMBER OPEN SOURCE PROJECT. *History of BDD* [online]. 2025. [cit. 2026-02-01]. Dostupné z: <https://cucumber.io/docs/bdd/history>.
23. GOOGLE. *Material Design 2* [online]. [cit. 2026-01-21]. Dostupné z: <https://m2.material.io/>.

24. HARTSON, Rex; PYLA, Pardha. *The UX Book: Agile UX Design for a Quality User Experience*. 2. vyd. Cambridge, MA: Morgan Kaufmann, 2019. ISBN 978-0-12-805342-3.
25. W3. *Web Content Accessibility Guidelines (WCAG) 2.2* [online]. 2024. [cit. 2026-01-22]. Dostupné z: <https://www.w3.org/TR/WCAG22>.
26. GOOGLE. *The color system* [online]. [cit. 2026-01-21]. Dostupné z: <https://m2.material.io/design/color/the-color-system.html>.
27. PAVLÍČEK, Josef. *Colors* [online]. 2025. [cit. 2026-01-21]. Dostupné z: https://docs.google.com/presentation/d/1l_T7H8KyT43RTaGzcsLXo4t1w08s30aD6yLaMZZ1InQ.
28. FIGMA. *Light blue* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://www.figma.com/resource-library/what-is-color-theory/>.
29. FIGMA. *What is color theory?* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://www.figma.com/resource-library/what-is-color-theory/>.
30. FIGMA. *Color Wheel* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://www.figma.com/color-wheel/>.
31. MATERIAL UI. *Palette* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://mui.com/material-ui/customization/palette/>.
32. GOOGLE. *Dark theme* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://m2.material.io/design/color/dark-theme.html>.
33. ANDERSSON, Rasmus. *The Inter typeface family* [online]. 2026. [cit. 2026-01-23]. Dostupné z: <https://rsms.me/inter/>.
34. JETBRAINS. *JetBrains Mono* [online]. 2026. [cit. 2026-01-28]. Dostupné z: <https://www.jetbrains.com/lp/mono/>.
35. ANAND, Surya. *Typographic Scales* [online]. 2025. [cit. 2026-01-28]. Dostupné z: <https://designcode.io/typographic-scales>.
36. GOOGLE. *The type system* [online]. 2026. [cit. 2026-01-23]. Dostupné z: <https://m2.material.io/design/typography/the-type-system.html>.
37. GOOGLE. *Applying sound to UI* [online]. 2026. [cit. 2026-01-23]. Dostupné z: <https://m2.material.io/design/sound/applying-sound-to-ui.html>.
38. FIGMA. *Guide to variables in Figma* [online]. 2026. [cit. 2026-01-29]. Dostupné z: <https://help.figma.com/hc/en-us/articles/15339657135383-Guide-to-variables-in-Figma>.
39. FIGMA. *Guide to auto layout* [online]. 2026. [cit. 2026-01-29]. Dostupné z: <https://help.figma.com/hc/en-us/articles/360040451373-Guide-to-auto-layout>.

40. FIGMA. *Modes for variables* [online]. 2026. [cit. 2026-01-29]. Dostupné z: <https://help.figma.com/hc/en-us/articles/15343816063383-Modes-for-variables>.
41. TALARICO, Donna. What is mobile-first design? Why does it matter? *Recruiting & Retaining Adult Learners*. 2019, roč. 22, č. 2, s. 3–3. ISSN 2155-644X. Dostupné z DOI: <https://doi.org/10.1002/nsr.30533>.
42. PAVLÍČEK, Josef. *Responsive design* [online]. 2025. [cit. 2026-01-29]. Dostupné z: <https://docs.google.com/presentation/d/1IAILjefRAxMu9vQ3qWQzDgaQCqWYbVLnc77j9cqz1XU>.
43. NIELSEN, Jakob. Enhancing the explanatory power of usability heuristics. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Boston, Massachusetts, USA: Association for Computing Machinery, 1994, s. 152–158. CHI '94. ISBN 0897916506. Dostupné z DOI: 10.1145/191666.191729.
44. PRESTON-WERNER, Tom. *Semantic Versioning 2.0.0* [online]. 2026. [cit. 2026-03-23]. Dostupné z: <https://semver.org/>.
45. GOOGLE. *Service workers* [online]. 2021. [cit. 2026-03-23]. Dostupné z: <https://web.dev/learn/pwa/service-workers>.
46. PWABUILDER. *Helping developers build and publish PWAs* [online]. 2026. [cit. 2026-03-23]. Dostupné z: <https://www.pwabuilder.com/>.
47. PAVLÍČEK, Josef. *User Interface Testing* [online]. 2025. [cit. 2026-03-31]. Dostupné z: <https://docs.google.com/presentation/d/1t-4kCvHJSqpzqff30JhoAzPvaJQQElJ990geai3K9e8>.
48. IBM. *What is End-to-End (E2E) Testing?* [online]. [cit. 2026-03-31]. Dostupné z: <https://www.ibm.com/think/topics/end-to-end-testing>.
49. MORAN, Kate. *Usability Testing 101* [online]. 2019-12-01. [cit. 2026-03-31]. Dostupné z: <https://www.nngroup.com/articles/usability-testing-101/>.
50. NIELSEN, Jakob. *Why You Only Need to Test with 5 Users* [online]. 2000-03-18. [cit. 2026-03-31]. Dostupné z: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>.

Obsah příloh

/	
└─ readme.txt.....	stručný popis obsahu média
└─ exe.....	adresář se spustitelnou formou implementace
└─ src	
└─ impl.....	zdrojové kódy implementace
└─ thesis.....	zdrojová forma práce ve formátu $\text{\LaTeX}$
└─ text.....	text práce
└─ thesis.pdf.....	text práce ve formátu PDF